LONDON METROPOLITAN UNIVERSITY

CS6P05 Project

Stateful vs. Stateless Defence: A Comparative Analysis of DoS Mitigation Strategies in Virtualised Enterprise Networks

Interim Submission

Name: Valerio Alberto Novas Casilla

D Number: 21049834

Date: 05/05/2026 Supervisor: Maitreyee Dey

Declaration

Module: CS6P05 **Deadline:** 3pm Tuesday 5th May 2026

Module Leader: Dr. Elaheh Homayounvala **Student ID:** 21049834

PLAGIARISM

You are reminded that there exist regulations concerning plagiarism. Extracts from these regulations are printed below. Please sign below to say that you have read and understand these extracts:

Student signature: Valerio Alberto Novas Casilla Date: 05/05/26

This header sheet should be attached to the work you submit. No work will be accepted without it.

Extracts from University *Regulations* on Cheating, Plagiarism and Collusion

Section 2.3: "The following broad types of offence can be identified and are provided as indicative examples…

(i) Cheating: including taking unauthorised material into an examination; consulting unauthorised material outside the examination hall during the examination; obtaining an unseen examination paper in advance of the examination; copying from another examinee; using an unauthorised calculator during the examination or storing unauthorised material in the memory of a programmable calculator which is taken into the examination; copying coursework.

(ii) Falsifying data in experimental results.

(iii) Personation, where a substitute takes an examination or test on behalf of the candidate. Both candidate and substitute may be guilty of an offence under these Regulations.

(iv) Bribery or attempted bribery of a person thought to have some influence on the candidate's assessment.

(v) Collusion to present joint work as the work solely of one individual.

(vi) Plagiarism, where the work or ideas of another are presented as the candidate's own.

(vii) Other conduct calculated to secure an advantage on assessment.

(viii) Assisting in any of the above.

Some notes on what this means for students:

Copying another student's work is an offence, whether from a copy on paper or from a computer file, and in whatever form the intellectual property being copied takes, including text, mathematical notation and computer programs.

Taking extracts from published sources *without attribution* is an offence. To quote ideas, sometimes using extracts, is generally to be encouraged. Quoting ideas is achieved by stating an author's argument and attributing it, perhaps by quoting, immediately in the text, his or her name and year of publication, e.g. "$e = mc^2$ (Einstein 1905)". A *references* section at the end of your work should then list all such references in alphabetical order of authors' surnames. (There are variations on this referencing system which your tutors may prefer you to use.) If you wish to quote a paragraph

Table of Content

Declaration 1
Table of Content 3
Table of Figures 6
List of Tables 9
Abstract 10
Chapter 1: Introduction 11
Chapter 2: Background Research 12
2.1 Literature Review of Related Work 12
2.2 Critical Evaluation of Related Products/Solutions 13
2.2.1 Identification of the Research Gaps 14
2.3 The Scope of the Project 15
2.3.1 Functional Scope 15
2.3.2 Environmental Scope 15
2.4 Review and Justification of Technical Tools and Platform 16
Chapter 3: Problem Analysis and Specification 20
3.1 Problem Statement and Context 20
3.2 Aims and Second-Level Objectives 20
3.3 Definition of Empirical Performance Metrics 21
3.4 Research Questions and Core Hypothesis 22
Chapter 4: Research and Software Methodology Design 24
4.1 The Research Onion Framework 24
4.2 Methodological Data Selection: Qualitative and Secondary Data Analysis 25
4.3 Software Methodology 25
4.3.1 Scrum Ceremonies and Artefacts 26
4.4 Network Architectural Design 28
4.4.1 Hierarchical Three-Tier Architecture with Centralised Core Routing 28
4.4.2 Technical Resources Requirements 32
Chapter 5: Technical Implementation and Testing 33
5.1. Server Purchased Process 33
5.2. Connect to the Rescue System and Install the Operating System 35
5.2.1 Connecting to “Rescue System 35
5.2.2 Install Linux Ubuntu as the server OS 36
5.3. Tailscale Deployment on the Server 38
5.3.1 Binding the Server to the Tailscale Network 38

5.3.2 Enabling SSH and Testing Connectivity 40
5.4 Installing GNS3 on the Server 41
5.4.1 Configuration Secure Access to the GNS3 Server via Tailscale 42
5.5 Downloading and Installing GNS3 GUI on the Local Machine 43
5.5.1 Configuring the GNS3 Client for Remote Server Use 46
5.6 Installing Tailscale on the Laptop 48
5.6.1 Local Machine Integration and Tailscale Authentication 49
5.7 Creating the Invisible Server Strategy 50
5.7.1 Testing Server Security Invisibility Strategy 52
5.7.2 Testing SSH Session From Tailscale Dashboard 53
5.8 Uploading, Configuring Network Devices and VMs Images to GNS3 for Network Deployment 54
5.8.1 Importing Core Router Image 54
5.8.2 Creating the Core Router Qemu Template 55
5.8.3 Selecting the Cisco IOS .qcow File from the Directory 55
5.8.4 Configuring Name and selecting Qemu Binary 56
5.8.5 Ram Allocation and Selecting Telnet as Device Connection Type 57
5.8.6 Creating the Qemu Template for the Layer-2 Switch 58
5.8.7 Importing Ubuntu-Server 58
5.8.8 Creating a Qemu Template for Kali Linux 60
5.9 Network Topology Technical Configuration 60
5.9.1 Phase 1: Environment Orchestration 61
5.9.2 Phase 2: Network Configuration 62
5.9.2.1 Core-Switch Configuration 63
5.9.2.2 DS-1 and DS-2 Configuration 64
5.9.2.3 Access-Switch Configuration 65
5.9.2.4 Empirical Verification of STP on the Switches 67
5.9.2.5 Edge-Router Configuration 69
5.9.2.6 Empirical Routing Verification 70
5.9.2.7 Endpoint Provisioning and Connectivity Validation 71
5.9.2.8 Empirical End-to-End Baseline Benchmarking 73
5.9.7 Phase 3: Attack Execution 75
5.9.3.1 Active ICMP Flood Attack Emulation (Unmitigated State) 75
5.9.3.1 Active SYN Flood Attack Emulation Unmitigated State) 77
5.9.3.2 Empirical Results and Analysis 79
5.9.4 Phase 4: Mitigation Interaction 81

5.9.4.1 ZBF Defensive Countermeasures (Mitigated State) 81
5.9.4.2 ZBF Empirical Result and Analysis 83
5.9.4.3 Control Plane Mitigation Strategy (CoPP) 85
5.9.4.4 CoPP Empirical Result and Analysis 86
Chapter 6: Discussion and Analysis 88
6.1 Architectural Dichotomy in the Unmitigated State 88
6.2 The Efficacy and Overhead of Stateful Mitigation (ZBF) 88
6.3 The Efficiency of Stateless Mitigation (CoPP) 89
6.4 Comparative Synthesis: Security vs. Performance Trade-off 90
Chapter 7: Conclusion 91
7.1 Summary of the Development to Date 91
7.2 Reflections and Lessons Learned 91
7.3 Legal, Social, Ethical, and Professional Issues (LSEP) 92
6.3 Future Work 92
Appendices 94
Appendix 1: Project Management 94
1.1 Project Plan 94
Appendix 2: Practical 98
2.1 Amendments to the Original Plan 98
2.2 Practical Configuration Commands 99
References 106

Table of Figures

Figure 1 - Agile Software Methodology (Stigasoft, 2025) 26
Figure 2 - Project's Agile Network Development Methodology 28
Figure 3 - Hierarchical Three-Tier Architecture with Centralised Core Topology 29
Figure 4 - Choosing the Ideal Dedicated Server 33
Figure 5 - Completing the Server Order and Choosing Access Method 33
Figure 6 -Hetzner Robot Server Dashboard 34
Figure 7 - Dedicated Server's Login Details and IP Addresses 34
Figure 8 - The Server is Up and Running 35
Figure 9 - Successfully logged in to the Rescue System 36
Figure 10 - Selecting Ubuntu 22.04 as OS 36
Figure 11 - Adding Parameters to the Installimage Configuration File 37
Figure 12 - OS installation Completed & Rebooting the System 37
Figure 13 - Installing Tailscale on the Server 38
Figure 14 - Tailscale Requiring Authentication 38
Figure 15 - Tailscale Log In website 38
Figure 16 - Selecting Email Account for the Server's 39
Figure 17 - The Server is Successfully Connected to Tailscale Network 39
Figure 18 - Enabling an SSH Session on the Server for Tailscale 40
Figure 19 - SSH is Enabled and Ready to Establish Connection 40
Figure 20 - Establishing SSH Session Using Private Tailscale IP Address 40
Figure 21 - Initiating GSN3 Installation on the Server 41
Figure 22 - GNS3 has been Installed Successfully on the Server. 41
Figure 23 - Original Nano Text Editor's GNS3 Configuration File 42
Figure 24 - Updated Nano Text Editor's GNS3 Configuration File 42
Figure 25 - Confirming GNS3 Software Version 43
Figure 26 - Downloading GNS3 GUI to the Local Machine (Laptop) 43
Figure 27 - GNS3 GUI Installation License Agreement 44
Figure 28- Opening the Download Folder and 44
Figure 29 - GNS3 GUI's Component Tool Installation 44
Figure 30 - GNS3 GUI's Installation Start Menu Folder 44
Figure 31 - GNS3 GUI's Install Folder Selection 45
Figure 32 - GNS3 GUI & Component Tools Installation Process 46
Figure 33 - GNS3 GUI installation completed. 46
Figure 34 - GNS3 GUI's Wizard Set Up 47
Figure 35 - GNS3 GUI's Credential, Host and Port 47

Figure 36 - Server Summary 47
Figure 37 - Downloading Tailscale for the Local Machine 48
Figure 38 - Tailscale Installation Process on the Local Machine 48
Figure 39 - Sign in to Tailscale Network 49
Figure 40 - Get Tailscale Authentication Started 49
Figure 41 - Authenticating the Local Machine to Tailscale Network 49
Figure 42 - Selecting Email Account for Local Machine's Tailscale Authentication 49
Figure 43 - Server and Local Machine Added to Tailscale Network 50
Figure 44 - Hetzner Server Dashboard 50
Figure 45 - Weak Server's Firewall Rules 51
Figure 46 - Invisible Server Firewall's Rules 52
Figure 47 - Testing Public & Private IP for SSH Session 52
Figure 48 - SSH the Server from the Tailscale Dashboard 53
Figure 49 - Selecting User as Root to Start SSH Session 53
Figure 50 - Starting Authentication for SSH 53
Figure 51 - Selecting Account for SSH Authentication 53
Figure 52 - SSH Session Establishes through the Tailscale Dashboard 54
Figure 53 - GNS3 Preferences Window Showing Navigation to Qemu VMs 55
Figure 54 -Router Image Status Bar Progress 55
Figure 55 - Selecting Router Image From the Directory 55
Figure 56 - Providing a Name to the New Router Template 56
Figure 57 - Selecting the Standard x86_86(v6.2.0) as Qemu Binary 56
Figure 58 - Allocating the Router's RAM and Telnet as Connection Type 57
Figure 59 - Router Qemu Template for Successfully Created 57
Figure 60 - Switch Qemu Template for Successfully Created 58
Figure 61 - Ubuntu-Server Template Created 59
Figure 62 - - Linux Ubuntu-Server Node is Ready for Deployment 59
Figure 63 - GNS3 Preference Menu for Kali Linux Shows Specification 60
Figure 64 - Building the Hierarchical Three-Tier Architecture with Centralised Core 61
Figure 65 - Completed GNS3 Network Topology with All Nodes in Operational State 62
Figure 66 - Core-Switch Configuration Commands 63
Figure 67 - DS-1 Cisco Configuration Commands 64
Figure 68 - DS-2 Cisco Configuration Commands 65
Figure 69 - Access-Switch Cisco Configuration Commands 66
Figure 70 – Core-Switch STP Configuration Verification 67
Figure 71 - Access-Switch 'show spanning-tree' Verification command. 68

Figure 72 - Core-Router Configuration Commands 69
Figure 73 - 'show ip interface brief' and 'show ip route' Verification Commands from Edge-Router 70
Figure 74 - Assigning the IP Address and Default Gateway to the Attacker-Kali Node 71
Figure 75 - Assigning IP Address and Default Gateway to the User-Node 72
Figure 76 - Assigning IP Address and Default Gateway to the Target-Server 72
Figure 77 - Baseline ICMP Latency Results (ping command) 73
Figure 78 - Baseline TCP Throughput Results (Iperf3) 74
Figure 79 - Edge-Router Control Plane Utilisation During Volumetric ICMP Flood 76
Figure 80 - Volumetric ICMP Flood Attack Execution 77
Figure 81 - Iperf3 Throughput Degradation During Unmitigated SYN Flood 79
Figure 82 - Edge-Router Queue Drops and CPU Utilisation 80
Figure 83 - Figure 5.z: Edge-Router ZBF Zone Assignment Configuration 82
Figure 84 - Iperf3 Throughput During ZBF-Mitigated SYN Flood 83
Figure 85 - Edge-Router CPU Utilisation During ZBF Mitigation 84
Figure 86 - Deploying CoPP Policy on Edge-Router 85
Figure 87 - Relaunching ICMP Flood Attack From Kali-Linux Attacker Node 86
Figure 88 - CoPP Policy-Map Statistics During Volumetric ICMP Flood 86
Figure 89 - Actual Project Work Breakdown Structure 94
Figure 90 - Actual Project Gantt Chart Diagram 97

List of Tables

Table 1 - Second-Level Objectives 21
Table 2 - Quantitative Metrics Tools 21
Table 3 - IP Addressing Table 31
Table 4 - Technical Resources Requirements for the Virtual Lab. 32
Table 5 - GNS3 GUI's Installation Component Tools 44
Table 6 - ICMP Attack Commands Execution Process 76
Table 7 - SYN Flood Attack Execution Process 78
Table 8 - The Actual Project WBS Activities 94

Abstract

In today's highly connected world, Denial-of-Service (DoS) attacks continue to pose a serious threat to service availability. These attacks deliberately overwhelm networks and systems by exhausting bandwidth and computing resources. Although the mechanics of such attacks are well known, there is still limited empirical data on the real computational cost of defending against them using built-in network hardware features.

This project addresses that gap. It presents a quantitative comparison of two powerful Cisco IOS-XE defence mechanisms: stateless Control Plane Policing (CoPP) and stateful Zone-Based Firewalls (ZBF).

To ensure accurate results free from local hardware limitations, a high-fidelity enterprise network was built on a dedicated bare-metal cloud server using GNS3. The entire environment was secured with a Zero-Trust Tailscale overlay network. This virtual infrastructure faced realistic volumetric attacks, including ICMP floods against the Control Plane and TCP SYN floods targeting the Data Plane. Through iterative Agile testing, the study measured baseline performance, degradation under unmitigated attacks, and the resource overhead introduced by each mitigation strategy.

The results reveal a clear architectural trade-off. The stateful Zone-Based Firewall effectively protected application traffic and prevented queue exhaustion, but it introduced a measurable increase in CPU utilisation. In contrast, the stateless CoPP policy delivered strong protection for the Control Plane with almost zero additional overhead.

Ultimately, this research offers network engineers practical, reproducible evidence that a layered "defence-in-depth" approach is essential. By combining the efficiency of CoPP with the deeper inspection capabilities of ZBF, organisations can build far more resilient networks capable of withstanding modern volumetric attacks while keeping performance impact to a minimum.

Chapter 1: Introduction

As digital infrastructure becomes the backbone of modern society, Denial-of-Service (DoS) attacks continue to pose a serious and growing threat to service availability. Although enterprise routing platforms like Cisco IOS-XE include powerful built-in defences, network architects still lack clear empirical data on the real computational cost of using them (Zargar et al., 2013).

This project investigates the vulnerabilities of a Collapsed Core network architecture when placed under volumetric and state-exhaustion attacks. Its primary goal is to evaluate the performance trade-offs involved in layered DoS mitigation strategies. Specifically, the research set out to build a high-fidelity virtualised enterprise network, establish baseline latency and throughput benchmarks, emulate unmitigated TCP SYN and ICMP floods to identify saturation points (Harshita, 2017; Khalaf et al., 2019), and deploy both stateless Control Plane Policing (CoPP) and stateful Zone-Based Firewalls (ZBF) to measure their defensive effectiveness and associated CPU overhead.

To maintain scientific rigour, the study followed a positivist philosophy and a deductive approach guided by the Research Onion framework (Saunders *et al.,* 2009; Alturki, 2021). An Agile Scrum methodology was used to manage the technical work, allowing rapid adjustments to the cloud-native GNS3 environment during active attack emulation sprints (Sachdeva, 2016; Singh, 2008).

The remainder of this report follows the natural flow of the research lifecycle. Chapter 2: reviews relevant literature and core defence mechanisms. Chapter 3: defines the problem statement and key performance metrics. Chapter 4: outlines the research methodology and architectural design. Chapter 5 documents the detailed lab implementation, attack execution, and mitigation deployment. Chapter 5: presents a comparative analysis of the empirical results, and Chapter 7 concludes the study with recommendations for future research.

Chapter 2: Background Research

This chapter addresses the foundational requirement of performing a critical literature review to demonstrate a specialist understanding of the problem and validate the necessity of the proposed empirical study. It translates the general threat of Denial-of-Service (DoS) attacks into specific, measurable architectural vulnerabilities, justifying the choice of our comparative methodology and simulation platform (Dawson 2015; Jalloul 2012).

2.1 Literature Review of Related Work

The Denial-of-Service (DoS) and its distributed version, Distributed Denial-of-Service (DDoS), continue to be complex and common threats in today's online environment (Peng et al., 2007; Zargar et al., 2013). Unlike straightforward data theft, DoS attacks aim to deliberately drain the victim's computing or network resources. This action prevents legitimate users from accessing services (AL-Musawi, 2012; Palo Alto Networks, 2019).

The scale of the problem has only increased. Recent reports show that DDoS attacks rose by 55% from early 2020 to early 2021 (Shroff et al., 2022). The significant financial and operational risks from these attacks have driven the growth of the DDoS protection market. It is projected to reach $7.3 billion by 2027 (Markets&Markets, 2022). For network designers, the challenge is twofold. They need to filter large-scale volumetric floods that jam bandwidth. They also have to defend against sophisticated attacks that exploit protocol weaknesses intended to consume device-level resources and deny legitimate users access to services (AL-Musawi, 2012; Palo Alto Networks, 2015).

To address this, our investigation focuses on two typical protocol attacks that target different functional planes of a network device:

SYN Flood Attacks (Targeting Session State): The SYN flood is a classic and effective technique. It exploits a session-state weakness in the Transmission Control Protocol (TCP) three-way handshake (Bogdanoski et al., 2013; Eddy, 2006). By sending many SYN requests with fake source addresses, the attacker forces the target server to use resources and keep an entry in its connection backlog queue in a "half-open" state. This state is left waiting for a final ACK that will never come (Bekravi et al., 2012; Jamali et al., 2012). This rapid use of resources quickly fills the limited queue size, denying legitimate connection requests (Safa et al., 2008). Traditional host-based defences, such as SYN Cookies or SYN Cache, try to manage this

resource use at the server level, but they often fall short against modern distributed attacks (Goldschmidt et al., 2021). Consequently, the task falls to upstream network devices, such as firewalls, which must perform stateful inspection to approve session legitimacy before traffic reaches the server (AL-Musawi 2012; Goldschmidt et al., 2021).

ICMP Flood Attacks (Targeting Device CPU): Unlike the SYN flood, which focuses on session state, the ICMP flood is a purely volumetric attack. It aims to absorb the raw computing power of the target device (Harshita, 2017). When a high quantity of ICMP echo request packets is sent to a router's local interface address, the device's central processing unit (CPU) is forced to allocate processing cycles to manage and respond to the constant flood. This competition for resources can starve critical system processes, culminating in Control Plane failure, routing issues, and administrative inaccessibility (Liu et al., 2023). Recent studies on critical systems, such as Autonomous Driving infrastructure, have revealed that even fairly resilient embedded operating systems can experience serious resource exhaustion. This confirms that external hardware interfaces are constant attack points vulnerable to basic ICMP flooding (Stübler et al., 2024). This attack is a key test for any system designed for device self-preservation.

2.2 Critical Evaluation of Related Products/Solutions

The latest investigations reveal that defence mechanisms are becoming more fragmented by location and the resources they protect (Zargar et al., 2013).

Evasion and Adaptation: Simple volumetric packet filtering, once considered sufficient, is now often bypassed. Attackers use IP spoofing to hide their true origin and bypass basic ingress and egress filtering, making source traceback extremely complicated (AL-Musawi, 2012; Peng et al., 2007). More advanced techniques, such as Low-Rate DoS (LDoS), exploit flaws in TCP's retransmission timeout and congestion control algorithms. These attacks can deny service with traffic volumes low enough to evade traditional high-threshold detection systems (Zhijun et al., 2020). LDoS attacks present a serious threat, as they imitate legitimate traffic behaviour, ratifying that network defence should rely on smart behavioural analysis instead of just traffic volume thresholds.

The Dual-Plane Defence Requirement: The vital operational division in modern network operating systems, such as Cisco IOS-XE, requires a segmented approach to security. Packets destined for the router's CPU (Control Plane) demand protection that is different from traffic

that simply flows through the router to a server (Data Plane) (Santos, 2020). Given this architectural reality, a single defensive method is deficient for comprehensive protection.

Control Plane Policing (CoPP): CoPP serves as the primary protection for the router's local CPU. It helps maintain operational stability by explicitly limiting unwanted traffic intended to reach the processor (Cisco Community, 2025; Paquet, 2013). It keeps computational costs low by only conducting simple packet classification and traffic rate limiting.

Zone-Based Firewall (ZBF): ZBF, on the other hand, is a stateful inspection engine designed to maintain session integrity for traffic moving between defined security boundaries (Integrate UK, 2025; Study-CCNP, 2025). Its key advantage is its ability to handle state-heavy attacks such as SYN floods, but its detailed packet processing requires more computational resources compared to CoPP (Juniper Networks, 2025).

2.2.1 Identification of the Research Gaps

The literature consistently acknowledges that both control-plane and stateful data-plane protection are crucial for layered defence (Zargar et al., 2013). However, while some papers discuss the different roles of these tools, such as ZBF compared to older Context-Based Access Control (CBAC) techniques (Shaikh et al., 2019), they frequently fail to offer clear comparisons of their resource trade-offs.

The predominant gap in research is the lack of measurable quantitative data detailing how CoPP (a low-overhead, rate-limiting method) and ZBF (a high-overhead, stateful inspection method) affect key network performance metrics under stress.

1. **CPU/Resource Utilisation:** What is the exact computational cost of each method during successful mitigation?
2. **Legitimate Traffic Latency:** How much extra latency and throughput loss does each defence create for authorised traffic?

By isolating and measuring the performance profiles of CoPP and ZBF against specific attacks they are designed to manage, such as ICMP and SYN floods. This project will provide the measurable evidence needed to help network designers create a strong, performance-focused layered defence strategy.

2.3 The Scope of the Project

The scope of this project is on the particular architectural vulnerabilities of the Cisco IOS-XE platform and a comparative analysis of its integrated defence mechanisms.

2.3.1 Functional Scope

The research is strictly focused on evaluating two types of Denial of Service (DoS) attack vectors:

- **ICMP Floods**: These attacks target the router's Control Plane to drain CPU resources.
- **SYN Floods:** These attacks aim at the Data Plane to exhaust session state tables.

The analysis is limited to a quantitative comparison of two defence scopes: Control Plane Policing (CoPP), which uses stateless rate-limiting, and Zone-Based Firewalls (ZBF), which employ stateful inspection (Paquet, 2013). It is important to understand that other Cisco-centric protocols, such as VLANs and OSPF, will be used to ensure the full functionality of the network architecture. Other defence methods, such as external scrubbing services, are not included in this study to maintain the comparison centred on device-level self-defence.

2.3.2 Environmental Scope

The project does not employ physical network hardware. All experiments are conducted in a simulated environment. The tests are conducted in a high-performance Hetzner dedicated server running on GNS3. Physical factors such as cabling issues, power consumption, or electromagnetic interference are therefore excluded from this project. This is due to the fact that a virtual environment is selected for this project with the aim of eliminating any hardware limitations that could otherwise be experienced in a desktop environment. Simulating Denial of Service (DoS) attacks, including TCP SYN floods and ICMP floods, is computationally expensive. Conducting such tests could compromise the results due to resource exhaustion on the hypervisor (e.g., slow CPU or memory limits), which could distort packet drop results and compromise the evaluation of the ZBF policies.

To mitigate these issues, the core simulation environment will be hosted on a dedicated Hetzner bare-metal cloud server with 64GB of RAM and modern multi-core processors. This guarantees that any throughput degradation or packet loss recorded while conducting the

experiment is due only to the configuration of the Cisco Zone-Based Firewall (ZBF) and Control Plane Policing (CoPP), rather than any inherent hardware limitations.

In addition, deploying the simulation within a public cloud environment results in inherent security vulnerabilities as it exposes the control plane of the GNS3 hypervisor to the public internet. To mitigate this and adhere to today's DevSecOps best practices, the management plane is secured with Tailscale and the built-in Hetzner server firewall.

2.4 Review and Justification of Technical Tools and Platform

The selection of appropriate technical tools and platforms is of critical importance to ensure that the DoS mitigation strategies can be evaluated within an environment that is representative of the actual behaviour of a modern enterprise network. For this professional network security project, a set of high-quality tools has been carefully selected and structured into hardware and software components to enable a thorough and rigorous technical analysis.

Hardware Tools

- **Hetzner Dedicated Server** - Intel Core i7-6700, 2x SSD SATA 512 GB, 64GB RAM: The Hetzner server acts as the host to run the GNS3 Server and all heavy VMs: IOS XE, Kali, and Servers.

- **HP Pavilion Laptop** 16in Intel(R) Core (TM) Ultra 5 125U (1.30 GHz), 16.0 GB RAM, 512 GB Storage serves as a guest, it connects to the cloud server via the Tailscale tunnel. The laptop is where the topology is designed, but the simulation is run on the Hetzner server.

Software Virtualisation Platform

- **QEMU** (Quick Emulator), the virtualisation platform of choice for this project, is under the **GNS3** platform. QEMU is an open-source machine emulator and virtualisation solution that supports near-native code execution of guest code directly on the CPU under **KVM** (Kernel-based Virtual Machine) virtualisation (QEMU, 2024). The reason for using this platform over other emulators, such as Dynamips, was the support for multi-core processors, which is a requirement for the project to function without the emulator freezing under the DoS packet flood.

- **GNS3** acts as the network emulation platform. It emulates real Cisco router hardware by running actual IOS-XE images. This setup accurately replicates Data Plane and Route Processor interactions during DoS attacks (Balyk et al., 2017; Shea and Liu, 2012).

Secure Connectivity Platform

Tailscale: a Zero Configuration Virtual Private Network (VPN) based on the WireGuard protocol, creates a fast and direct peer-to-peer (P2P) connection between devices, while providing secure and direct end-to-end encryption for the transmission of data (Tailscale, 2024; Hrițcan and Balan, 2024). The creation of this overlay network means that a "Zero Trust" model has been implemented; as such, the public-facing firewall for the Hetzner server will be closed to all external connections and therefore will not be able to detect or respond to any external port scan attempts (i.e., Nmap) on this public IP address (Hrițcan and Balan 2024). Furthermore, the use of this VPN will eliminate any NAT traversal issues and protect against unauthorised access to the remote management channel (Hrițcan et al., 2025).

Network Infrastructure Images

To properly assess ZBF and CoPP, the latest cloud-native Cisco images have been provisioned:

- **Edge Router:** The Cisco Cloud Services Router 1000v (CSR1000v) running IOS-XE (version 16.07.01) is the primary security boundary. The Cisco Cloud Services Router 1000v is a software router designed to be deployed as a virtual machine in cloud and enterprise virtual environments (Cisco, 2020). What is most significant is the IOS-XE operating system's support for the advanced C3PL (Cisco Common Classification Policy Language) necessary to implement modern stateful firewalls and control-plane policing.

- **Switches:** Cisco Virtual IOS Layer 2 (IOSvL2 version 15.2) images are being used to build the Core, Distribution, and Access Switching Topology. Unlike generic switches, Cisco IOSvL2 images have full support for enterprise Layer 2 switching features, including VLAN trunking, Spanning Tree Protocol (STP), and Switched Port Analyser (SPAN) functionalities (Cisco Developer, 2024).

Endpoints and Data Collection Tools

The network endpoints are comprised of unique Linux distributions, each designed for its unique footprint:

- **Ubuntu Cloud Guest:** Canonical provides Ubuntu Cloud Guest as its official, optimised, and regularly updated image for cloud-based environments (Canonical Ltd., n.d.a; Canonical Ltd., 2015). In this configuration, it supports both the target and user nodes, offering a dependable full Linux kernel for essential networking functions such as ARP handling, Netplan configuration, and persistent IP addressing (Canonical Ltd and Ubuntu, 2024). It also remains lightweight. By excluding a graphical desktop, the image avoids unnecessary resource use while retaining the networking components needed to represent a realistic DMZ web server and an internal corporate client.

- **Kali Linux**: As the Attack Node, it is deployed in the "Outside" untrusted zone to launch DoS attack simulations using **hping3,** which is an industry-standard command-line packet crafter capable of generating rapid SYN and ICMP floods. Kali is the standard OS for penetration testing in the industry. This allows for the simulation of complex volumetric or low-rate DoS attack testing, focusing on stateful inspection (Khalaf et al., 2025; Khalaf et al., 2019; Joseph et al., 2023).

- A second **Kali Linux** instance operates as a dedicated monitoring station and is connected directly to the Core Switch through a configured SPAN port. This port-mirroring configuration securely replicates all legitimate and attack traffic to the monitoring node while keeping it fully outside the active attack path. This separation is essential, as it ensures that monitoring activities do not influence router performance or distort the results of the simulation. From this isolated vantage point, **Iperf3** is used to measure legitimate TCP/UDP throughput and latency during active attack scenarios (Moldstud, 2024). **Wireshark** further complements this process by providing packet-level visibility, enabling verification of the exact attack volume and identification of dropped or malformed packets during each test iteration (Chowdappa et al., 2023).

In parallel, system-level monitoring is conducted using SNMP polling and Cisco IOS CLI logging tools. Commands such as **"show processes cpu"** are used to quantify the Edge Router's CPU utilisation throughout the attack and mitigation cycles, providing a clear assessment of resource consumption under stress (Fortinet, 2025). Collectively, these monitoring mechanisms ensure that the dataset captured is both accurate and comprehensive, allowing the study to rigorously evaluate the performance and overhead of the implemented DoS mitigation strategies, turning theoretical ideas into measurable insights on performance (Bugnion, 2012; Balyk et al., 2017).

Chapter 3: Problem Analysis and Specification

This chapter outlines the formal specification for the research investigation. It translates the theoretical challenge identified in the critical review in Chapter 2: into measurable, achievable, and verifiable objectives and hypotheses. The focus here is on defining what the project will prove. It also sets precise criteria for the rigorous methodology that will be detailed in Chapter 4:.

3.1 Problem Statement and Context

The research discusses the persistent threat of Denial-of-Service (DoS) attacks, which continue to put service availability at risk for both public and private infrastructure (Zargar et al., 2013). Modern network devices, such as Cisco IOS-XE routers, have a divided architecture. The Control Plane manages device operations and routing stability, while the Data Plane handles fast packet forwarding. This creates two potential attack areas (Santos, 2020).

Specific threats include ICMP Floods, which overload the Control Plane's CPU by bombarding it with diagnostic requests and can disrupt operations (Harshita, 2017; Liu et al., 2023). There are also SYN Floods that exploit TCP handshakes to drain session resources in the Data Plane (Bekravi et al., 2012; Jamali and Shaker, 2012). The main problem is the lack of studies measuring the effectiveness and costs of integrated defences such as Control Plane Policing (CoPP). CoPP provides low-overhead CPU protection through stateless rate-limiting (Cisco, 2025; Paquet, 2013). Another defence is the Zone-Based Firewall (ZBF), which uses resource-heavy stateful inspection to secure transit sessions (Integrate, UK 2025; Study-CCNP, 2025). This situation emphasises the need for measurable data to support their combined use and compare the overheads.

3.2 Aims and Second-Level Objectives

The project aims to conduct a quantitative comparison to resolve the empirical gap regarding resource consumption and effectiveness in a layered network defence model.

Top-Level Aim

Design, implement, deploy, and meticulously evaluate a three-tier network topology in a simulated virtual network environment to mitigate complex DoS attack vectors, using detailed Cisco-centric network configurations as mentioned in the previous project proposal.

Table 1 - Second-Level Objectives

Objectives	Description
Design and Implementation	Establish a complex, segmented three-tier virtual network architecture utilising GNS3 in a remote dedicated server, incorporating standard enterprise routing protocols such as IEEE 802.1q for VLAN trunking, and network segmentation to serve as a high-fidelity experimental testbed in a Router-on-Stick architecture.
Baseline and Attack Simulation	Quantitatively measure the network baseline performance in terms of latency and throughput, and establish the saturation point of the edge router when it faces unmitigated SYN Flood and ICMP Flood attacks using Iperf3 and hping3, ensuring the attack volume is sufficient to induce failure (Harshita, 2017; Khalaf et al., 2019).
Comparative Mitigation Policy Deployment	Implement and configure two distinct, layered defence policies, Control Plane Policing (CoPP) and Zone-Based Firewall (ZBF), on the edge router, specifically targeting the mitigation of the two simulated DoS attack types (Paquet, 2013; Study-CCNP, 2025).
Empirical Analysis and Quantification	Critically assess the performance and resource overhead of CoPP against ZBF by comparing three key metrics: (a) Attack Packet Drop Rate, (b) Legitimate Traffic Latency Degradation, and (c) Edge Device CPU/Resource Utilisation under controlled attack conditions.

3.3 Definition of Empirical Performance Metrics

The assessment of the second-level objectives relies on the careful collection and examination of the following quantitative metrics:

Table 2 - Quantitative Metrics Tools

Metric	Measurement Tool	Justification of Relevance
Metric 1: Attack Packet Drop Rate	Use Cisco policy counters, such as the "show policy-map control-plane" command, and network traffic analysers.	Verifies the immediate success rate of the defence policy against the attack volume. It measures the number of packets discarded at the inspection point.

Metric 2: Legitimate Traffic Latency Degradation	Iperf3 measurements, showing latency in milliseconds and the corresponding throughput data.	Quantifies the negative effects of the security policy on authorised user traffic. It shows how this impacts the Quality of Service (QoS) and service resilience (Keysight, 2025).
Metric 3: Edge Device CPU Resource Performance	SNMP polling and Cisco IOS CLI logging, such as using the command "`show processes cpu`," are performed on the router being tested.	The most crucial metric for comparative analysis is measuring the computational cost, or overhead, of stateful inspection (ZBF) compared to stateless policing (CoPP) (Fortinet, 2025).

3.4 Research Questions and Core Hypothesis

This investigation will focus on a primary hypothesis and will be validated by answering three essential research questions:

Core Research Hypothesis

Implementing stateful session protection with the Zone-Based Firewall (ZBF) to defend against Data Plane attacks, such as SYN floods, results in much higher peak CPU usage on edge routers. This contrasts with the rate-limiting method of Control Plane Policing (CoPP) for reducing Control Plane attacks, such as ICMP floods. This difference shows a performance trade-off. It supports the need for a multi-layered defence strategy to improve overall security.

Essential Investigative Questions

1. Efficacy and Overhead of Control Plane Protection: How effectively and efficiently does Control Plane Policing (CoPP) mitigate the negative impact of a localised ICMP Flood attack on the router's CPU? What measurable amount of resource overhead (CPU load) occurs during successful mitigation?

2. Efficacy and Overhead of Data Plane Protection: How successfully does the Zone-Based Firewall (ZBF) mitigate service disruption during a SYN Flood attack on a downstream server, and what is the resulting impact on the latency and throughput of legitimate traffic?

3. Comparative Resource Cost Analysis: What is the measured difference in peak CPU usage on the edge device between a successful mitigation performed by CoPP and a successful mitigation performed by ZBF, and how does this comparison help us understand the resource allocation trade-offs in layered security design?

Chapter 4: ValerioResearch and Software Methodology Design

For this project, the methodology is divided fundamentally into two essential components. The first part is an Academic Research Methodology based on the Saunders Research Onion, which helps maintain scientific accuracy. The second part is an Agile Development Lifecycle, which manages the technical implementation of the Cisco-focused simulation environment.

4.1 The Research Onion Framework

To ensure scientific accuracy, the research design follows the Research Onion model (Saunders *et al.,* 2009). This model offers a step-by-step methodology that forces the researcher to explain their philosophical beliefs before moving on to practical data collection approaches (Sahay, 2016). By "peeling" the layers of the onion, the study maintains internal consistency. Furthermore, it provides a "methodological cookbook" that enables other researchers to replicate the findings (Alturki, 2021).

The study uses a positivist philosophy (Alturki, 2021). It views reality as stable, objective, and measurable. This perspective is crucial for network security research on DoS attack mitigation. The research relies on objective data, such as packet drop rates and CPU cycles, to uncover factual truths (Sahay, 2016).

It follows a deductive approach (Alturki, 2021). The study starts with theoretical frameworks and tests a hypothesis. This hypothesis states that a layered defence strategy, which includes Control Plane Policing (CoPP) and Zone-Based Firewall (ZBF), can significantly reduce resource exhaustion from volumetric floods on edge routers. This is compared to an unprotected state, using data to validate or challenge the theory (Sahay, 2016).

The main research strategy is experimental. It takes place in a controlled virtual lab environment with GNS3 (Alturki, 2021). In this environment, independent variables such as DoS attack volume and type generated using Kali Linux are adjusted. The study observes their impact on dependent variables such as router stability and legitimate traffic throughput. This helps establish clear cause-and-effect relationships between attacks and mitigation measures (Alturki, 2021).

4.2 Methodological Data Selection: Qualitative and Secondary Data Analysis

This project adopts a qualitative research method combined with secondary data analysis. Qualitative research refers to strategies that aim to uncover how phenomena are created and understood. It regularly focuses on verbal description and interpretation instead of numerical measurement (Hammersley, 2013).

Secondary data analysis involves examining data collected by someone else for a different main purpose (Johnston, 2014). This decision is supported by several important technology aspects (ASTAs).

Viability and Efficiency: Using existing high-quality datasets is a functional and efficient option, particularly for researchers with limited time and resources (Johnston, 2014). It enhances the pace of the research process by eliminating the most time-consuming steps in measurement development and primary data collection (Johnston, 2014).

Empirical Rigour: Although secondary analysis is flexible, it remains an empirical exercise that adheres to the same fundamental research principles as studies, using primary data (Johnston, 2014).

Contextual Insight: Qualitative methods emphasise richness and detail, incorporating context directly into the analysis. This creates strong arguments about how things work in specific settings (Hammersley, 2013).

In this project, secondary data, such as journals, peer-reviewed papers, books, and industrial technical reports, have been carefully selected in accordance with the London Met University library guidelines. This data was acquired from highly reputable searchable databases such as IEEE. The purpose of this selection is to combine a set of baselines for DoS attack impact and to evaluate the effectiveness of Cisco-centric mitigation techniques. The combination of a Positivist/Experimental research design with an Agile project management framework ensures that the investigation is both scientifically sound and professionally managed.

4.3 Software Methodology

Agile methodology was adopted for the technical development of this project. While the Research Onion provides the scientific foundation, an Agile software methodology, along with

the Scrum framework, governs the technical project management (Sachdeva, 2016). Agile is particularly effective for network security projects that have "requirements volatility." In these projects, technical configurations must be adjusted in response to real-time feedback from the simulation environment (Sachdeva, 2016; Singh, 2008).

Agile is defined not just as a process, but as a mindset originated from the principles of the Agile Manifesto. It prioritises individuals and interactions over strict processes and tools (Rodgi and Bhirud, 2024). Unlike traditional plan-driven or "Waterfall" models, which are linear and sequential, Agile is cyclic and iterative. This approach allows for quick delivery of value through frequent builds (Nikitin, 2025; Sachdeva, 2016).

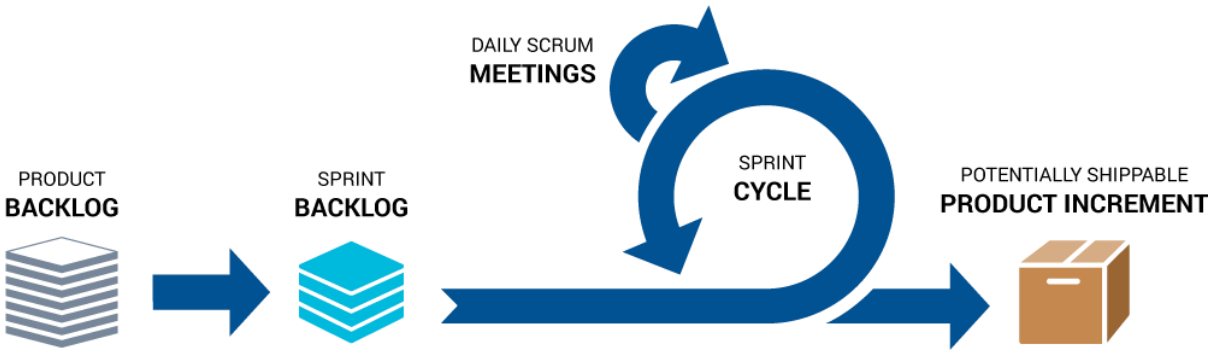


Figure 1 - Agile Software Methodology (Stigasoft, 2025)

The principal benefit of Scrum in a network security simulation is its ability to manage "Requirements Volatility." Technical needs and attack vectors often change during development. Traditional predictive planning usually struggles with unexpected technical issues. Scrum addresses this by implementing an empirical approach that recognises the problem cannot be completely defined from the start. It emphasises transparency, inspection, and adaptation, using real-time decision-making based on actual data from the simulation (Sachdeva, 2016).

4.3.1 Scrum Ceremonies and Artefacts

In the technical build process, transparency is maintained through vital Scrum ceremonies and artefacts, as described by Sachdeva (2016). The Product Backlog serves as the master list that includes all functional and security requirements for the simulation. This list covers everything from topology constructions to targeted DoS vector implementations (Sachdeva, 2016). It helps

reduce the chances of falling short on project goals by the end (GJCST, 2020). At the start of each sprint, the researcher selects the top-priority tasks from this backlog to create the Sprint Backlog (Sachdeva 2016). Daily Scrums, which serve as quick self-checks, help identify and address technical issues such as GNS3 image failures or Qemu/KVM resource conflicts (Sachdeva, 2016). Finally, Sprint Reviews and Retrospectives will give opportunities to showcase completed increments, such as baseline performance reports, while reflecting on the workflow to improve future testing phases (Sachdeva 2016; Singh 2008).

Scrum allows for the incremental construction of the simulation environment through four key technical phases in its sprints, as outlined in the development process.

In Phase 1, which focuses on environment orchestration and infrastructure, sprints prioritise establishing the virtualisation foundation using Qemu/KVM for hardware-independent encapsulation (QEMU, 2024; Bugnion, 2012). This is followed by integrating the GNS3 emulator to run binary Cisco IOS-XE images (Balyk et al., 2017; Franklin Fitch, 2023).

Phase 2 proceeds to network configuration and performance testing. Here, sprints create a three-tier hierarchical architecture that includes routing protocols such as OSPF and VLAN segmentation (Network Academy 2025). They also measure performance baselines using tools such as Iperf3 (Sachdeva, 2016; Moldstud, 2024). Moving to Phase 3, the focus shifts to executing attacks and documenting stress. This phase involves repeatedly launching DoS attacks, such as SYN and ICMP floods, using Kali Linux and hping3 (Khalaf et al., 2019). The goal is to record the saturation points of the edge router's control and data plane (Fortinet 2025; IBM, 2025). Finally, Phase 4 concentrates on integrating mitigation measures and evaluating their effectiveness. Sprints in this phase apply Control Plane Policing (CoPP) and Zone-Based Firewall (ZBF) strategies (IBM, 2025; Paquet, 2013). They assess effectiveness by comparing stress and post-mitigation data (Singh, 2008).

The diagram below illustrates the entire Scrum framework implemented in the project's network technical development:

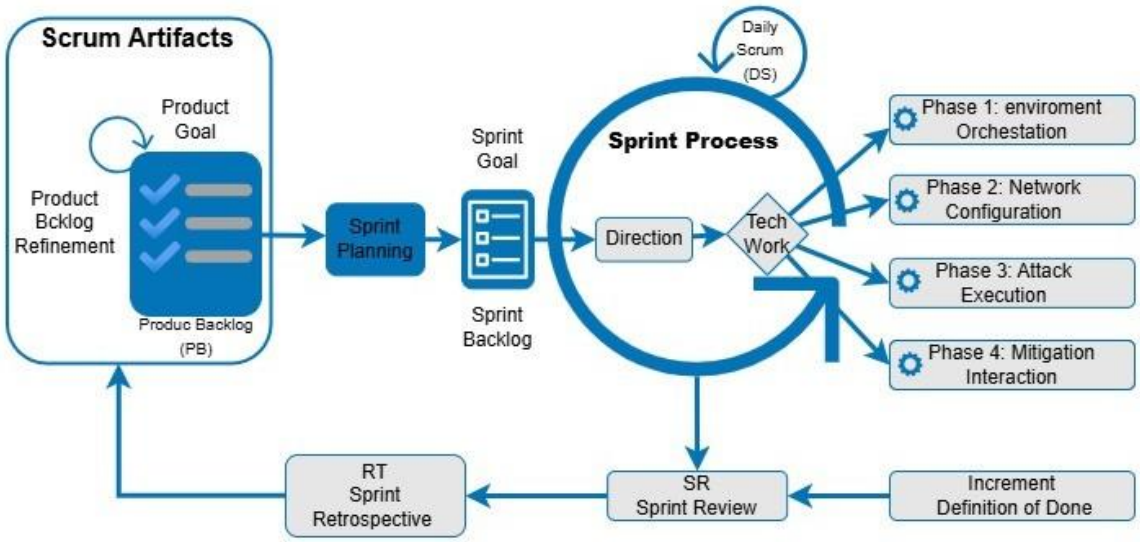


Figure 2 - Project's Agile Network Development Methodology

4.4 Network Architectural Design

To satisfy the learning outcomes for structured design, the system is documented using the C4 Model (Context, Container, Component). The C4 model facilitates the design of visual software architecture with an "abstraction-first" approach. It represents static structures at four levels: context, containers, components, and code (Jokste and Kampars, 2024). This framework is important because it provides different viewpoints and levels of detail. This allows each stakeholder to interact with the architecture design based on their specific information requirements. Additionally, the model enhances communication within complex, modular systems by offering a visualisation that does not require stakeholders to have specialised knowledge of complex diagramming notations (Jokste and Kampars, 2024).

4.4.1 Hierarchical Three-Tier Architecture with Centralised Core Routing

The internal network uses a Hierarchical Three-Tier Architecture with Centralised Core Routing. This design modifies the traditional three-tier model to improve both performance and security (Tiso, 2012; Cisco Systems, 2010).

Instead of distributing inter-VLAN routing across the Distribution layer or using a classic 802.1Q router-on-a-stick approach, this architecture centralises all internal routing at the Core Switch. The Core Switch employs Switched Virtual Interfaces (SVIs) and serves as the default gateway for all internal networks.

This logical separation delivers clear benefits. Lateral traffic between the Internal User and the DMZ Target moves at wire-speed within the switching fabric. It completely avoids the latency and hairpinning problems that often plague edge-based routing (Study-CCNA, 2021).

By contrast, every packet entering or leaving the external network must pass through the Edge Router. The Edge Router functions strictly as the WAN boundary. Here, the Zone-Based Firewall and Control Plane Policing engines carefully inspect all external traffic (Hartpence, 2015, pp. 346, 380).

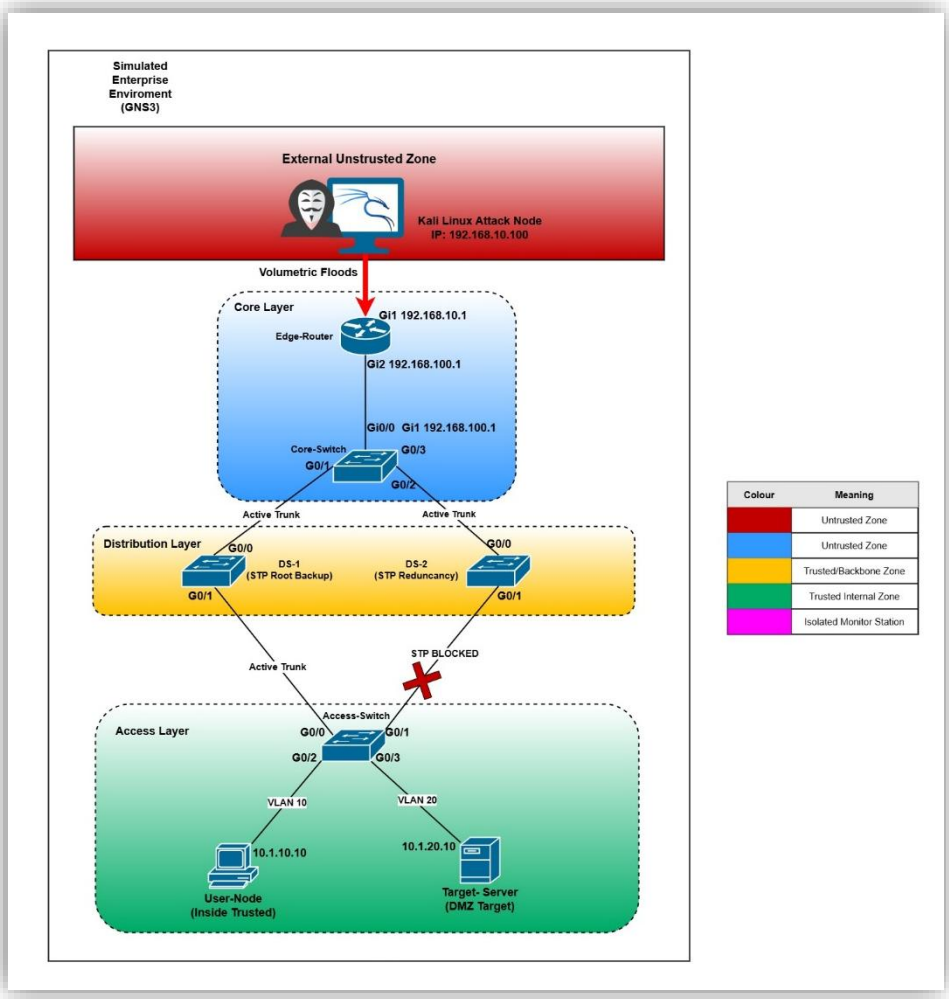


Figure 3 - Hierarchical Three-Tier Architecture with Centralised Core Topology

External Untrusted Zone

The Kali Linux Attack Node simulates real-world threats. It launches flooding attacks directly at the Edge Router. Penetration testers commonly use this type of node. The node employs tools such as hping3 to generate volumetric attacks. These attacks help researchers evaluate the network's resilience under pressure (Balyk et al., 2017).

The Core Layer

The Core Switch acts as the network's high-speed Layer 3 routing backbone. It efficiently routes traffic between internal VLANs through Switched Virtual Interfaces (SVIs). It also maintains a default route that forwards external traffic to the Edge Router. To create a stable Layer 2 topology and prevent broadcast loops, the Core Switch serves as the Spanning Tree Protocol Primary Root Bridge for all VLANs (Odom, 2020a, p. 213).

The Distribution Layer

Two separate switches, DS-1 and DS-2, form the Distribution layer. They meet modern enterprise redundancy standards. These switches aggregate uplinks from the Access layer and provide multiple physical paths to the Core.

Layer 2 loops can trigger broadcast storms that bring down the entire network (Odom, 2020a, p. 213). For this reason, the design implements Rapid Per-VLAN Spanning Tree Plus (Rapid PVST+). The protocol builds loop-free logical topologies. It selects the most efficient paths for data and keeps backup links in standby mode. This approach delivers strong resilience without the complexity of extending Layer 3 routing protocols to the Distribution layer (Odom, 2020a, p. 264).

The Access Layer

The Access Switch sits at the network edge. It provides physical connectivity to internal devices, including the Internal User endpoint and the DMZ Target Server. The switch performs strict Layer 2 segmentation. It assigns ports to VLAN 10 or VLAN 20, placing each device into its own isolated broadcast domain (Odom, 2020a, p. 179).

As traffic arrives from these endpoints, the switch adds 802.1Q tags. It then forwards the frames up the trunk links to the Distribution layer and onward to the Core. Internal routing decisions

occur at the Core layer. External security policies take effect at the Edge boundary (Odom, 2020a, p. 396).

This layered design creates a secure, efficient, and highly available foundation. Internal routing remains fast and responsive. Perimeter security stays firmly controlled at the edge. The architecture offers an excellent platform for realistic enterprise network simulations. It combines hierarchical scalability, virtualisation flexibility, and layered security to deliver genuine resilience. Early work with Quick Emulator (QEMU) first demonstrated these virtualisation capabilities. Today, GNS3 builds on that foundation. It enables practical, hands-on testing in a controlled, high-fidelity environment where network designers can safely recreate and evaluate real-world threats.

To achieve perfect networking communication and logical segmentation across these layers, the following IP addressing table have been created:

Table 3 - IP Addressing Table

Device	**Interface / VLAN**	**IP Address**	**Subnet Mask**	**Default Gateway**
Attacker-Kali	eth0	192.168.10.100	255.255.255.0	192.168.10.1
Edge-Router	Gi1 (Outside WAN)	192.168.10.1	255.255.255.0	N/A
	Gi2 (Transit to Core)	192.168.100.1	255.255.255.0	N/A
Core-Switch	Gi0/0 (Transit to Edge)	192.168.100.2	255.255.255.0	192.168.100.1
	VLAN 10 (SVI)	10.1.10.1	255.255.255.0	N/A
	VLAN 20 (SVI)	10.1.20.1	255.255.255.0	N/A
DS-1 (Distribution)	Gi0/0, Gi0/1	N/A (L2 Trunk)	N/A	N/A
DS-2 (Distribution)	Gi0/0, Gi0/1	N/A (L2 Trunk)	N/A	N/A
Access-Switch	Gi0/0, Gi0/1 (Uplinks)	N/A (L2 Trunk)	N/A	N/A

	Gi0/2 (VLAN 10)	N/A (Access)	N/A	N/A
	Gi0/3 (VLAN 20)	N/A (Access)	N/A	N/A
Target-Server	eth0	10.1.20.10	255.255.255.0	10.1.20.1
User-Node	eth0	10.1.10.10	255.255.255.0	10.1.10.1

4.4.2 Technical Resources Requirements

The following table describes the technical resources that are required to deploy the lab virtualisation.

Table 4 - Technical Resources Requirements for the Virtual Lab.

Name	Description
Hardware	HP Pavilion Laptop 16in Intel(R) Core (TM) Ultra 5 125U (1.30 GHz), 16.0 GB RAM, 512 GB Storage. A remote server (Hetzner Intel Core i7-6700, 2x SSD SATA 512 GB, 64GB RAM).
Software	QEMU/KVM serves as a virtualisation emulator for Linux, while GNS3 is a free and open-source virtual network simulator. Cisco IOS images are used to simulate various devices, and Kali Linux images facilitate attack distribution. Iperf3 allows the simulation of baseline legitimate traffic, and Tailscale provides a secure peer-to-peer connection between the host-dedicated server and the guest laptop.

Chapter 5: Technical Implementation and Testing

The implementation phase of this project focused on translating the logical network design into a functional, high-fidelity virtualised environment capable of operating with the resource exhaustion inherent in Denial of Service (DoS) simulations. This section details the deployment of the core hypervisor infrastructure, the security of the management plane, and the provisioning of the specific network nodes.

5.1. Server Purchased Process

To get a server from the Hetzner Server Auction, visit https://www.hetzner.com/sb/ and browse the available servers and use filters for price, RAM, CPU, or storage. The auction follows a Dutch-style model, starting with high prices that go down over time until someone clicks "Order" to buy. These servers are used but fully functional. For this project, a powerful server was chosen due to the magnitude of the project, which was justified in the section 2.4

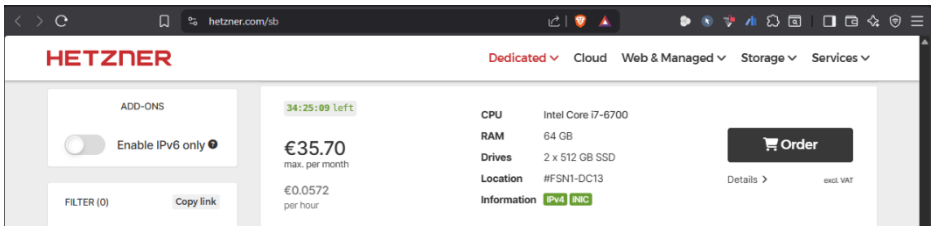


Figure 4 - Choosing the Ideal Dedicated Server

To complete the order, a Hetzner account and access Robot for server management must be created, and the access method (HSS password or public key) must be chosen.

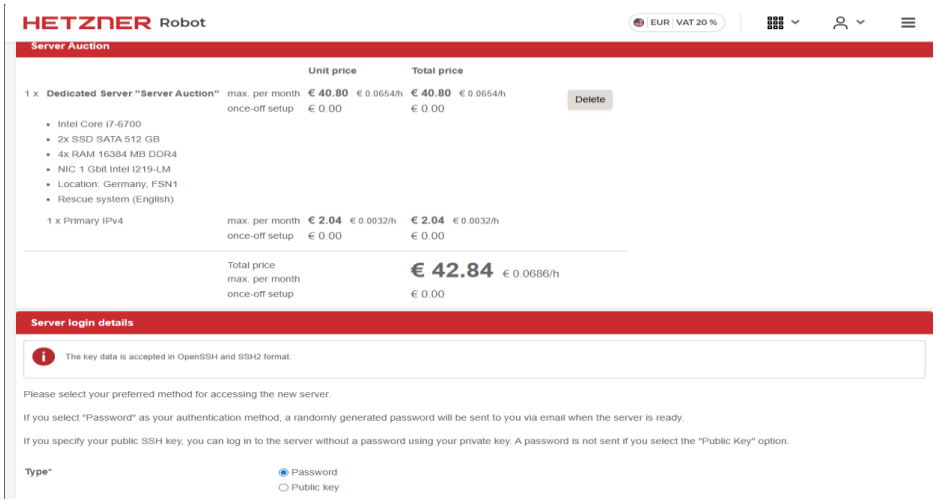


Figure 5 - Completing the Server Order and Choosing Access Method

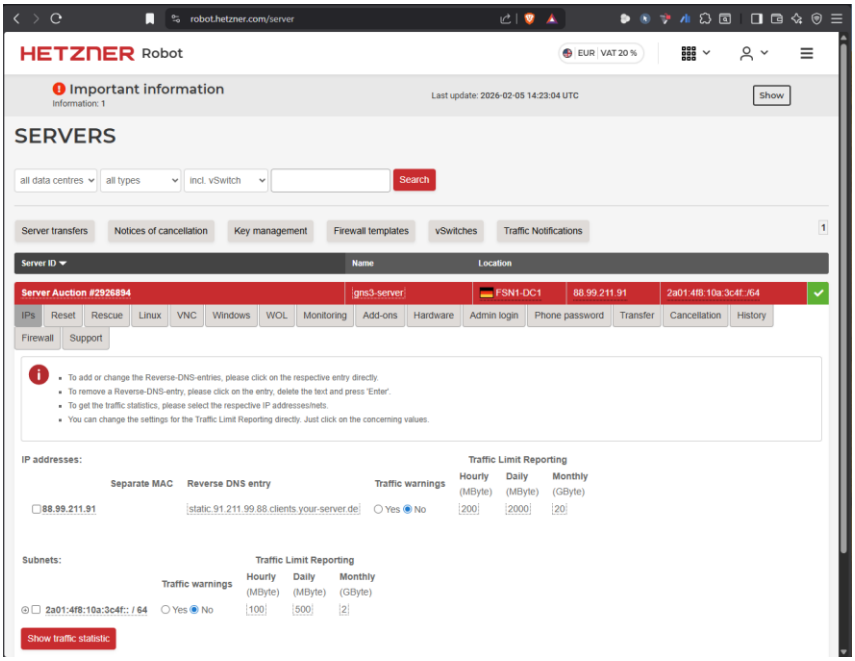


Figure 6 -Hetzner Robot Server Dashboard

Servers are usually set up within one business day and delivered in rescue mode, requiring manual OS installation via installimage. They are unmanaged, so the user is responsible for all software, security, and maintenance (Hetzner.com, 2025). Once the server is ready, an email with login details will be sent to start configuration.

Figure 7 and Figure 8 show the server login details and that the current server status is 'running'.

HETZNER

Dear Mr Alberto Novas

Thank you for your order B20260218-3369720-2944728 from 18/02/2026 15:38 CET.

Login Details

Your server Server Auction #2926894 (88.99.211.91, 2a01:4f8:10a:3c4f::/64) has been activated and is now ready for use. Currently it is booted in the Rescue System. You can access the server via SSH2 using the following details:

IPv4 Address:	88.99.211.91
IPv6 Address:	2a01:4f8:10a:3c4f::2
Username:	root
Password:	_3mv_rXWU8G%xm
Host key:	[redacted] OG9g7bWCVwI (ECDSA 256) [redacted] fay/m1jwVRN2U (ED25519 256) +KPnBtLPtpaRoenoVfpfjivjaWwQ5cL [redacted]

Figure 7 - Dedicated Server's Login Details and IP Addresses

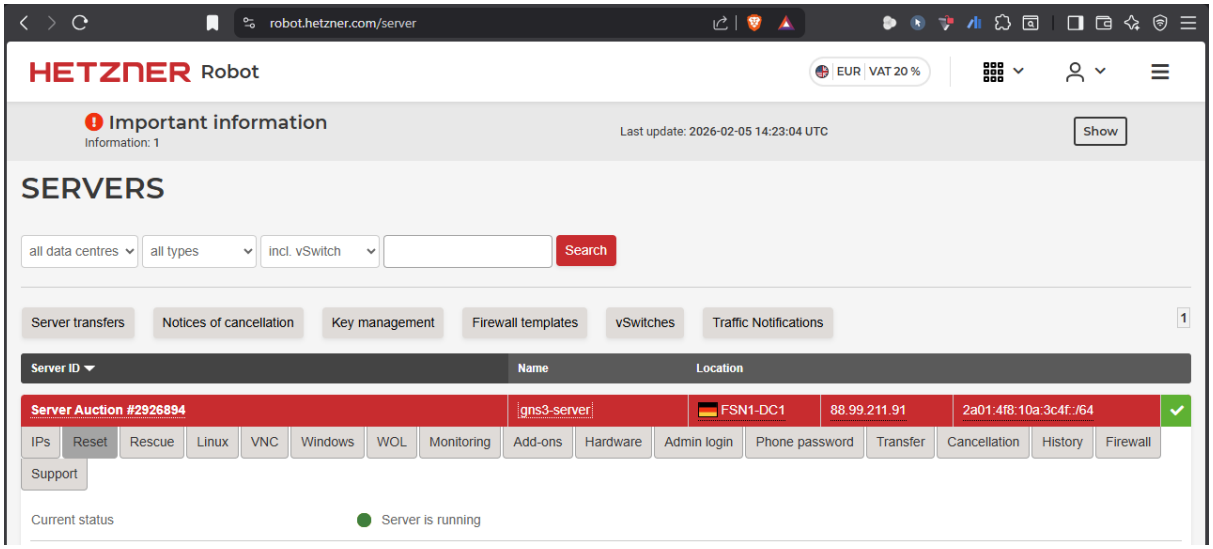


Figure 8 - The Server is Up and Running

5.2. Connect to the Rescue System and Install the Operating System

Once the server is provisioned, it boots into a temporary repair environment known as the Rescue System. In this 'blank slate' state, the machine has no operating system installed on its drives. The Rescue System gives the administrator full control to troubleshoot, configure the server, and install an operating system using the provider's built-in tools, such as Hetzner's **installimage** script (Hetzner.com, 2025).

To access the server's rescue system securely, the connection must be made over Secure Shell (SSH). SSH is a network protocol that encrypts all data sent between the computer and the server, ensuring safe access even over the internet (Barrett and Silverman, 2001). Using this method, we can log in to the remote system and run configuration commands as though we were working on the machine directly.

This section explains the server's initial setup process, including connecting to the rescue system via SSH and installing the operating system.

5.2.1 Connecting to "Rescue System

Utilising the PowerShell terminal, and typing **`ssh root@<the_server_ip_address>`** command and the passwords received in the email after purchasing the server, detailed in Figure 7, we

can log in to the server, a welcome message and a prompt like **root@rescure ~ #** will indicate that we have successfully connected to rescue mode as shown in Figure 9.

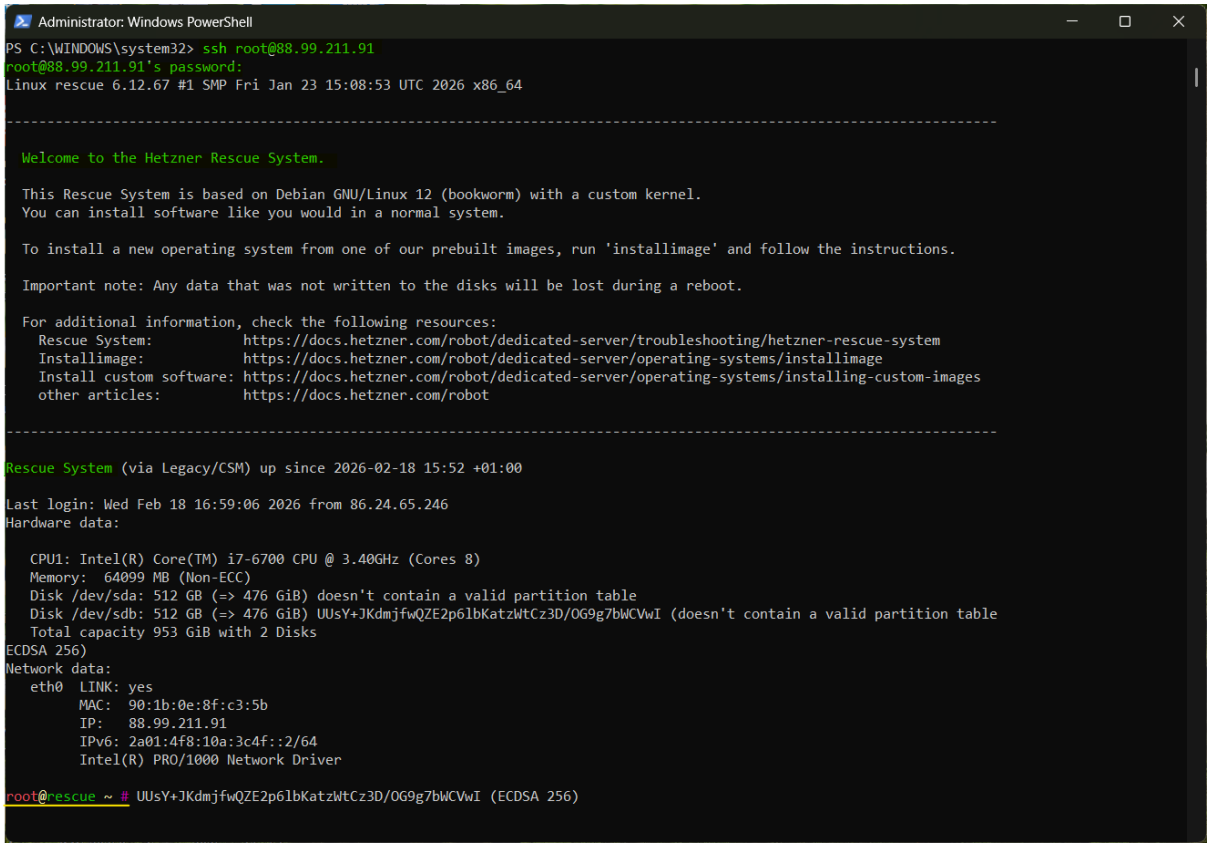


Figure 9 - Successfully logged in to the Rescue System

5.2.2 Install Linux Ubuntu as the server OS

Since this is a blank machine, it is necessary to wipe it and install an OS. For this project, Ubuntu 22.04, a secure and open-source Linux distribution, was selected due to its built-in virtualisation capabilities based on Qemu/KVM, as previously explained on section 2.4.

In the terminal, we run the **installimage** command and then select Ubuntu 22.04.x TLS.

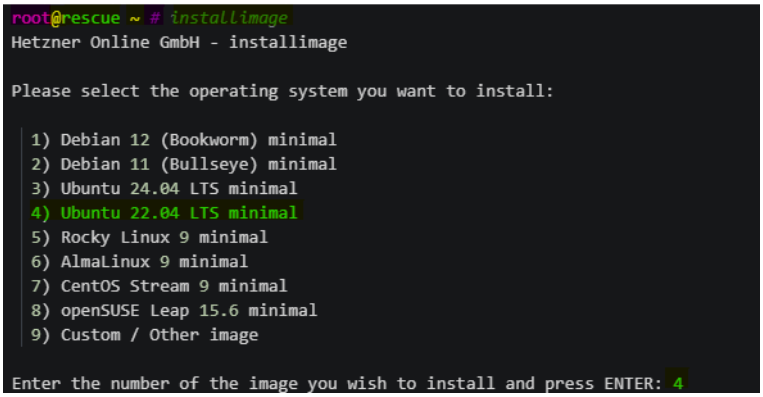


Figure 10 - Selecting Ubuntu 22.04 as OS

After selecting the OS, in the text file configuration, we change the hostname to “gns3-server”. It is also important to keep the SWRAID value set to 1, as this creates an exact copy of the data across both SSDs. This setup provides redundancy and fault tolerance, protecting the project from data loss if one of the drives fails (Diskinternals.com, 2024).

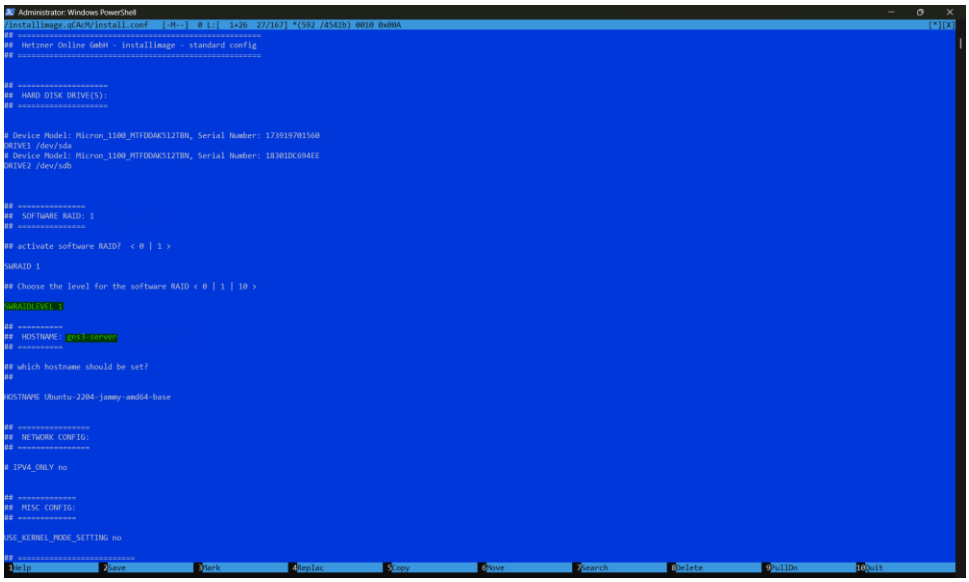


Figure 11 - Adding Parameters to the Installimage Configuration File

Once the configuration file is successfully configured, we save the changes by pressing the **`Fn + F10`** keys and wait until the installation is completed and then run the **`reboot`** command, which switches from RS mode to the newly installed OS, as shown Figure 12.

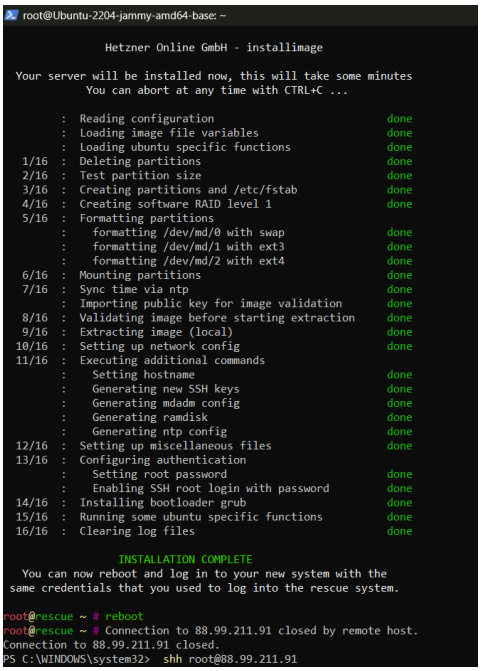


Figure 12 - OS installation Completed & Rebooting the System

5.3. Tailscale Deployment on the Server

This project adopted a Split-Architecture model, in which the Brain (Claud, Hetzner) server hosts the GNS3 server engine and all heavy VMs, and the Face (Local) laptop runs the GNS3 GUI for design and configuration. Because these devices connect over the internet, establishing a secure connection is essential. Tailscale is used to create an encrypted point-to-point tunnel between them, as described in section 2.4.

By running the **`curl -fsSL https://tailscale.com/install.sh | sh`** command in the server terminal, the Tailscale installation process will be initiated. It uses curl to fetch the script from https://tailscale.com/install.sh. It then sends the output directly to **`sh`**, which runs the script instantly without saving it to a file, as shown in Figure 13.

```
root@gns3vm: ~
root@gns3vm ~ # curl -fsSL https://tailscale.com/install.sh | sh
Installing Tailscale for ubuntu jammy, using method apt
+ mkdir -p --mode=0755 /usr/share/keyrings
+ curl -fsSL https://pkgs.tailscale.com/stable/ubuntu/jammy.noarmor.gpg
+ tee /usr/share/keyrings/tailscale-archive-keyring.gpg
+ chmod 0644 /usr/share/keyrings/tailscale-archive-keyring.gpg
+ curl -fsSL https://pkgs.tailscale.com/stable/ubuntu/jammy.tailscale-keyring.list
+ tee /etc/apt/sources.list.d/tailscale.list
# Tailscale packages for ubuntu jammy
```

Figure 13 - Installing Tailscale on the Server

5.3.1 Binding the Server to the Tailscale Network

After successfully installing Tailscale on the server, the next step is to start the services by running the **`tailscale up`** command. This connects the server to the Tailscale network and asks for authentication by providing a link that needs to be pasted in the browser and logging in with an email address, as shown in Figure 15. Once the login process is completed, the server terminal confirms the connection with a 'Success' message, as shown in Figure 14.

```
root@gns3vm ~ # tailscale up

To authenticate, visit:

        https://login.tailscale.com/a/122093f201b157

Success.
root@gns3vm ~ #
```

Figure 14 - Tailscale Requiring Authentication

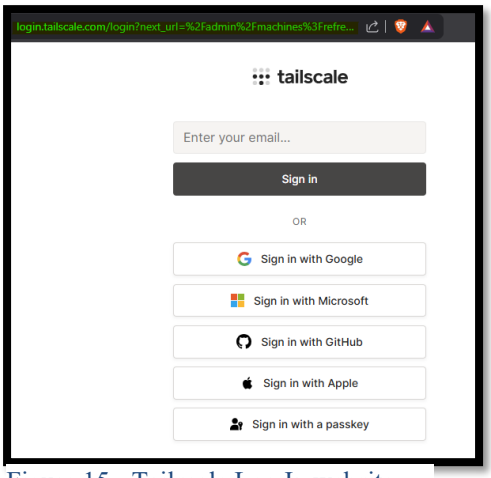


Figure 15 - Tailscale Log In website

The next step is to select the desired account, as shown in Figure 16. Once authenticated, the server immediately appears as connected to the Tailscale network, as illustrated in Figure 17 .

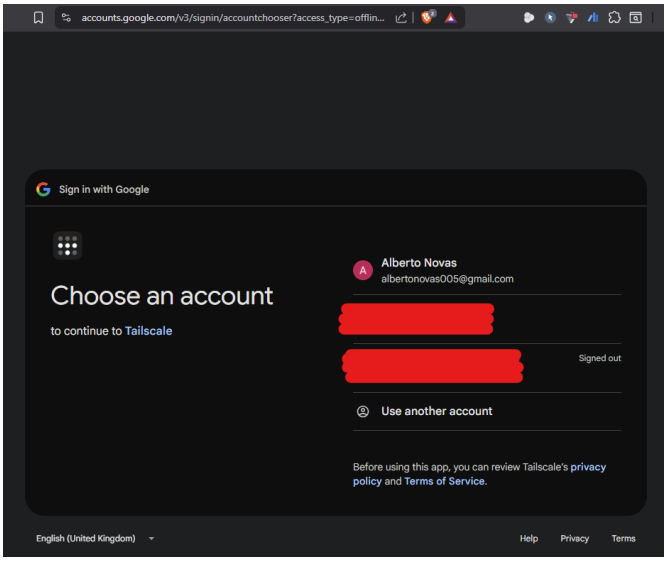


Figure 16 - Selecting Email Account for the Server's Tailscale Authentication

Figure 17 also display the server's name (gns3vm) and the private IP address (100.80.237.20). This private IP address is used to establish a secure connection to the server from anywhere around the world. Communication then takes place through an encrypted WireGuard tunnel, which discards port 22 on the public internet and moves to a zero-trust architecture, described in section 2.4.

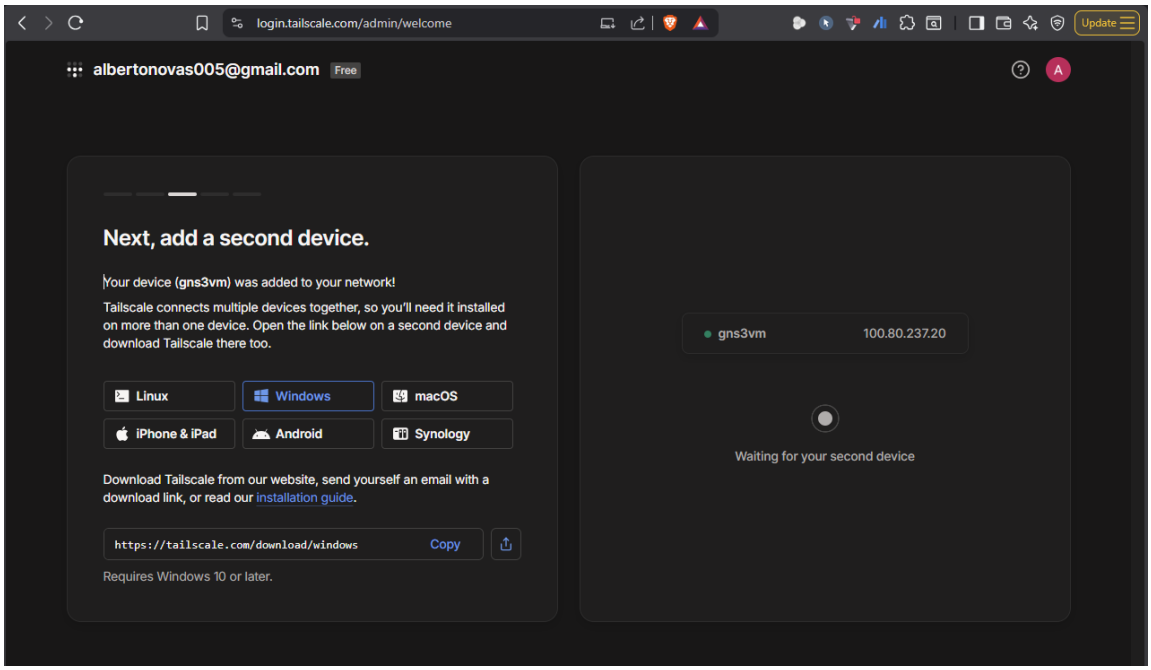


Figure 17 - The Server is Successfully Connected to Tailscale Network

5.3.2 Enabling SSH and Testing Connectivity

Now that the server is fully integrated into the Tailscale network, the next step is to enable SSH access by running the **`tailscale up -ssh`** command in the server terminal, as illustrated Figure 18.



Figure 18 - Enabling an SSH Session on the Server for Tailscale

Figure 19 shows a green SSH label in the Tailscale dashboard appearing beneath the server's name, indicating that it is ready to establish a secure connection.



Figure 19 - SSH is Enabled and Ready to Establish Connection

To establish a Zero-trust point-to-point connection, instead of using the public server IP address, it is essential to use the private Tailscale IP running the command **`ssh root@100.80.237.20,`** then complete the authentication process following the same steps described in Figure 15 and Figure 14 in Step 3.

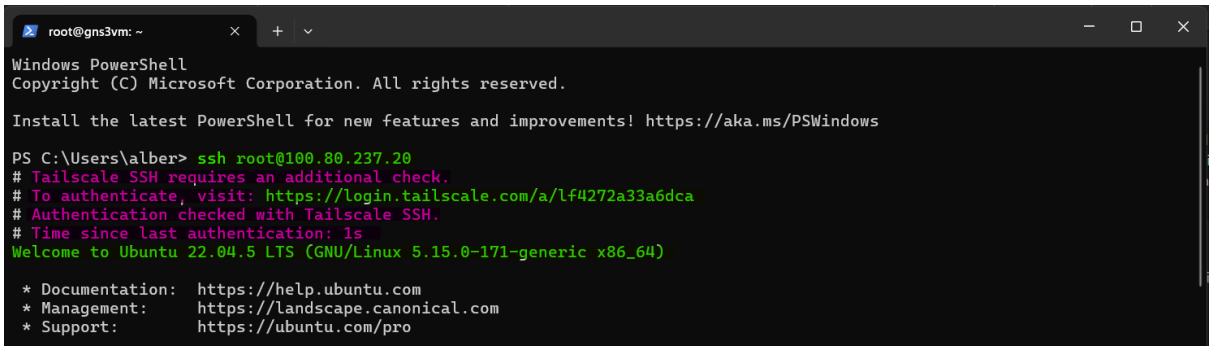


Figure 20 - Establishing SSH Session Using Private Tailscale IP Address

5.4 Installing GNS3 on the Server

According to the GNS3 Documentation (2017), the installation process starts by running the recommended command to download and install all required GNS3 packages on the remote server.

```
cd /tmp curl https://raw.githubusercontent.com/GNS3/gns3 server/master/scripts/remote-install.sh
> gns3-remote-install.sh
bash gns3-remote-install.sh --with-iou --with-i386-repositor
```

The **cd/tmp** command changes the working directory to a temporary and safe location for downloading and running the script. The **curl https://raw.githubusercontent.com/GNS3/gns3 server/master/scripts/remote-install.sh > gns3-remote-install.sh** command downloads the GNS3 from GitHub and saves it locally as **gns3-remote-install.sh.** The script is then executed using **bash gns3-remote-install.sh --with-iou --with-i386-repositor.** The **--with-iou** install Cisco IOU (Cisco on Unix) images, which are lightweight virtual Cisco routers and switches used in GNS3. Additionally, the **--with-i386-repositor** option adds the required 32-bit (i386) architecture support, allowing IOU images to run correctly on 64-bit Ubuntu OS.

Figure 21 and Figure 22 illustrate the command execution on the server terminal to install GNS3.

```
root@source ~ # cd /tmp
root@source /tmp # curl https://raw.githubusercontent.com/GNS3/gns3-server/master/scripts/remote-install.sh > gns3-remote-install
    .sh
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1xxx   100  1xxx    0     0  12345      0 --:--:-- --:--:-- --:--:-- 56789
root@source /tmp # bash gns3-remote-install.sh --with-iou --with-i386-repository
=> Checking if running as root... OK
=> Checking Ubuntu LTS... OK (Ubuntu 22.04.x LTS detected)

=> Updating system packages...
Hit:1 http://mirror.hetzner.com/ubuntu jammy InRelease
Get:2 http://mirror.hetzner.com/ubuntu jammy-updates InRelease [128 kB]
Get:3 http://mirror.hetzner.com/ubuntu jammy-backports InRelease [109 kB]
Get:4 http://security.ubuntu.com/ubuntu jammy-security InRelease [128 kB]
Fetched 12.3 MB in 2s (5,678 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
```

Figure 21 - Initiating GSN3 Installation on the Server

```
=> Installation completed successfully!

GNS3 has been installed with success.

**Important notes:**
- Please **reboot** the server now to ensure all changes (especially OpenVPN and Docker) take effect.
- After reboot, you can download your OpenVPN client configuration from:
  http://YOUR_SERVER_IP:8003/gns3-ovpn-config.ovpn
  (or the exact filename shown by the script)
- The GNS3 server is running on port 3080 (bound to 0.0.0.0 by default).
- OpenVPN is configured on UDP port 1194 with subnet 172.16.253.0/24.

For security on a Hetzner server, consider restricting firewall access (ufw or Hetzner firewall) to your IP for ports 3080 and
    1194.
```

Figure 22 - GNS3 has been Installed Successfully on the Server.

5.4.1 Configuration Secure Access to the GNS3 Server via Tailscale

Once GNS3 is running on the server, it is essential to confirm that it is listening for connections from the private Tailscale IP. This ensures secure access and maintains a strong security posture when managing communication between the NGS3 GUI on the local machine and the GNS3 server over the public internet.

When the installation script ran, it created a dedicated configuration file for GNS3. This file controls key GNS3 server settings, including project path, images, API access, authentication options and port ranges (GNS3, 2026). The **`nano /etc/gns3/gns3_server.conf`** command opens a text editor, as shown in Figure 23 which reveals that no user or password is set by default, meaning authentication is initially disabled.



Figure 23 - Original Nano Text Editor's GNS3 Configuration File

Setting the host IP to **`0.0.0.0`** ensures that GNS3 listens on all available interfaces, including the secure Tailscale tunnel. Adding the **`user`** and **`password`** entries, along with **`auth = True`**, enables forced authentication for secure communication, as shown in Figure 24.



Figure 24 - Updated Nano Text Editor's GNS3 Configuration File

5.5 Downloading and Installing GNS3 GUI on the Local Machine

This section outlines detailed steps for installing the GNS3 GUI. Since GNS3 is quite strict about compatibility, the laptop and the Hetzner server must run the exact same version. Running the **`apt policy gns3-server`** command on the server console displays the installed version, as illustrated by Figure 25.

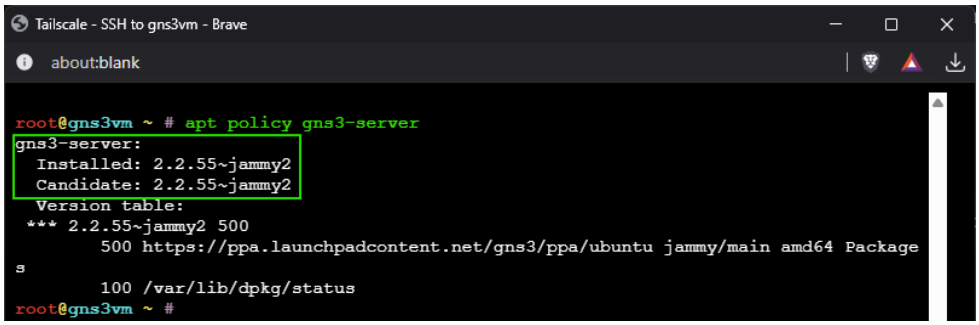


Figure 25 - Confirming GNS3 Software Version

The official website only offers the latest version, which is 2.2.57. To avoid compatibility issues, download the version that matches the server from the GitHub releases page. Once there, in the assets section, click on version 2.2.55 and download it as shown in Figure 26 .

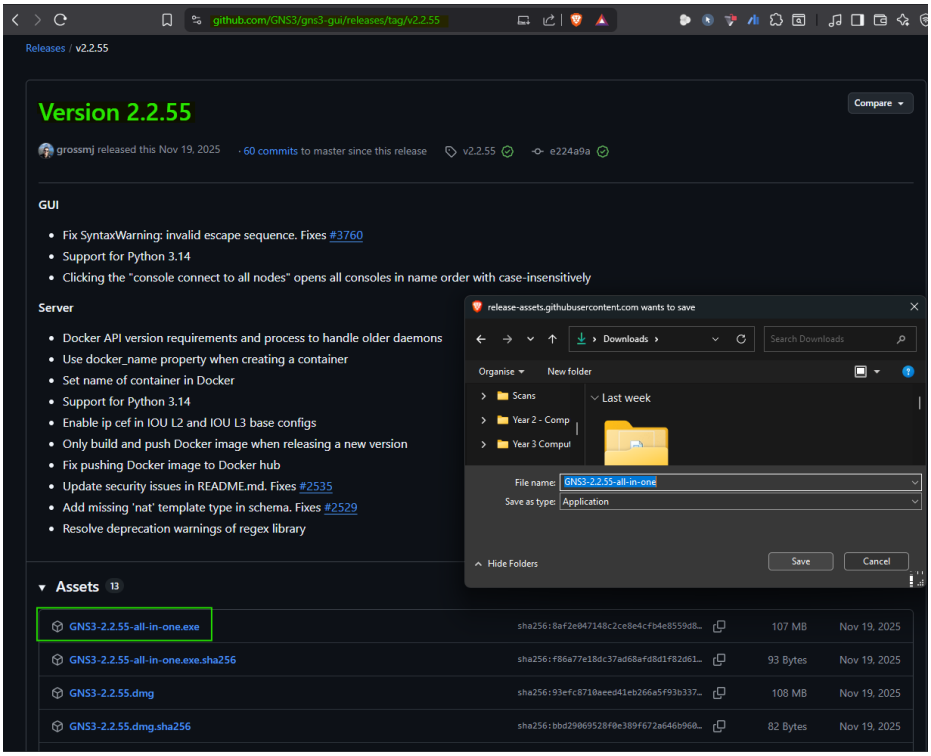


Figure 26 - Downloading GNS3 GUI to the Local Machine (Laptop)

Next, the downloaded installer must be run. The installation is pretty straightforward; just accept the license agreement and click Next through the standard options until it completes.

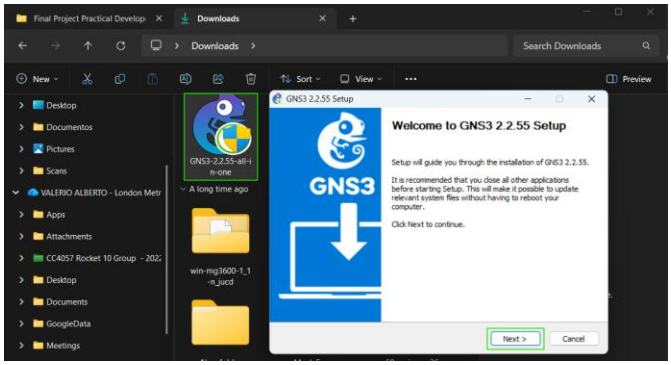


Figure 28- Opening the Download Folder and Clicking the Installer

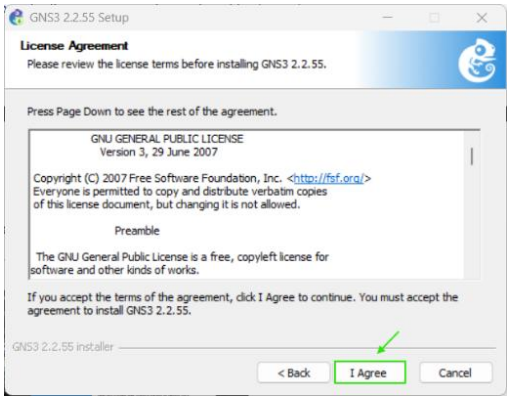


Figure 27 - GNS3 GUI Installation License Agreement

The next screen is for naming the 'Start Menu Folder' and selecting the required tools and components. The folder name is kept as GNS3, while the component tools are briefly described in Table 5.

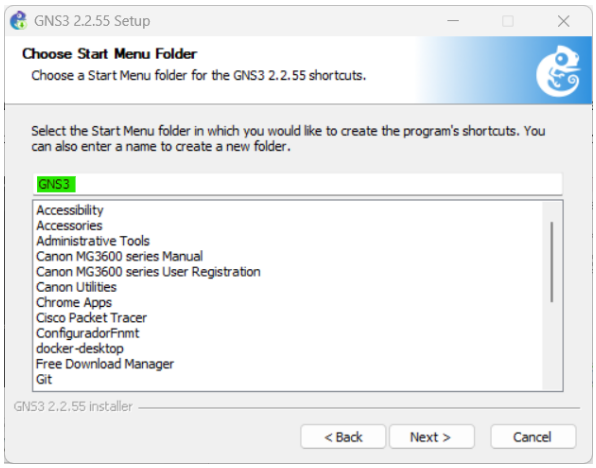


Figure 30 - GNS3 GUI's Installation Start Menu Folder

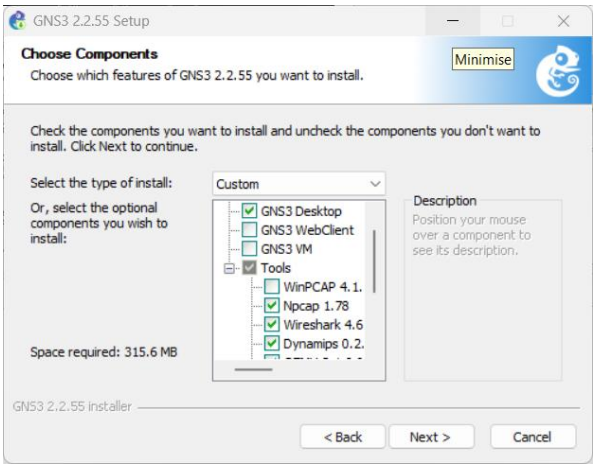


Figure 29 - GNS3 GUI's Component Tool Installation

Table 5 - GNS3 GUI's Installation Component Tools

Components	Type	Description
WindPCAP	Required	Enables GNS3 to interface with the host machine's network, allowing emulated devices to communicate with external systems.
Npcap	Optional	Modern replacement for WinPCAP. Recommended for newer Windows versions, with automatic compatibility based on the operating system.

Wireshark	Recommended	Provides detailed packet capture and inspection to analyse traffic between simulated network devices.
Dynamips	Required	Supports the execution of GNS3 directly on the host system for legacy Cisco device emulation.
QEMU 3.1.0 and 0.11.0	Optional	Set up a virtual machine and use it to run GNS3. Rather than relying on this component, consider using GNS3VM to execute GNS3 within a virtualised environment.
VPCS	Recommended	Lightweight virtual endpoints used for basic network testing and troubleshooting commands.
cpulimit	Optional	A utility that restricts QEMU CPU usage to prevent resource saturation on the host system.
GNS3	Required	Core application responsible for launching and managing all GNS3 simulations.
TightVNC	Recommended	Provides graphical access to virtual appliances via a VNC client interface.
Solar-Putty	Recommended	Primary console tool used by GNS3 to access device command-line interfaces
Virt-viewer	Optional	Optional QEMU visual interface providing an alternative graphical display for virtual machines.
HAXM	Optional	An Intel-based hardware acceleration module that improves virtualisation performance when supported.

The next screen is for selecting the installation folder location. The default location was maintained.

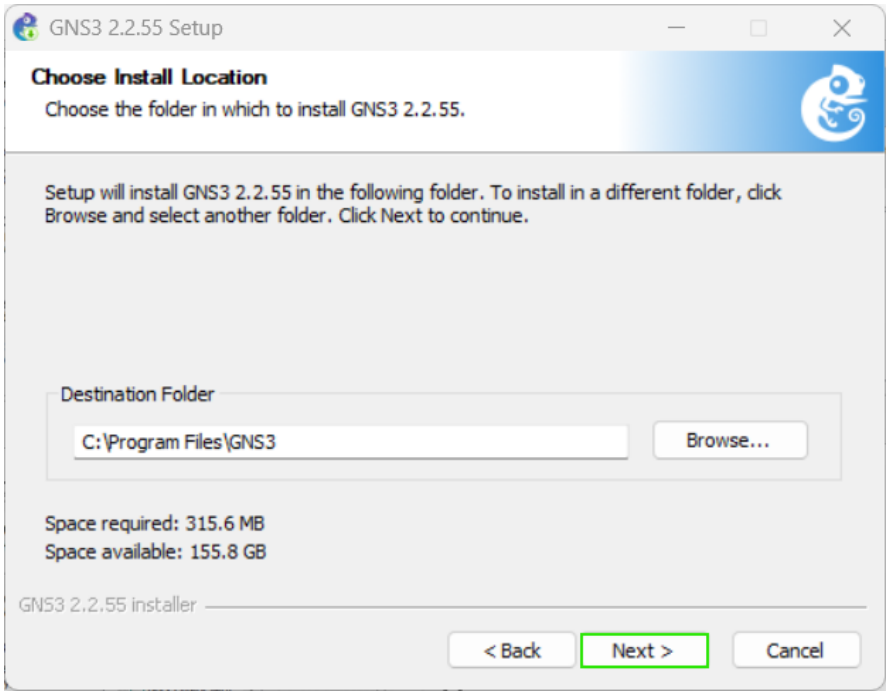


Figure 31 - GNS3 GUI's Install Folder Selection

Figure 30 shows the GNS3 installation process along with all the component tools selected in Figure 29

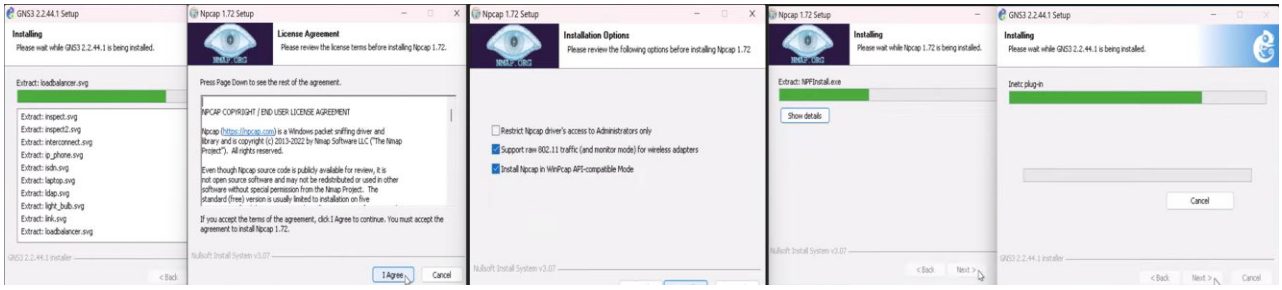


Figure 32 - GNS3 GUI & Component Tools Installation Process

The Wizard's final screen allows the GNS3 GUI to be launched immediately after installation, as illustrated in Figure 33

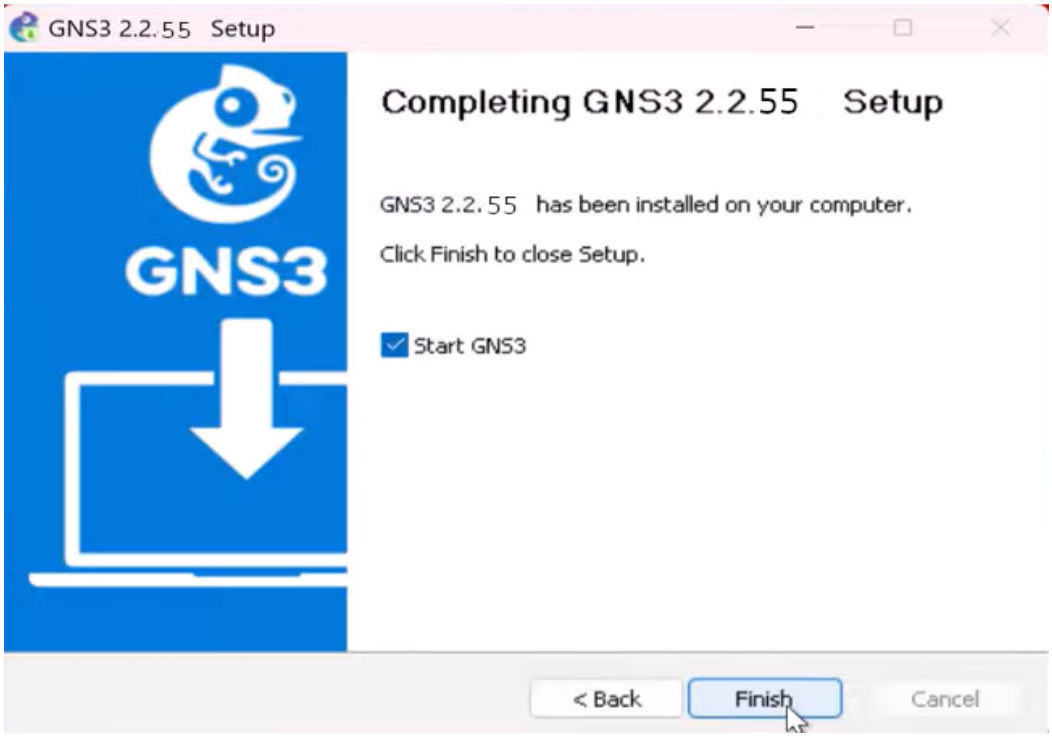


Figure 33 - GNS3 GUI installation completed.

5.5.1 Configuring the GNS3 Client for Remote Server Use

GNS3 launches and displays the setup wizard. This confirms a successful installation. The wizard then guides the initial GNS3 server configuration by selecting the ‘Run the appliances on a remote server’ option and clicking ‘Next’, as shown in Figure 34.

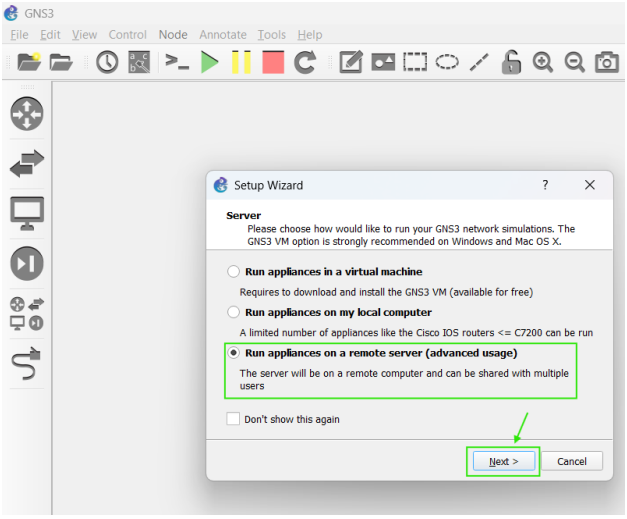


Figure 34 - GNS3 GUI's Wizard Set Up

The Wizard displays the remote server configuration screen as Figure 35 illustrates, where the private Tailscale IP, port number, and the credentials created in Step 4 Figure 24 are provided to establish the connection between the local machine and the server.

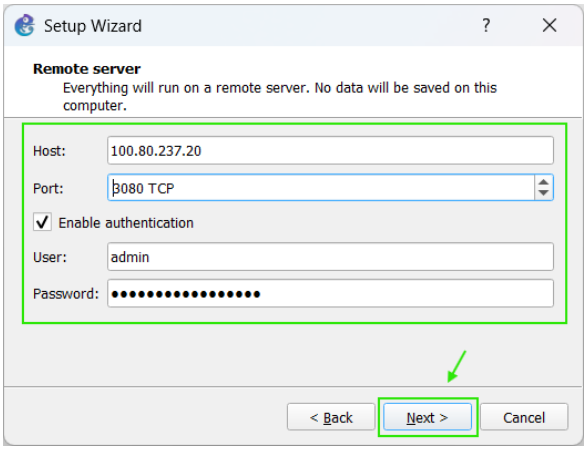


Figure 35 - GNS3 GUI's Credential, Host and Port

After completing the previous steps, the server engine initiates the authentication process and allows the local machine to log in using the previously defined credentials. Figure 36 shows the green 'Main server' indicator confirming that the infrastructure setup has been successfully completed.

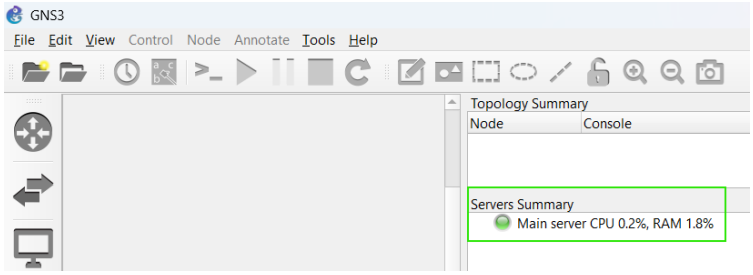


Figure 36 - Server Summary

5.6 Installing Tailscale on the Laptop

To establish an encrypted tunnel between the server and the local machine, it is essential to install Tailscale on the laptop. Simply by signing in using the same account used on the server, as explained in Figure 16.

This process begins by downloading Tailscale from the official website, running the installer, and waiting for the setup to complete, as shown in Figure 37 and Figure 38.

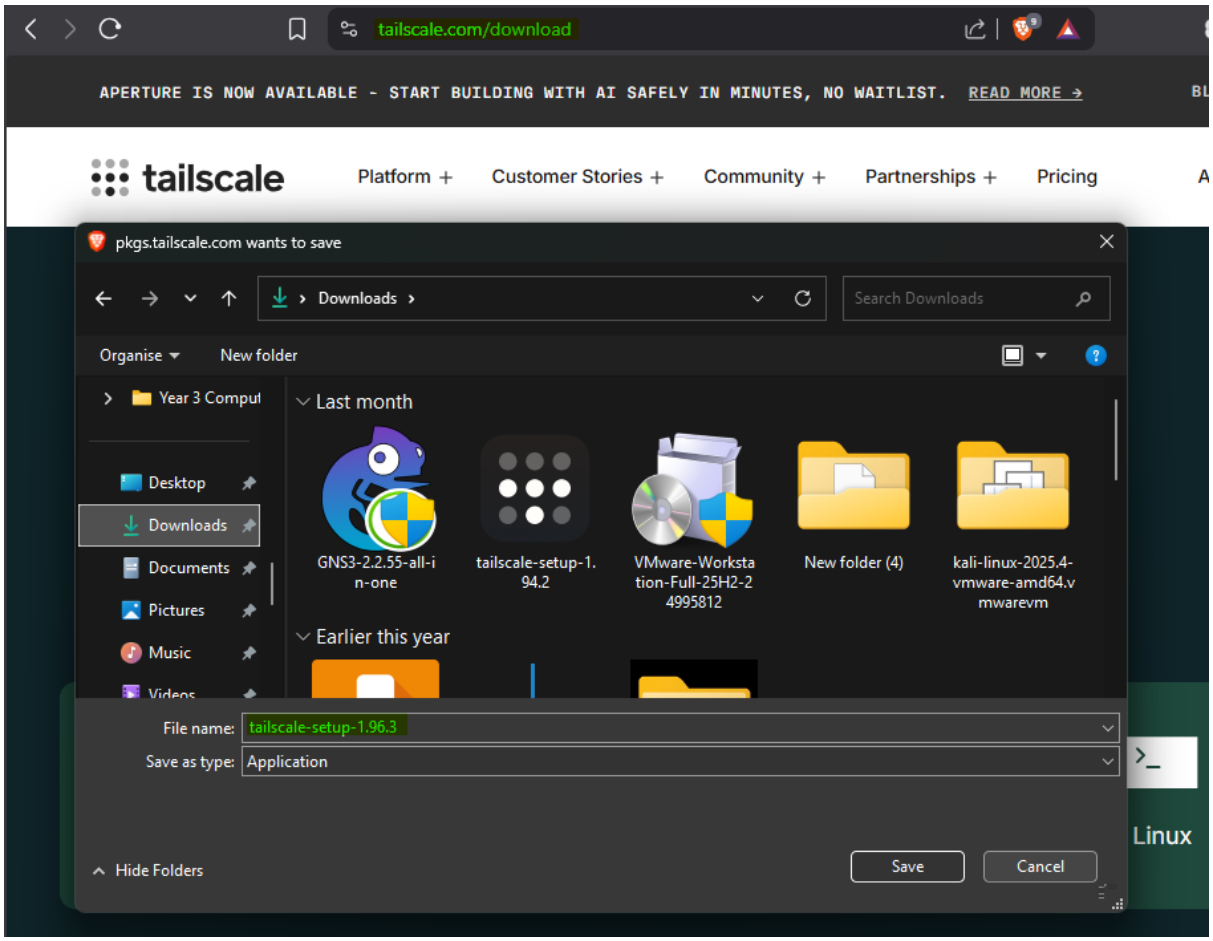


Figure 37 - Downloading Tailscale for the Local Machine

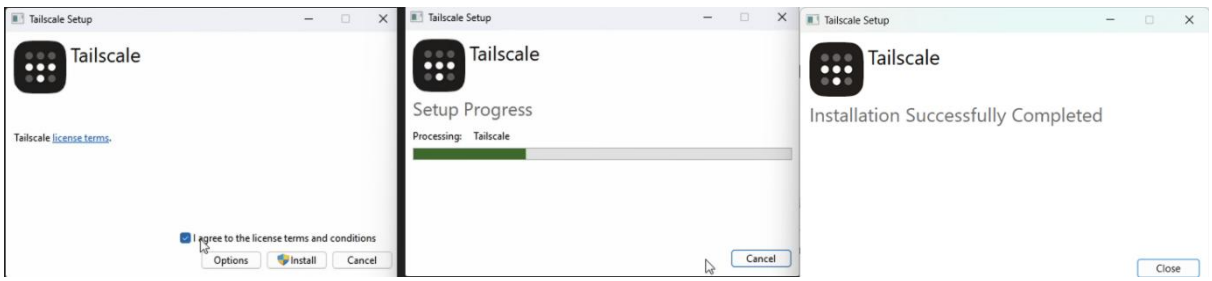


Figure 38 - Tailscale Installation Process on the Local Machine

5.6.1 Local Machine Integration and Tailscale Authentication

Once the installation is completed, the local machine must be authenticated so it can join the Tailscale network and establish the encrypted tunnel between the server and the laptop.

The authentication process is very straightforward, select get started and sign in to the network as shown in Figure 39 and Figure 40

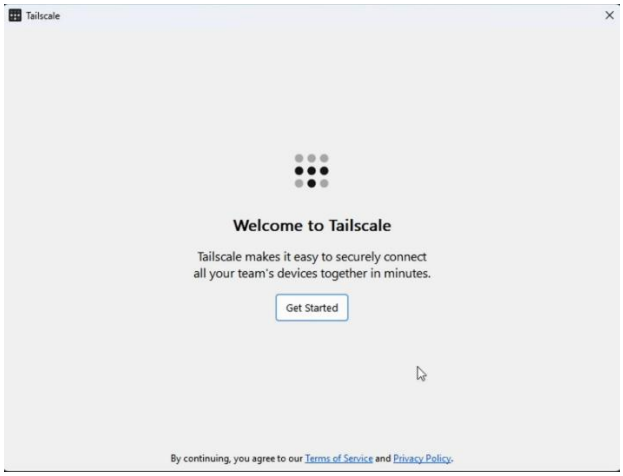


Figure 40 - Get Tailscale Authentication Started on The Local Machine

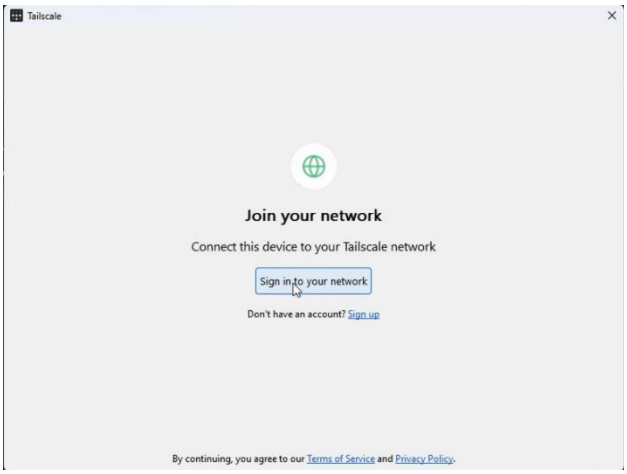


Figure 39 - Sign in to Tailscale Network

To complete the authentication, it is essential to provide the same login credentials used in Step 3 section 5.3.1, as Figure 16 illustrate. The steps include selecting an email account to log in, and Tailscale will link the local machine to the Tailscale account, and the tunnel will be successfully established, as shown in

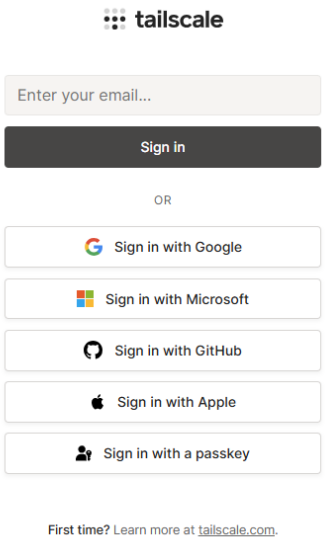


Figure 41 - Authenticating the Local Machine to Tailscale Network

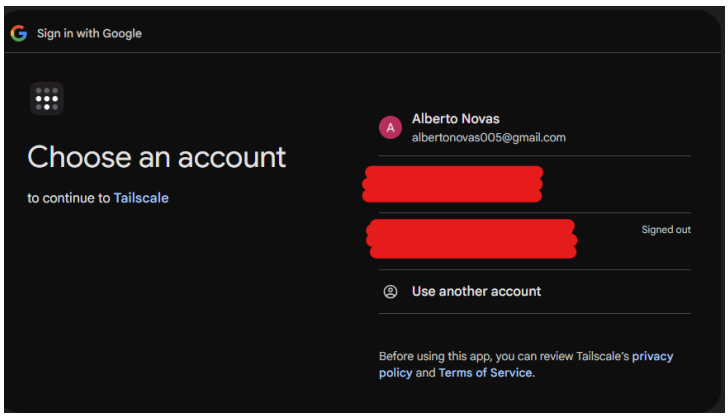


Figure 42 - Selecting Email Account for Local Machine's Tailscale Authentication

Once the authentication process is completed, Tailscale allows the remote server and the local machine to communicate and establish the encrypted tunnel between them, as Figure 43 illustrates.

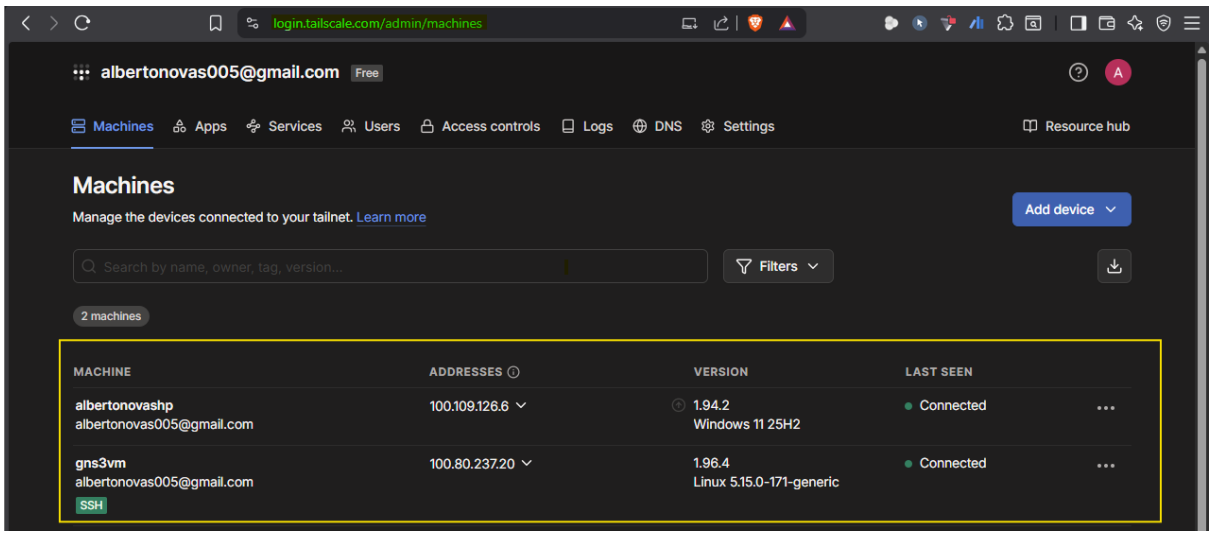


Figure 43 - Server and Local Machine Added to Tailscale Network

5.7 Creating the Invisible Server Strategy

This step strengthens the server security posture by removing all public access rules from the Hetzner Cloud Firewall, effectively placing the server into a hidden state. The server remains fully operational via Tailscale, yet appears unreachable, essentially a dead IP to anyone scanning the public internet.

By logging in to Hetzner Robot, navigating to the dashboard, and selecting the firewall tab, as shown in Figure 44, all firewall rules applied to the server are displayed.

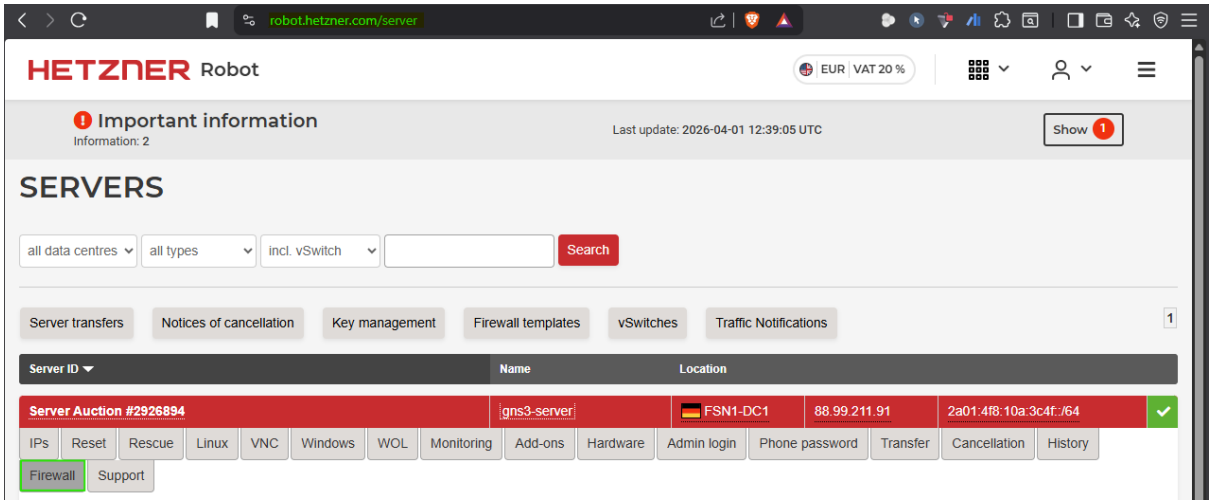


Figure 44 - Hetzner Server Dashboard

Figure 45 shows that the server is currently highly vulnerable due to the first rule, named "Allow All" with "Source IP" as **`0.0.0.0/0`**, meaning every IP in the world matches as an entry, and the rule action is tagged as "Accept". This means that not only is SSH port 22 exposed, but every port of the server is publicly accessible, posing a critical risk to the system's security and integrity.

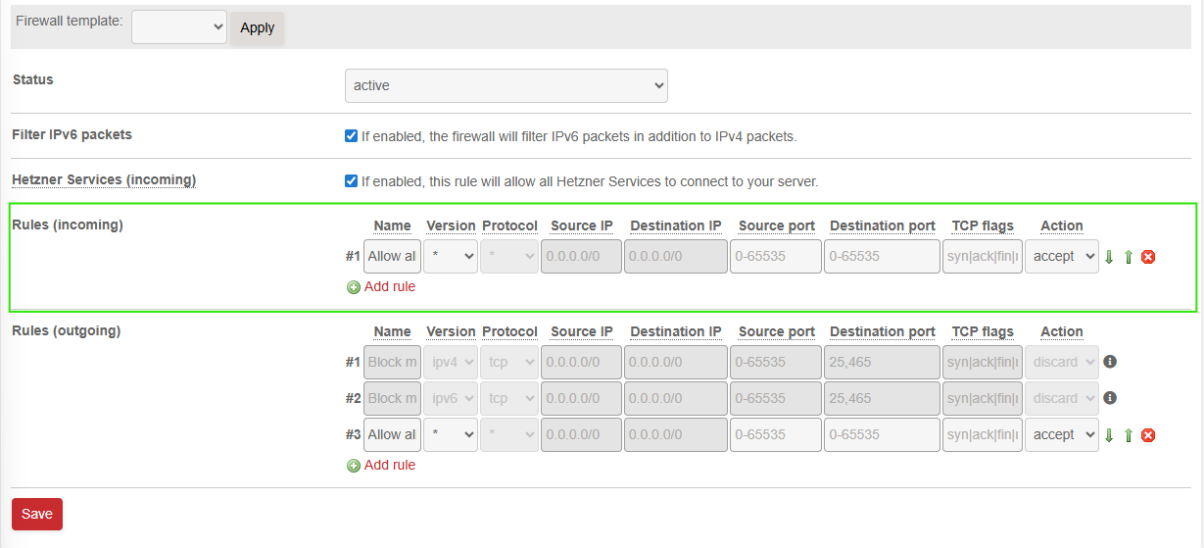


Figure 45 - Weak Server's Firewall Rules

To achieve the "Invisible" configuration strategy, the predefined firewall rule shown in Figure 45 must be removed and replaced with one that supports the Tailscale handshake. According to Tailscale (2026), WireGuard tunnels operate over UDP using port 41641, the firewall should be configured accordingly. Since firewalls read rules from top-to-bottom. Placing 'Blocks' for sensitive management ports first ensures they remain unreachable during scans, while Tailscale's background traffic will survive. The Hetzner incoming rules should be configured in this exact order:

```
Rule #1 (Block Publick SSH): * Protocol: TCP, Source: 0.0.0.0/0, Destination Port: 22, Action: Discard

Rule #2 (Hide GNS3 API/Web): * Protocol: TCP, Source: 0.0.0.0/0, Destination Port: 3080 Action: Discard

Rule #3 (Allow Tailscale / Background Traffic): * Protocol: *, Source: 0.0.0.0/0, Ports: 0-65535, Action: Accept
```

By blocking traffic on port 22 and the GNS3 port 3080 before the firewall reaches the 'Accept' rule, public scans will see services as 'Down'. Meanwhile, Tailscale's UDP handshakes, ICMP (ping), and DERP relay traffic continue to pass through Rule #3 and connect normally (Tailscale, 2026).

The Figure 46 illustrate the implementation of the rules on the server's firewall.

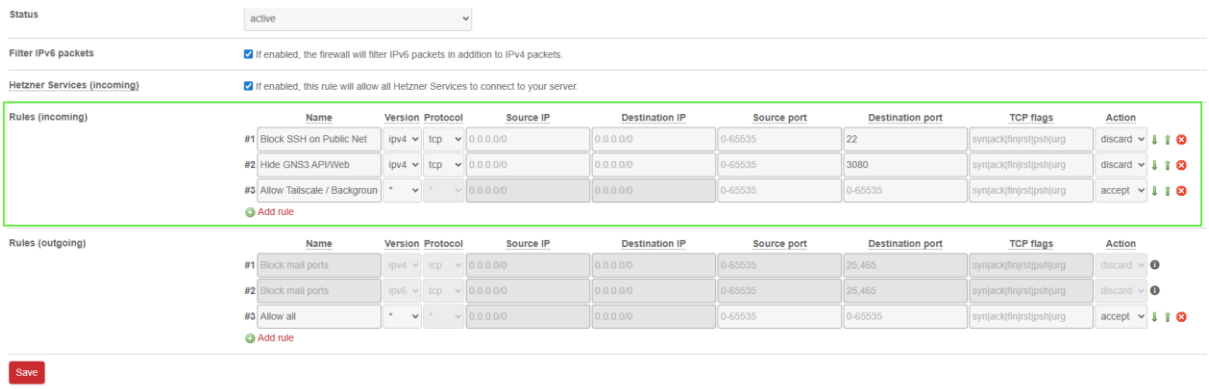


Figure 46 - Invisible Server Firewall's Rules

5.7.1 Testing Server Security Invisibility Strategy

By successfully establishing an SSH session through the Tailscale private IP, while the public server IP times out through port 22, as shown in Figure 47 the effectiveness of the 'Invisible Server' concept is confirmed. The Hetzner firewall now effectively hides the dedicated server, allowing only authenticated, encrypted traffic routed via the private Tailscale network.

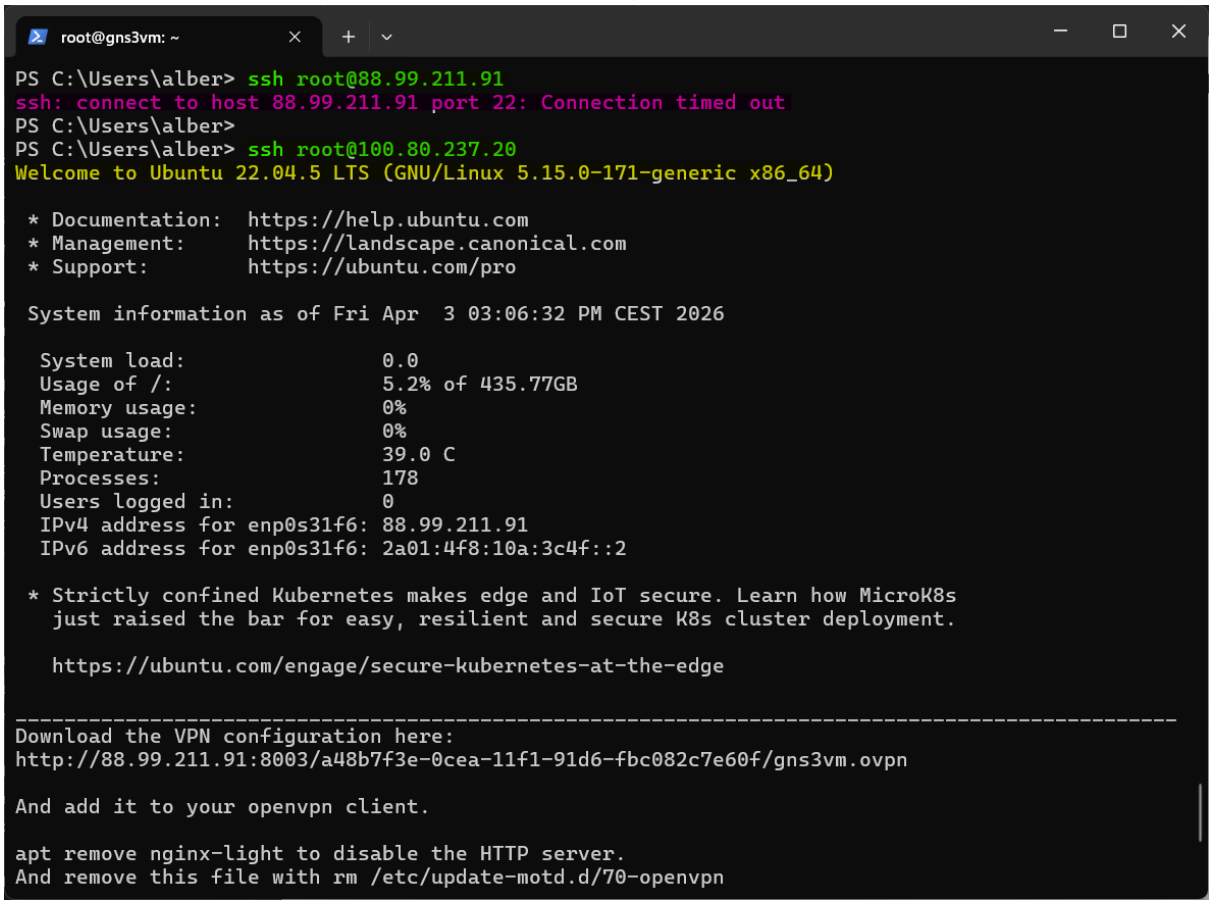


Figure 47 - Testing Public & Private IP for SSH Session

5.7.2 Testing SSH Session From Tailscale Dashboard

SSH sessions can also be established directly from the Tailscale dashboard. Since the server is fully integrated into the Tailnet, Tailscale provides a secure and convenient method to connect from anywhere by clicking the three dots on the gns3mv tab and selecting ‘SSH to machine’, as shown in Figure 48.

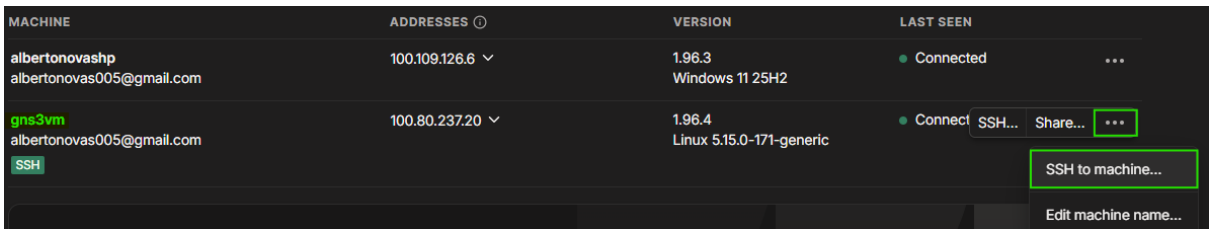


Figure 48 - SSH the Server from the Tailscale Dashboard

Next, select the local username as ‘root’ and click on ‘Start SSH Session,’ followed by completing the authentication process shown in Figure 50, Figure 49 and Figure 51.

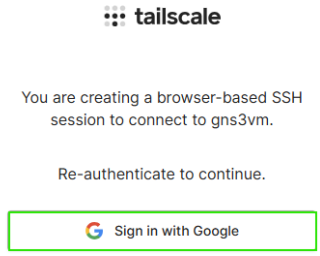


Figure 50 - Starting Authentication for SSH

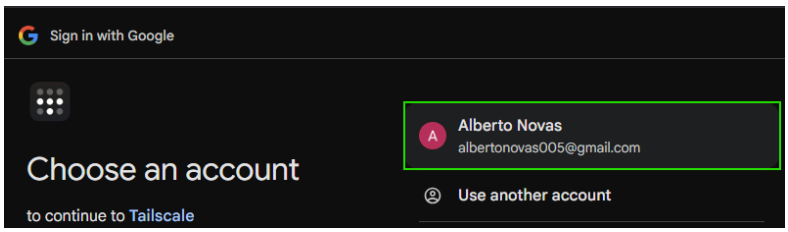


Figure 49 - Selecting User as Root to Start SSH Session

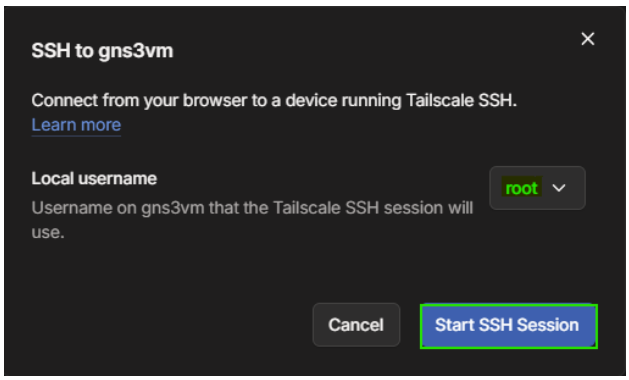


Figure 51 - Selecting Account for SSH Authentication

Once authentication is completed, Tailscale connects to the server through the private WireGuard tunnel, providing full access while isolating the session from the public network. The Figure 52 shows the server console once authentication is completed.

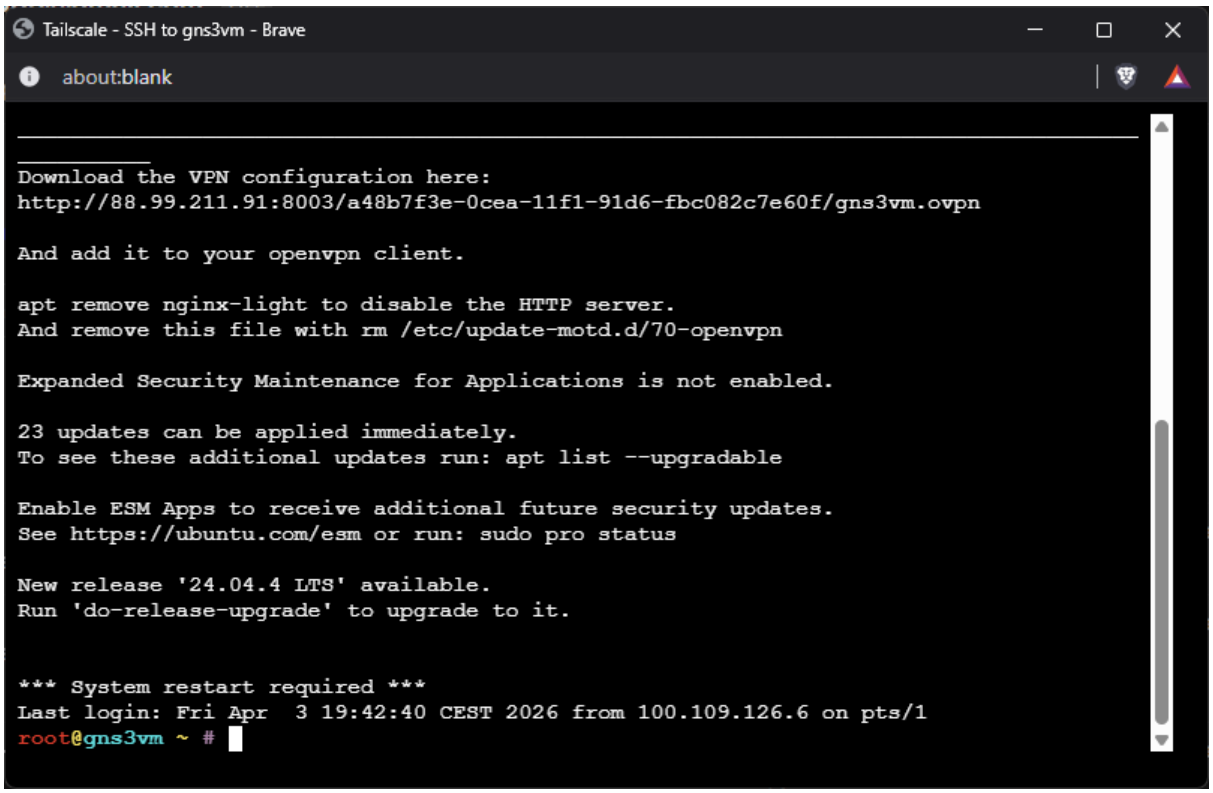


Figure 52 - SSH Session Establishes through the Tailscale Dashboard

5.8 Uploading, Configuring Network Devices and VMs Images to GNS3 for Network Deployment

This section details the steps required to import the images into the server, enabling topology construction and the execution of SYN flood and ICMP attack simulations necessary to produce the final experimental results.

5.8.1 Importing Core Router Image

As discussed in the Section 2.3, this project evaluates CoPP and ZBF, which require a specific virtual modern router. The Cisco CSR1000v (or Catalyst 8000v) is therefore used. This section outlines the steps required to upload it to the GNS3 server.

5.8.2 Creating the Core Router Qemu Template

To create the virtual router profile, in GNS3, from the launch bar, select ‘Edit’ and then ‘Preferences’. In the left navigation pane, expand ‘Qemu’ and select ‘Qemu VMs,’ then click on ‘New’, as illustrated in Figure 53 (GNS3 Documentation, 2026).

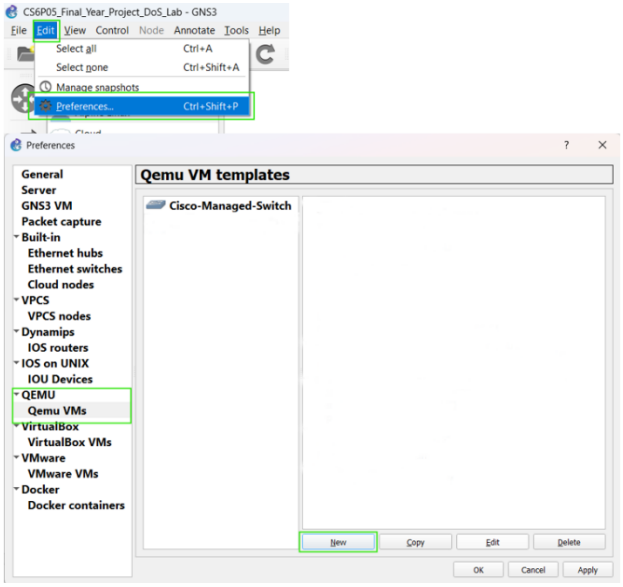


Figure 53 - GNS3 Preferences Window Showing Navigation to Qemu VMs

5.8.3 Selecting the Cisco IOS .qcow File from the Directory

The next step is to select the image file from the directory by clicking ‘New Image’, ‘Browse’ and choosing the directory containing the Cisco IOS **`.qcow2`** file and clicking ‘Open’. GNS3 validated the image, and the GUI performed an automatic validity check and began importing the image to the server (GNS3 Documentation, 2026).

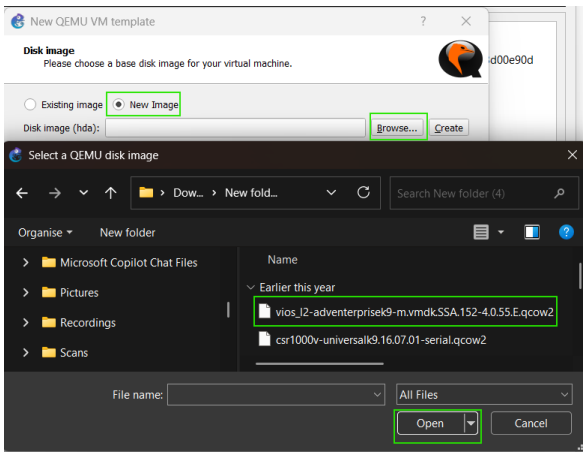


Figure 55 - Selecting Router Image From the Directory

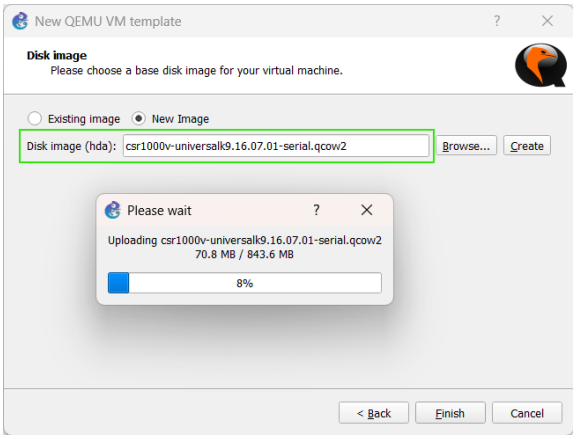


Figure 54 -Router Image Status Bar Progress

5.8.4 Configuring Name and selecting Qemu Binary

Assigning a clear device name enhances readability, while selecting the correct Qemu binary is essential to ensure the emulation server can successfully launch and manage the virtual Router (GNS3 Documentation, 2026).

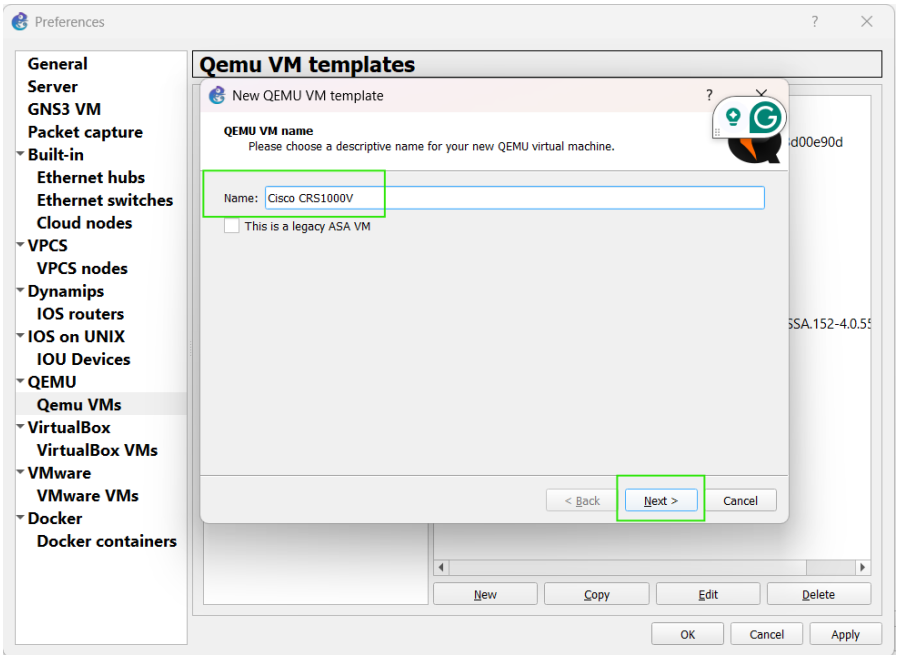


Figure 56 - Providing a Name to the New Router Template

The standard x86_64 enables GNS3 to launch the VMs and Virtual network devices that behave as real physical machines (GNS3 Documentation, 2026).

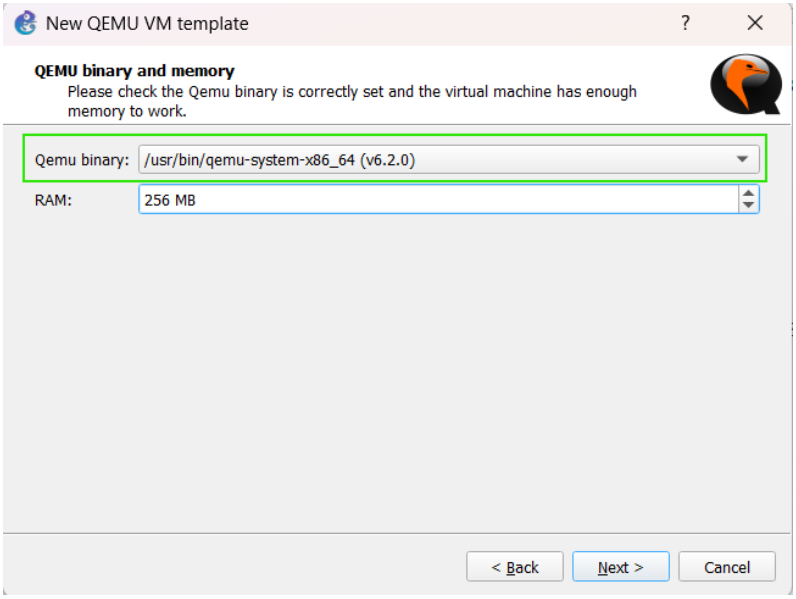


Figure 57 - Selecting the Standard x86_86(v6.2.0) as Qemu Binary

5.8.5 Ram Allocation and Selecting Telnet as Device Connection Type

Allocating sufficient RAM to the CRS1000v router is critical, as Cisco IOS-XE is a resource-intensive operating system (GNS3 Documentation, 2026). With 64GB of RAM on the Hetzner server, a generous RAM allocation helps prevent instability and crashes during attacks. 4096 MB (4GB) has been allocated, as shown in Figure 58. Telnet is the default console type for Qemu templates in GNS3, providing a lightweight, text-based serial connection ideal for network devices.

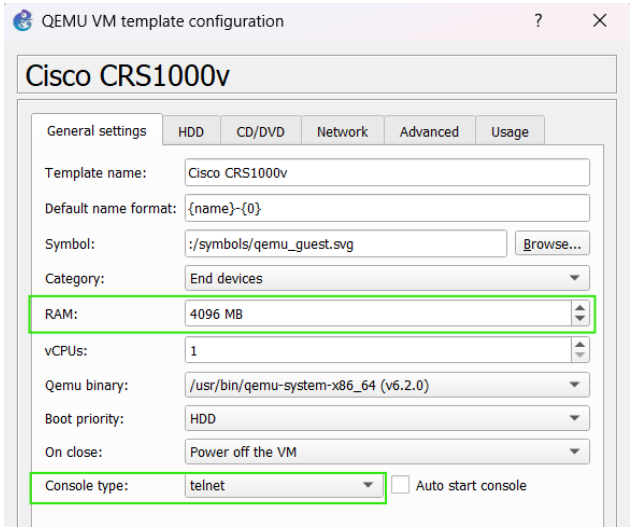


Figure 58 - Allocating the Router's RAM and Telnet as Connection Type

Once the router Qemu template is created and the configuration shown in Figure 59 is applied, the virtual router is ready for deployment. The figure highlights the general configuration displayed on the right-hand side.

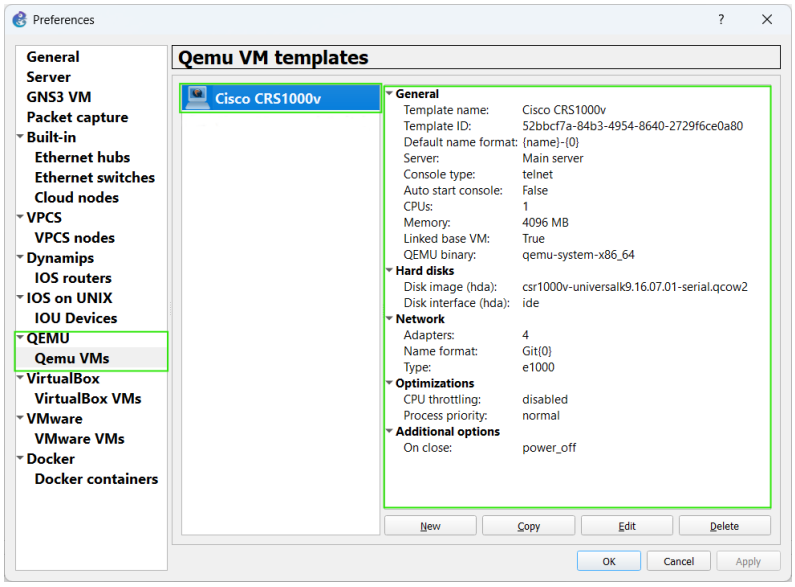


Figure 59 - Router Qemu Template for Successfully Created

5.8.6 Creating the Qemu Template for the Layer-2 Switch

The image import and Qemu template configuration process employed on the CRS1000v router (Section 5.8.1 - 5.8.5) was identical for the Cisco IOSvL2 switch. The only difference is the RAM allocation for the Cisco switch, set to 1024 MB (1GB), as shown in Figure 60, reflecting its lightweight requirements.

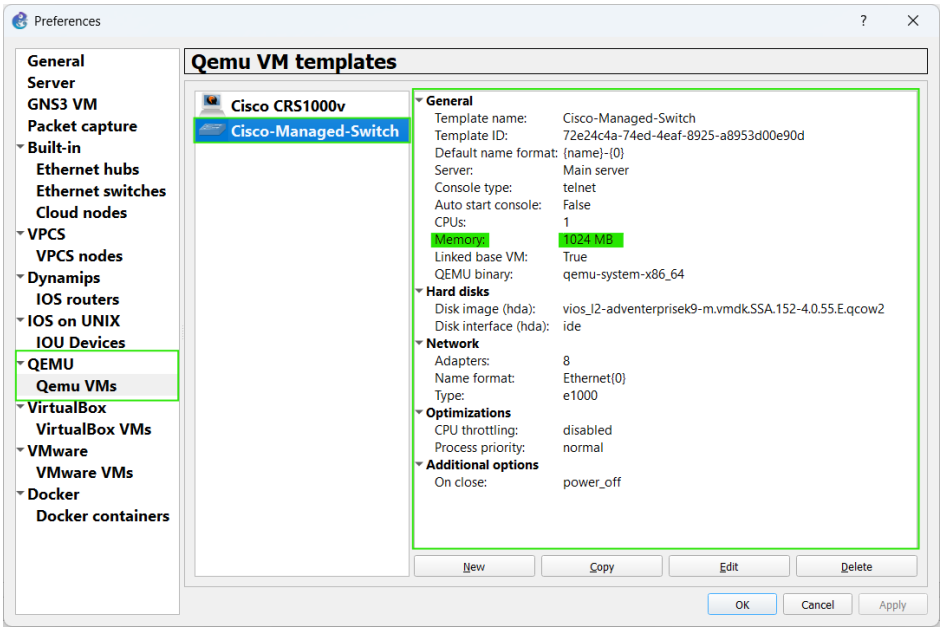


Figure 60 - Switch Qemu Template for Successfully Created

5.8.7 Importing Ubuntu-Server

The process for importing the image and configuring the QEMU template on the Ubuntu server follows the same steps used for the CRS1000v router, which Section 5.8.1 through 5.8.5 already covers in detail.

The only real difference lies in the memory allocation. Researchers assign the Ubuntu Server just 1024 MB (1 GB) of RAM. This modest amount suits its relatively lightweight nature perfectly and avoids wasting resources.

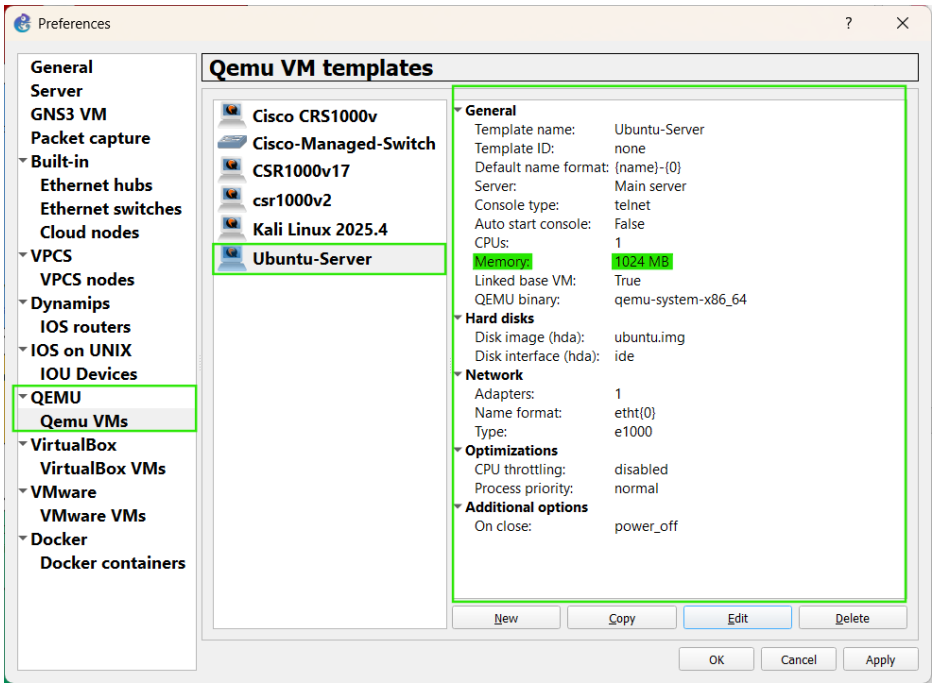


Figure 61 - Ubuntu-Server Template Created

From the left-hand sidebar, select ‘End Devices’ to view the Ubuntu-Server node, confirming that it is ready for deployment, as shown in Figure 62.

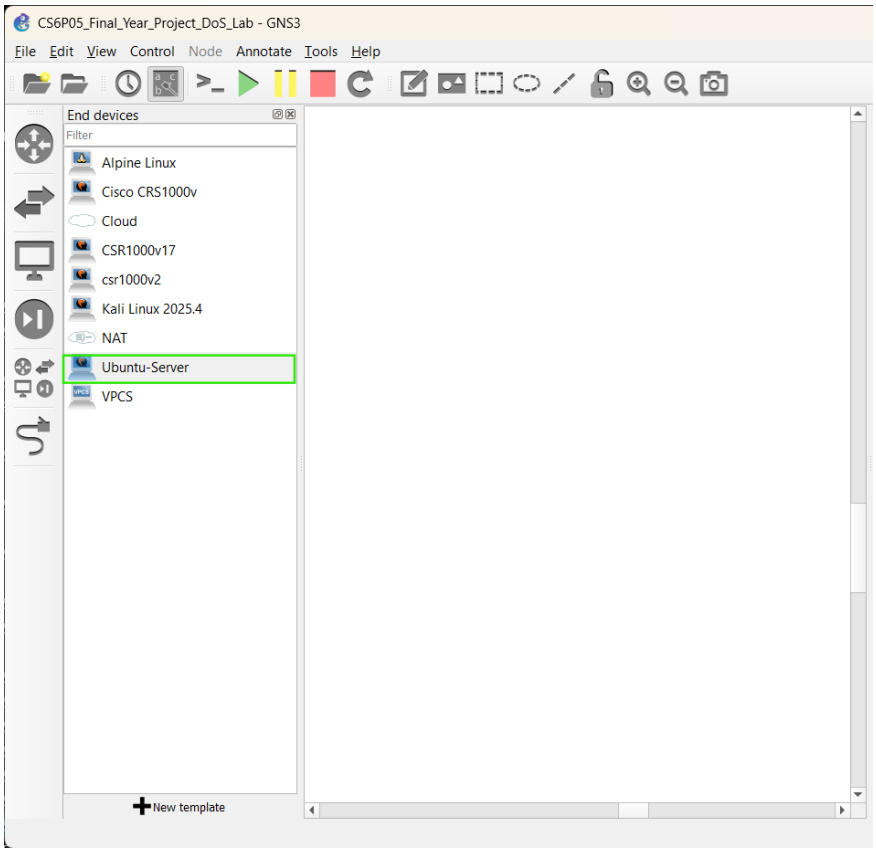


Figure 62 - - Linux Ubuntu-Server Node is Ready for Deployment

5.8.8 Creating a Qemu Template for Kali Linux

As outlined in Section 2.4 the Kali Linux VM is a core component of this project, serving as the primary attacker node, providing access to a wide range of network inspection and attack-simulation tools. Its import, installation, template creation and configuration mirror the RCS1000v process described in sections 5.8.1 - 5.8.5, with the only adjustment being an allocation of (4GB) RAM and two CPU cores to support improved performance.

Figure 63 shows the GNS3 'Preference' menu for the Kali Linux VM, confirming that it is correctly configured, fully integrated with the server, and ready for deployment.

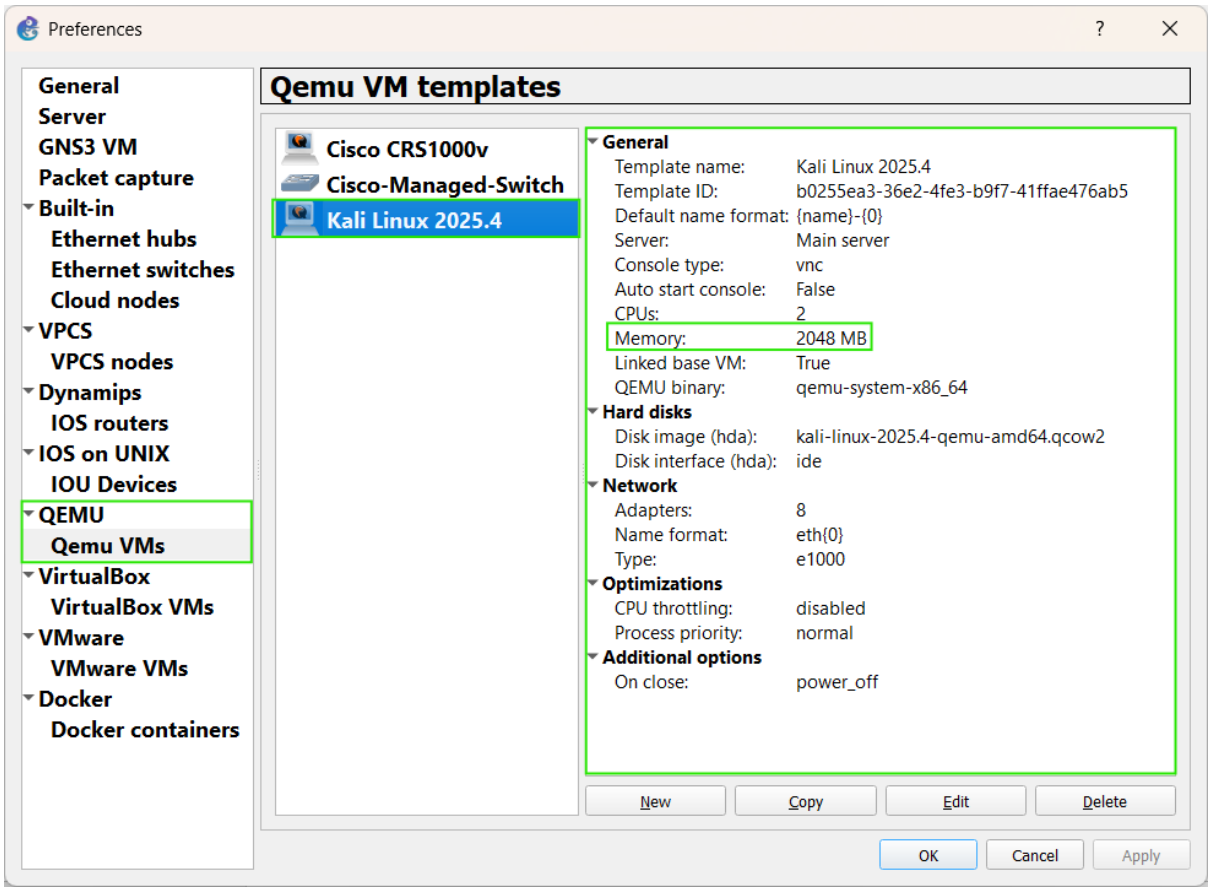


Figure 63 - GNS3 Preference Menu for Kali Linux Shows Specification

5.9 Network Topology Technical Configuration

Once the virtual appliances are uploaded and configured into the GNS3 hypervisor, the network engineering work begins. This configuration process unfolds as an iterative cycle divided into

four clear phases: Environment Orchestration, Network Configuration, Attack Execution, and Mitigation Integration.

Each phase builds directly on the results of the previous one. This step-by-step structure creates strong traceability, makes the entire setup easy to reproduce, and keeps every action firmly aligned with the project's core objectives.

5.9.1 Phase 1: Environment Orchestration

The simulation environment was physically instantiated on the canvas. The devices were dragged from the device toolbar menu to the GNS3 workspace and then cabled in strict accordance with the Hierarchical Three-Tier Architecture with Centralised Core outlined in Section 4.4.1.

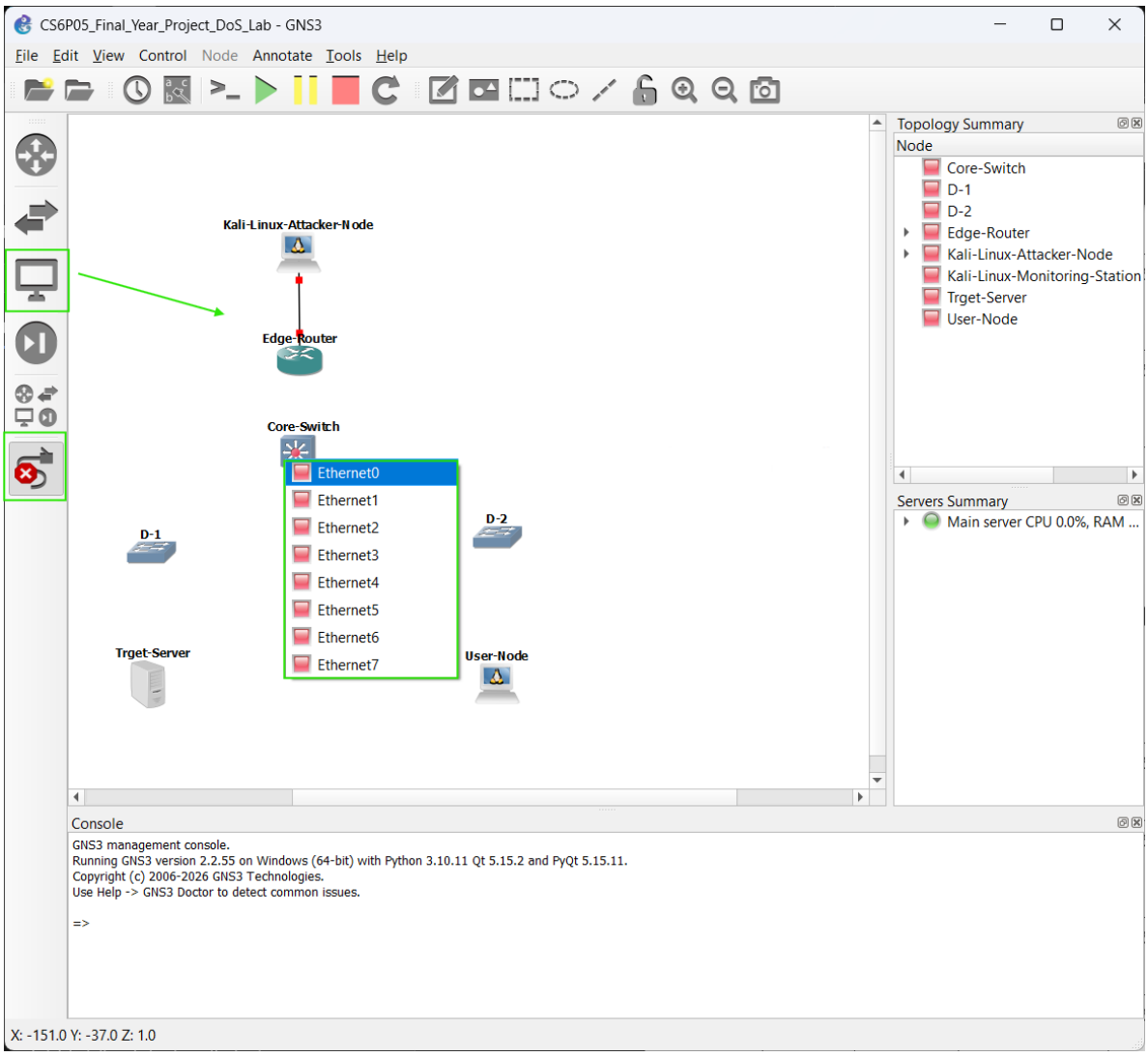


Figure 64 - Building the Hierarchical Three-Tier Architecture with Centralised Core

The Figure 65 illustrates the completed network topology implemented in GNS3, following a layered distribution and access architecture, as stated in Figure 3. The ‘Topology Summary’ panel in the upper-right corner confirms that all nodes are in an operational state as indicated by the green lights, while the ‘Server Summary’ panel indicates stable resource utilisation with CPU usage at 14.3% and RAM usage 13.6 %.

These metrics and all nodes powered on demonstrate that the Hetzner bare-metal server, the Tailscale tunnel, and the QEMU hypervisor are all operating correctly, establishing a stable foundation for the subsequent configuration phases outlined in Figure 2.

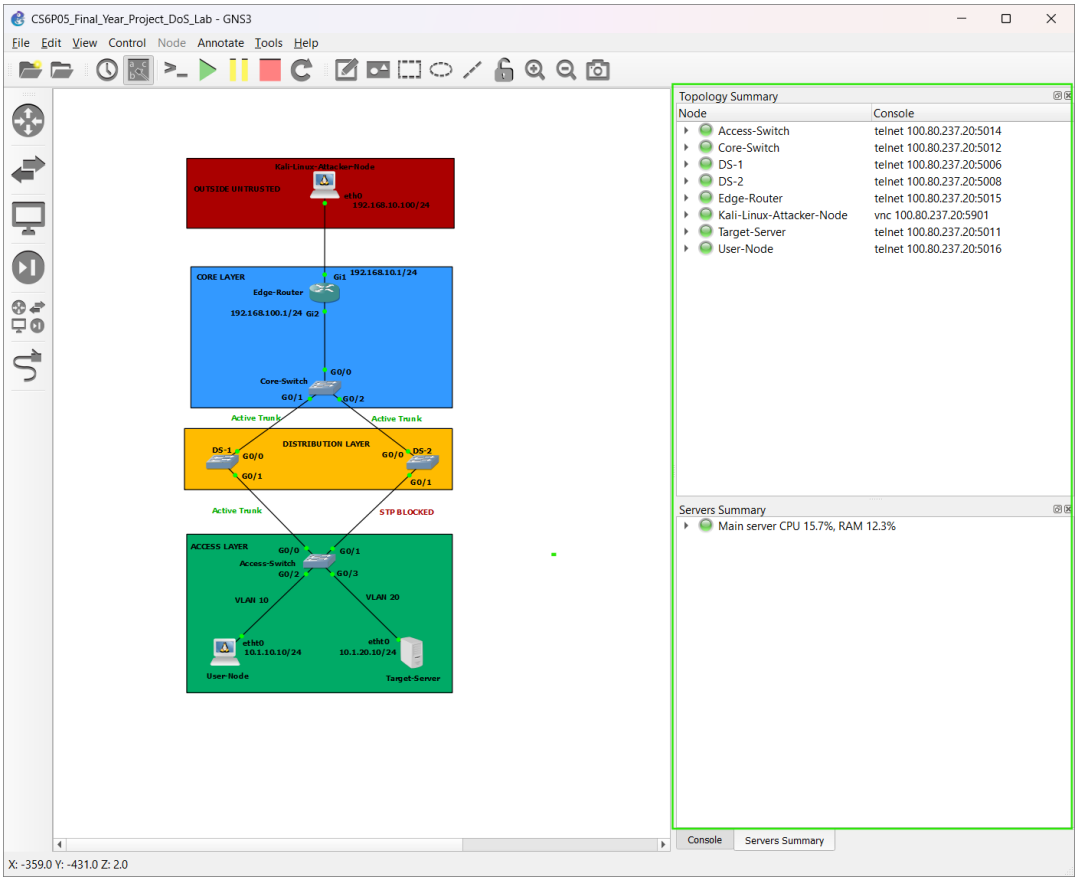


Figure 65 - Completed GNS3 Network Topology with All Nodes in Operational State

5.9.2 Phase 2: Network Configuration

This phase outlines the configuration of policies, controls, and data flows that enable secure communication across the network. All configurations were executed via Cisco IOS Command Interface (CLI).

5.9.2.1 Core-Switch Configuration

The Core Switch performs three key roles: Layer 3 routing nexus, internal default gateway, and Spanning Tree Primary Root Bridge. Layer 3 routing on the Core Switch has been enabled with the **`ip routing`** command. They then created Switched Virtual Interfaces (SVIs) for VLAN 10 and VLAN 20. This configuration makes the Core Switch the default gateway for all internal endpoints.

The design delivers wire-speed routing for traffic between internal networks. It also avoids hypervisor-related 802.1Q encapsulation issues. At the same time, the Core Switch serves as the Spanning Tree Primary Root Bridge, which maintains a stable, loop-free Layer 2 topology.

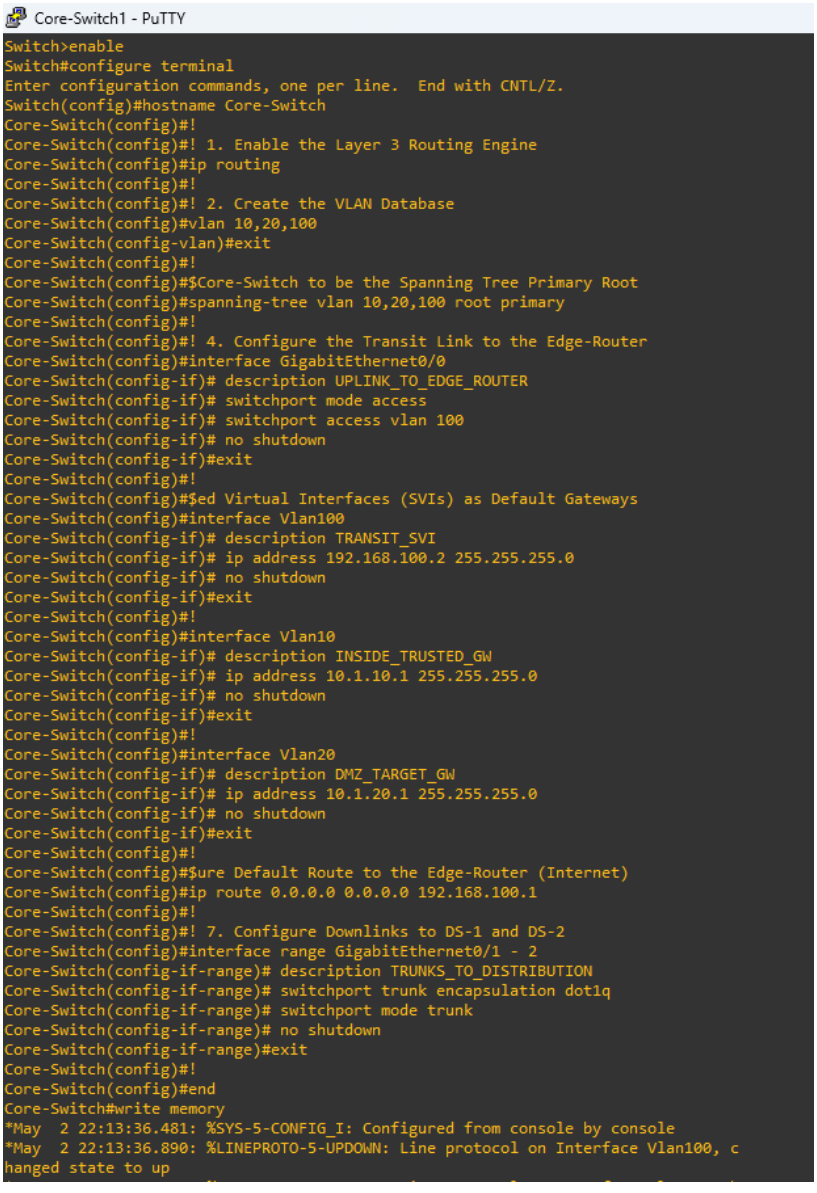


Figure 66 - Core-Switch Configuration Commands

5.9.2.2 DS-1 and DS-2 Configuration

These switches provide Layer 2 aggregation and high-availability redundancy. DS-1 and DS-2 operate purely at Layer 2 and form the resilient Distribution tier.

DS-1 was configured as the secondary STP root to guarantee predictable failover. Rapid PVST+ continuously calculates the most efficient paths across the network. It automatically blocks the redundant link from DS-2 to the Access Switch. This prevents dangerous broadcast storms while keeping the topology stable and efficient.

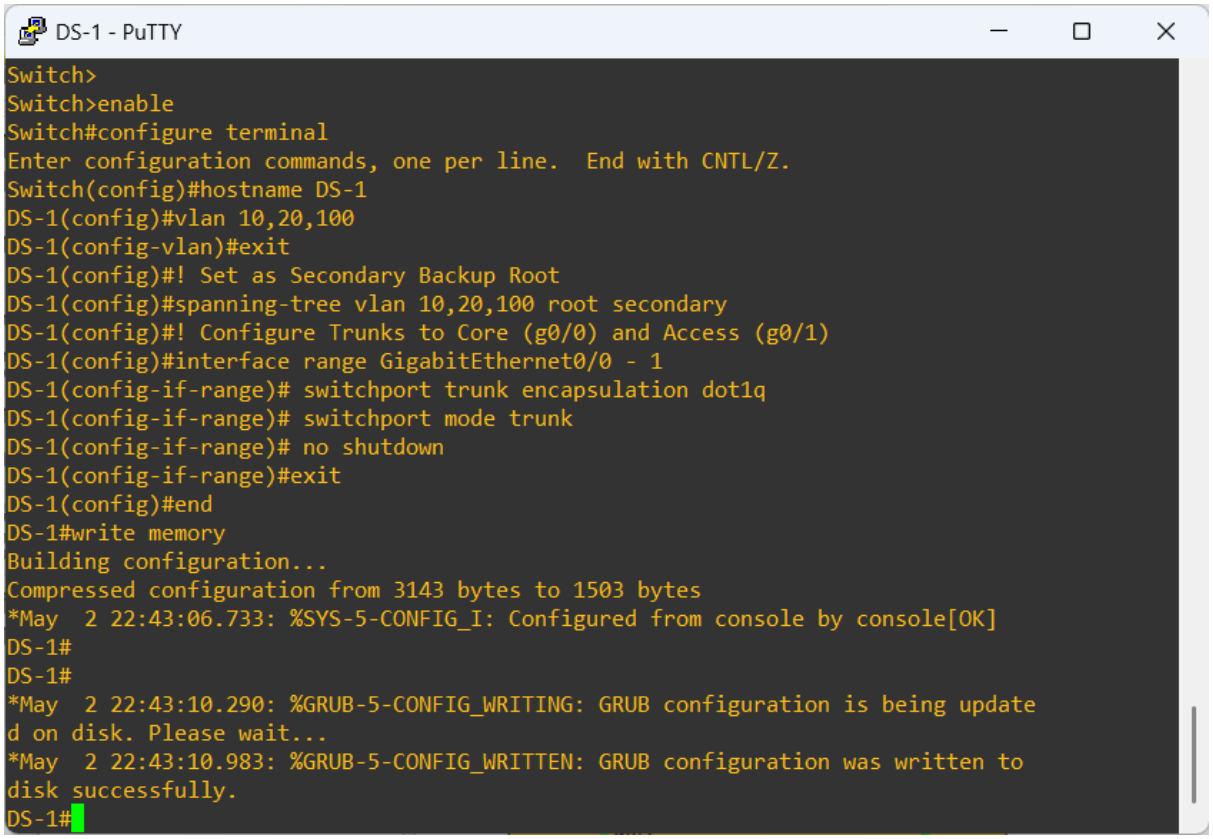


Figure 67 - DS-1 Cisco Configuration Commands

DS-1 Configuration as Standby

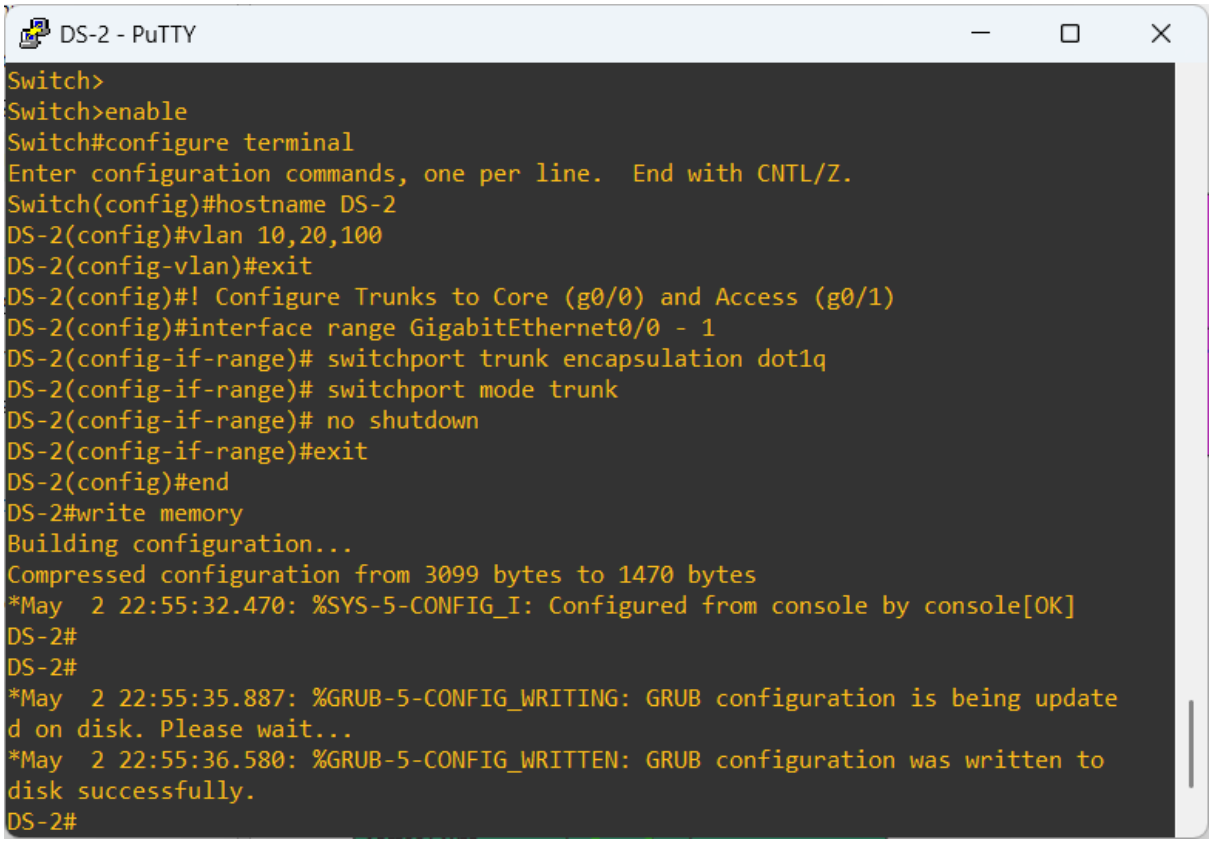


Figure 68 - DS-2 Cisco Configuration Commands

5.9.2.3 Access-Switch Configuration

The Access Switch handles endpoint port assignment and Layer 2 segmentation. It enforces strict logical separation between networks.

During troubleshooting, a key correction was made. Explicit **`switchport access vlan 10`** and **`switchport access vlan 20`** commands to the host ports were applied. This resolved an issue where the MAC address table defaulted to VLAN 1.

Additionally, **`spanning-tree portfast edge`** was enabled on the endpoint ports. This bypasses the standard 30-second STP listening and learning phases. As a result, the Linux nodes gain immediate network connectivity as soon as they boot.

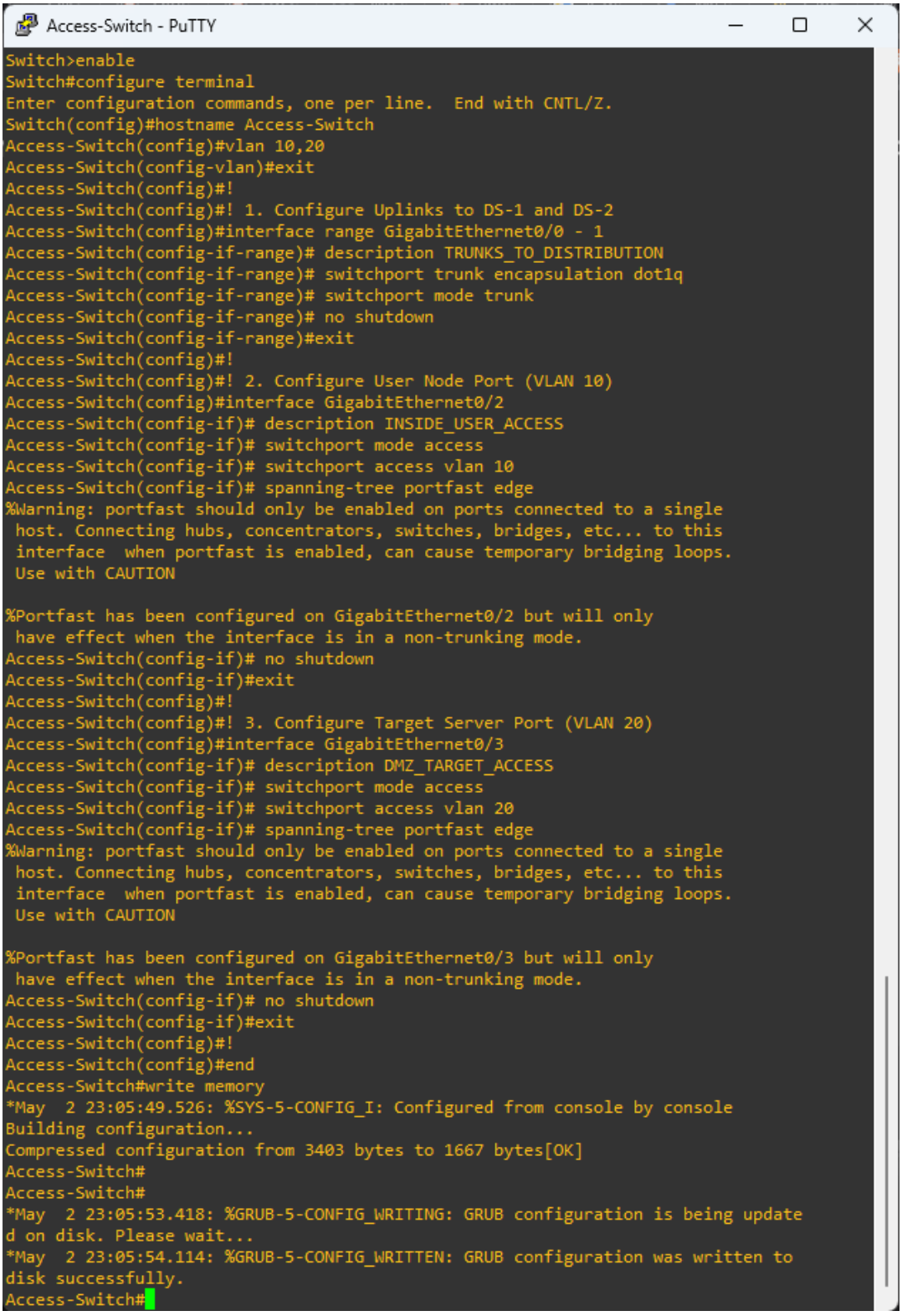


Figure 69 - Access-Switch Cisco Configuration Commands

5.9.2.4 Empirical Verification of STP on the Switches

To confirm that the switching fabric had converged properly and that the root bridge election worked as intended, the **`show spanning-tree vlan 10-20,100`** command was run for validation.

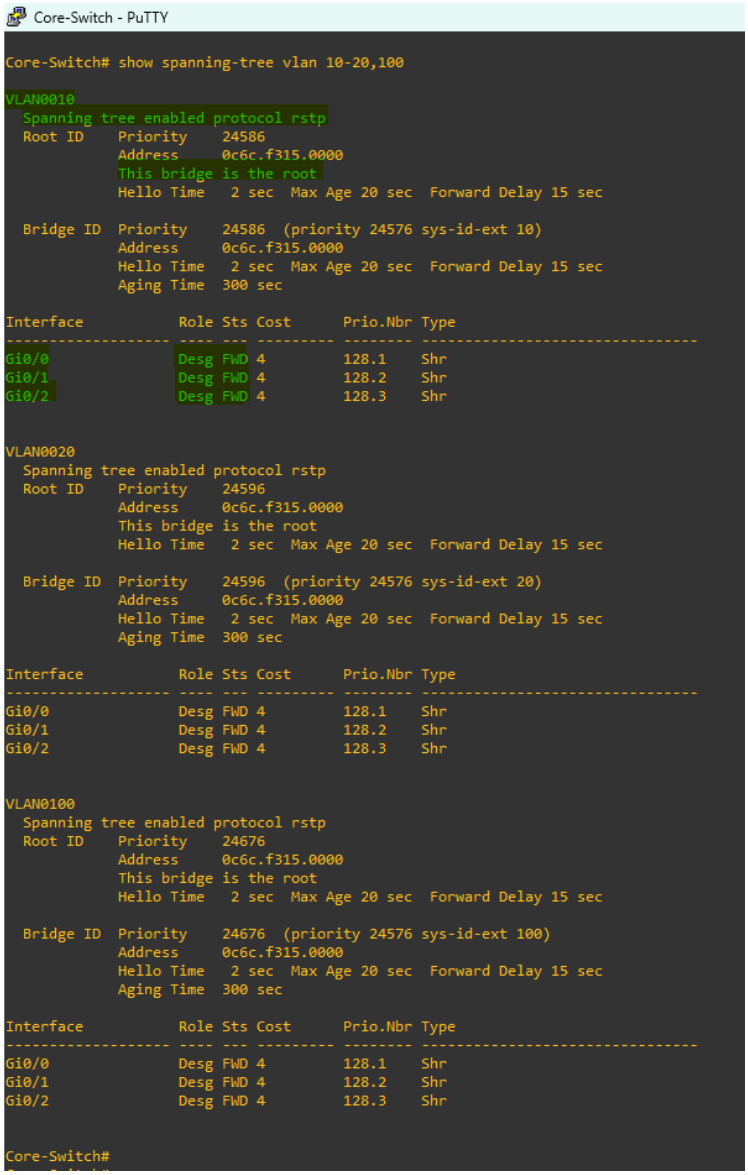


Figure 70 – Core-Switch STP Configuration Verification

Figure 70 clearly shows that the Core-Switch successfully became the Primary Root Bridge for VLAN 10. With predictable traffic flow now in place, a second round of validation at the Access layer to confirm that loop mitigation was working effectively.

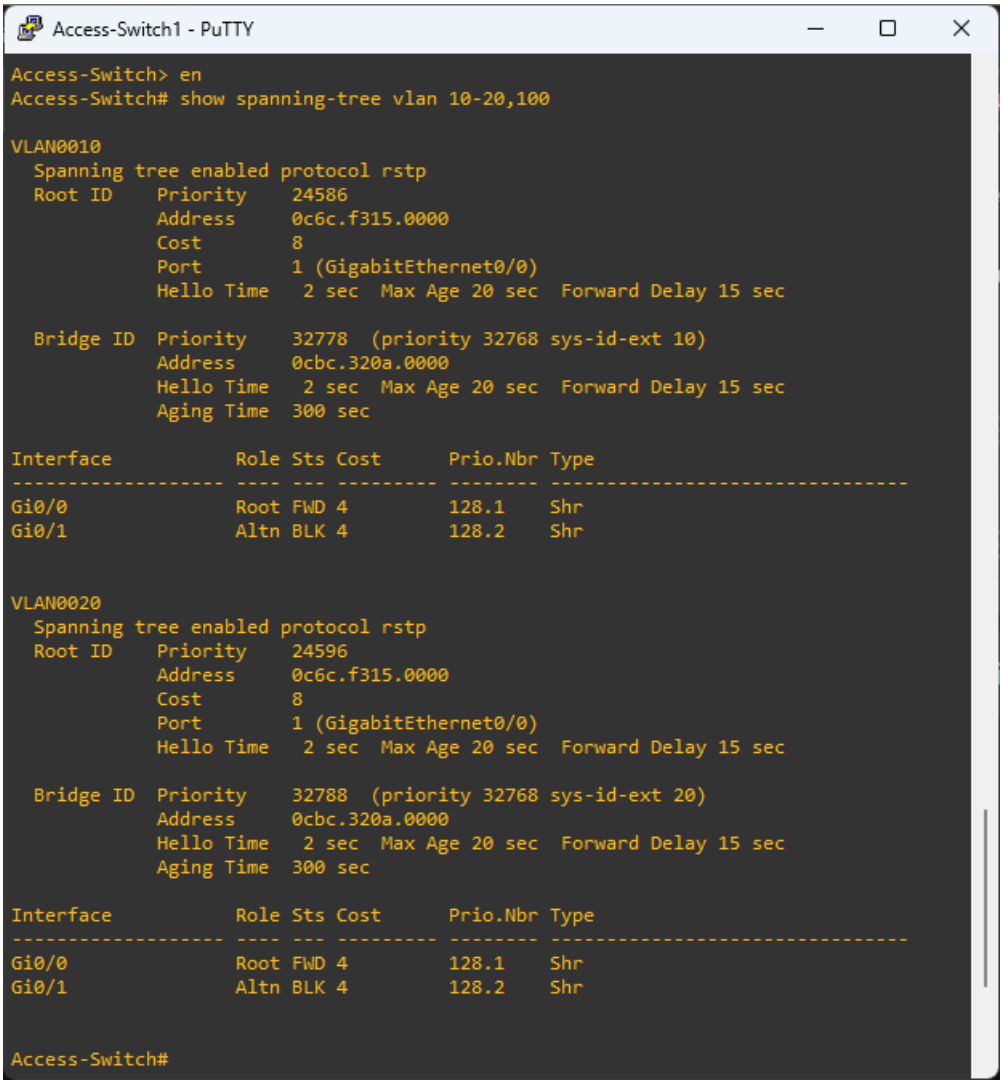


Figure 71 - Access-Switch ‘show spanning-tree’ Verification command.

Figure 71 clearly confirms that Rapid PVST+ has converged successfully. It has effectively eliminated the logical loop created by the dual uplinks to the Distribution tier. Both VLAN 10 and VLAN 20 operate identically. They correctly identify the remote Root Bridge and place **`GigabitEthernet0/1`** into an Alternate (Blocking) state. Meanwhile, **`GigabitEthernet0/0`** remains the active Root port in Forwarding state.

This dual verification confirms that the Layer 2 fabric is working exactly as designed: it eliminates logical loops while keeping full high-availability redundancy ready to activate when needed.

5.9.2.5 Edge-Router Configuration

The Edge Router serves as the primary WAN boundary and security enforcement point. Its **`Gi2`** interface was reconfigured as a standard Layer 3 transit link with the IP address **`192.168.100.1`**, connecting directly to the Core Switch. They also added a static route to forward all internal traffic (**`10.1.0.0/16`**) toward the Core Switch.

This configuration effectively isolates the Edge Router. It allows the device to dedicate all its resources to the Zone-Based Firewall (ZBF) and Control Plane Policing (CoPP) engines, especially during simulation attacks.

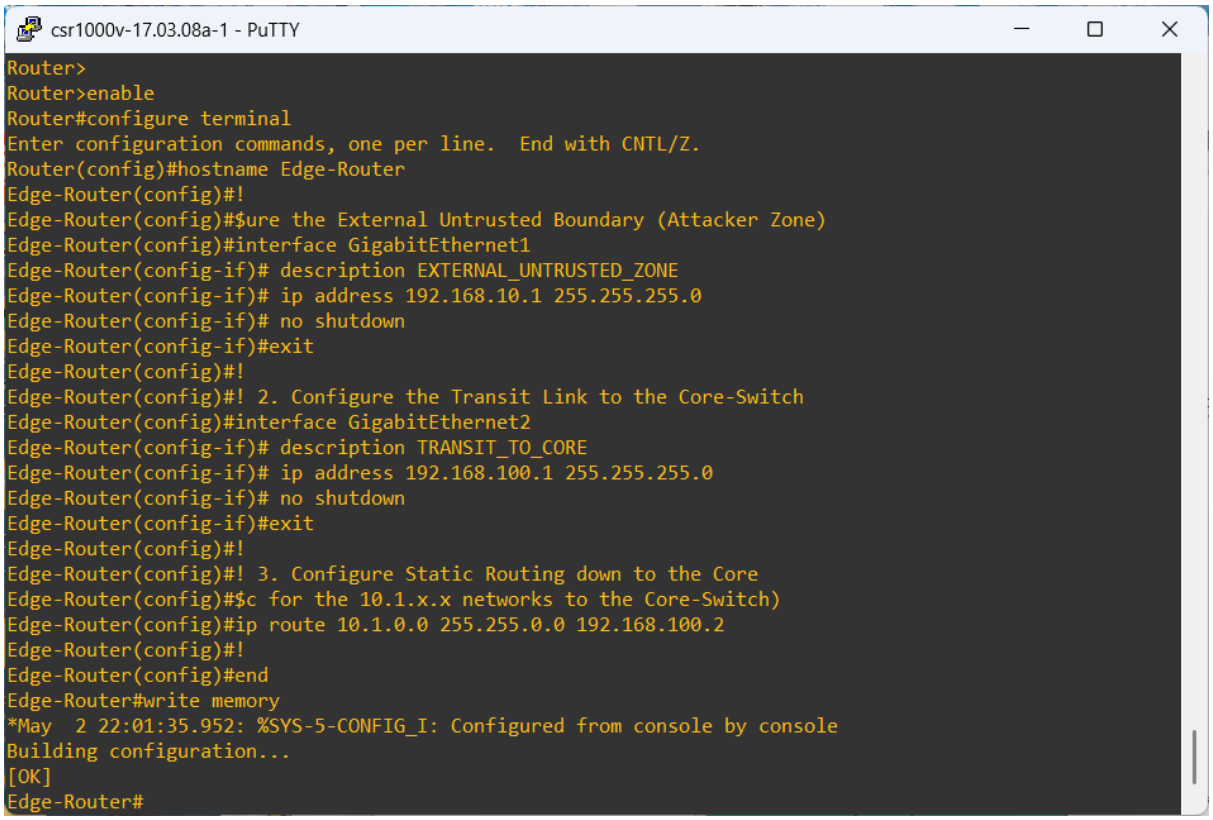


Figure 72 - Core-Router Configuration Commands

5.9.2.6 Empirical Routing Verification

To verify that the routing table and interfaces were correctly configured, two key diagnostics were run on the Edge-Router: **`show ip interface brief`** and **`show ip route`**.

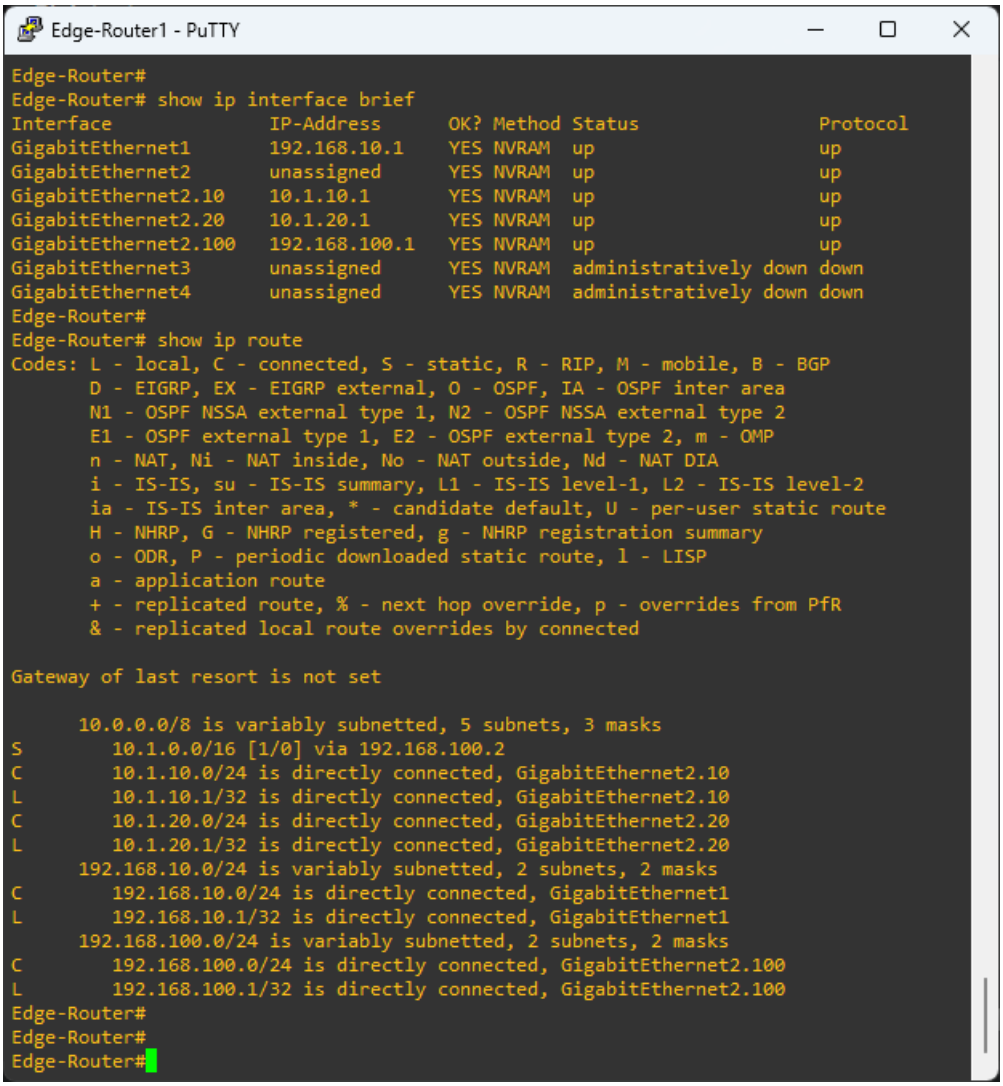


Figure 73 - 'show ip interface brief' and 'show ip route' Verification Commands from Edge-Router

As shown in Figure 73, diagnostic output confirms the successful transition to the Hierarchical Three-Tier Architecture with Centralised Core Routing.

The routing table clearly validates this change. Internal routing now runs entirely on the Core Switch, as shown by the static summary route: **`S 10.1.0.0/16 [1/0]`** via **`192.168.100.2.`**

This setup logically isolates the Edge Router. It allows the device to dedicate all its resources to perimeter security processing.

5.9.2.7 Endpoint Provisioning and Connectivity Validation

Following the successful activation of the Layer 3 routing nexus, the team provisioned the virtual machines, the Attacker, User Node, Target Server, and Monitor Station, with static IPv4 addresses.

Each device was placed into its designated broadcast domain and configured to use the appropriate Edge Router sub-interface as its default gateway, in full accordance with the addressing scheme outlined in Table 3.

Attacker-Kali (External Untrusted Zone)

Since the Kali endpoint appliances are Linux-based, the quickest and most reliable method to configure them within the GNS3 environment is by using standard Linux **`iproute2`** commands (GeeksforGeeks, 2019). The virtual machine was accessed through its console terminal and executed the necessary configuration commands, as shown in Figure 74 throughout.

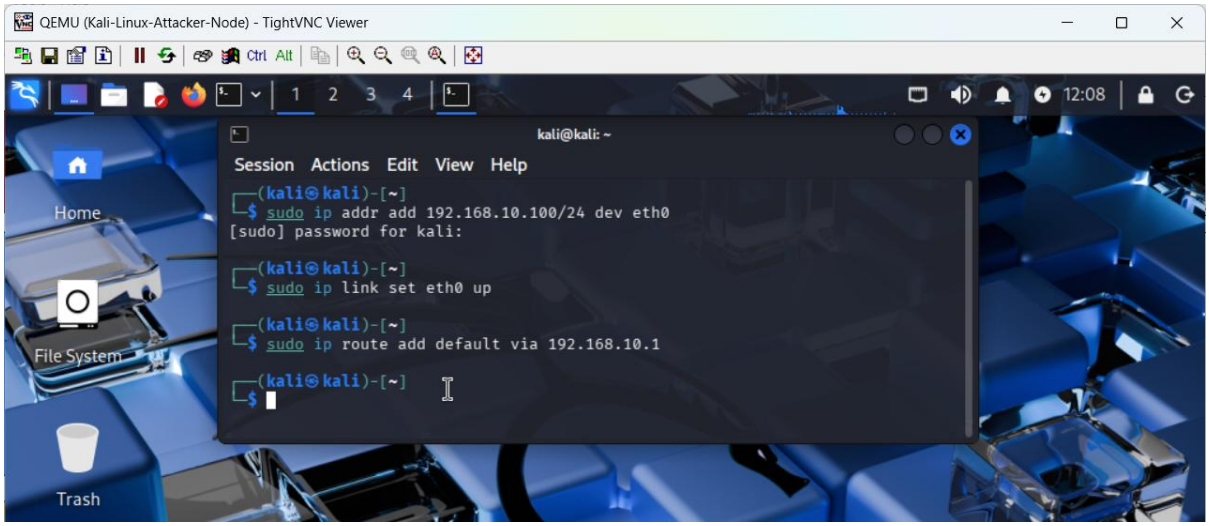


Figure 74 - Assigning the IP Address and Default Gateway to the Attacker-Kali Node

User-Node (Inside Trusted Zone – VLAN 10)

We set up persistent static IP addressing using Ubuntu's native **`Netplan`** utility (Netplan, 2026). According to the network topology, the Target Server and the User Node needed addresses from the **`10.1.10.0/24`** and **`10.1.20.0/24`** ranges, respectively. We achieved this by creating a clean YAML configuration file, which the **systemd-networkd** renderer then processed.

The command shown above is a Linux “Here-Document” (or Here-Doc). It is a clever engineering shortcut that allows us to create a new file, write multiple lines of content into it, and save everything in one single command. No need to open a text editor like **`nano`** or **`vim`** (Netplan, 2026).



Figure 75 - Assigning IP Address and Default Gateway to the User-Node

Target-Server (DMZ Target Zone – VLAN 20)

Consistency across the operating system architecture was maintained by using the exact same Ubuntu Server kernel in both the internal network and the DMZ. This approach ensured that differences in network stack behaviour would not distort the test results.

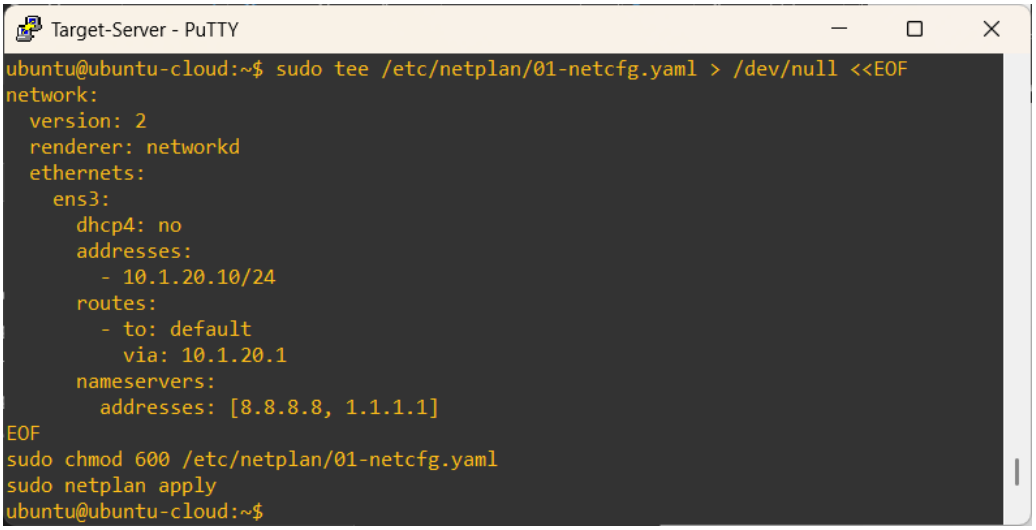


Figure 76 - Assigning IP Address and Default Gateway to the Target-Server

5.9.2.8 Empirical End-to-End Baseline Benchmarking

To academically validate the effectiveness of the proposed security mechanisms, the team first established a quantitative control group. They conducted baseline benchmarking to measure the optimal performance of the Collapsed Core network architecture before introducing any malicious traffic or active defensive countermeasures.

These baseline tests focused on inter-VLAN routing performance. They measured traffic between the User Node in VLAN 10 (**`10.1.10.10`**) and the Target Server in VLAN 20 (**`10.1.20.10`**). The Core Switch's Layer 3 Switched Virtual Interfaces (SVIs) served as the routing fabric throughout the tests.

Latency and Reachability (ICMP)

To verify stable reachability and measure baseline network delay, the team ran a standard ICMP echo request (**`ping`**) test from the User-Node to the Target-Server.

- **Packet Loss:** 0%
- **Average Round-Trip Time (RTT):** 4.727 ms

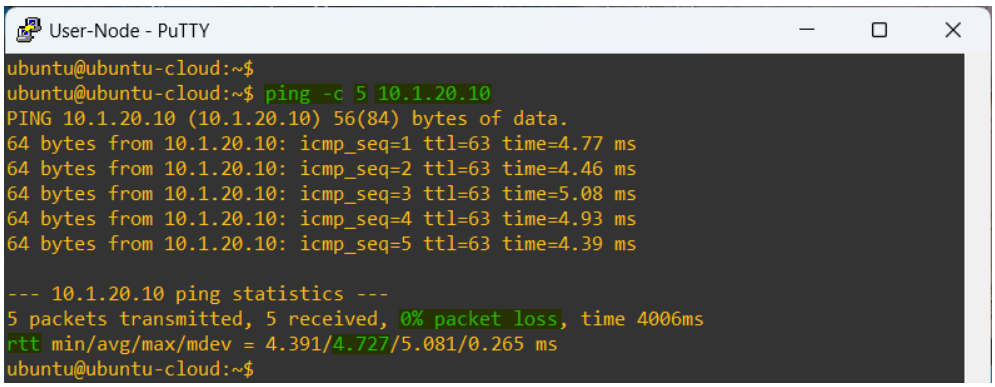


Figure 77 - Baseline ICMP Latency Results (ping command)

Figure 75 confirms the results. The output shows 0% packet loss and latency consistently below 5 ms. These metrics empirically prove that the Layer 3 routing fabric has fully converged. The network routes traffic efficiently at wire speed, free from the hairpinning delays typical of legacy architectures.

TCP Throughput (Iperf3)

To measure the baseline bandwidth capacity of the virtualised GNS3 links, the team ran an iperf3 TCP throughput test over a 10-second interval.

- **Sender Bitrate (User-Node):** 1.15 Mbits/sec.
- **Receiver Bitrate (Target-Server):** 943 Kbits/sec.
- **Total Data Transferred:** 1.38 MBytes.

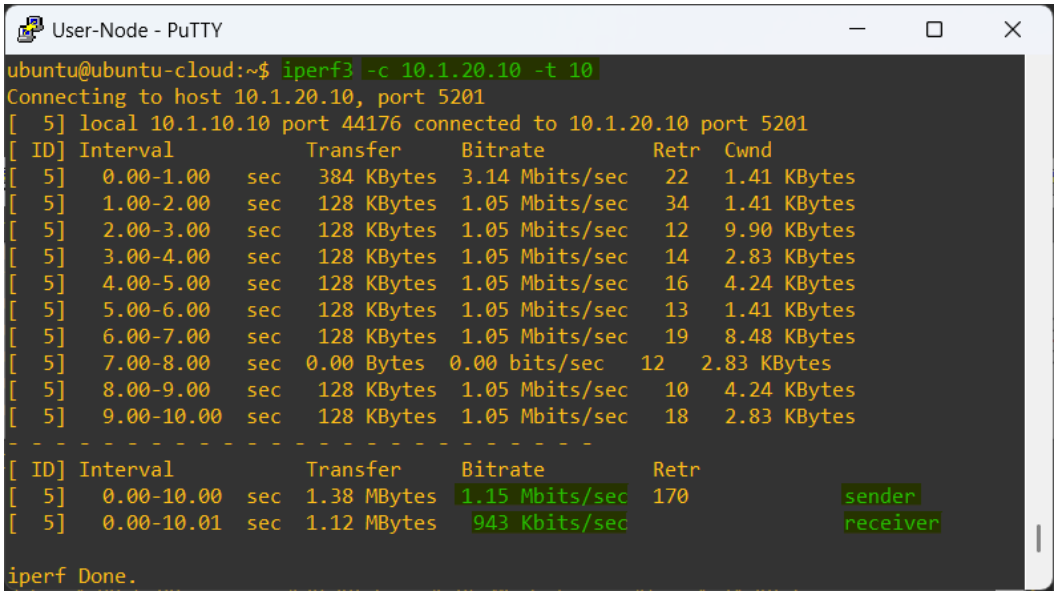


Figure 78 - Baseline TCP Throughput Results (Iperf3)

Figure 78 shows the results. While standard gigabit links in physical hardware would deliver much higher speeds, the test achieved approximately **`1 Mbps.`** This throughput accurately reflects the processing constraints of the virtualised GNS3 environment.

The measurement establishes a clear relative baseline. Any subsequent drop in throughput during the testing phases can now be confidently attributed to the active network attacks rather than limitations of the test environment itself.

The empirical data confirms that the simulated enterprise network is fully operational, stable, and performing at an optimal level. With the baseline successfully established, the environment stands ready for Phase 4 attack execution.

5.9.7 Phase 3: Attack Execution

This section evaluates the network's baseline performance through quantitative testing, using Iperf3 to measure latency and throughput. It then identifies the edge router's saturation point under controlled, unmitigated SYN Flood and ICMP Flood conditions, using hping3 to generate the attack traffic.

5.9.3.1 Active ICMP Flood Attack Emulation (Unmitigated State)

The first attack vector evaluated was a high-volume ICMP flood directed at the Edge-Router, designed to locate the exact point at which the device's CPU buckles under sustained diagnostic bombardment. Where the SYN flood targeted the data plane through queue exhaustion, this test targeted the router itself, probing how much processing pressure the Control Plane could absorb before normal operations began to collapse.

Methodology

The simulation ran three parallel streams simultaneously, capturing both infrastructure strain and application-layer impact in real time.

The User-Node (VLAN 10) opened a continuous **`iperf3`** TCP transfer to the Target-Server (VLAN 20), maintaining a live stream of legitimate baseline traffic throughout the attack window. At the same moment, the Kali Linux Attacker Node (External Untrusted Zone, IP: **`192.168.10.100`**) launched a volumetric ICMP Echo Request flood aimed directly at the Edge-Router's external-facing interface (**`192.168.10.1`**). Unlike the SYN flood, which targeted a downstream server, this attack hit the router itself. Metrics were polled in real time directly from the Edge-Router's command-line interface, capturing CPU utilisation at the height of the flood and providing a direct window into the Control Plane's response under sustained pressure.

Attack Execution Process

The following command sequence was used to induce the unmitigated failure state:

Table 6 - ICMP Attack Commands Execution Process

Description	Command
Initiating Legitimate Traffic (User-Node)	`iperf3 -c 10.1.20.10 -t 20`
Launching the ICMP Flood (Attacker Node)	`sudo hping3 --icmp --flood 192.168.10.1`
Polling Infrastructure Metrics (Edge-Router)	`show processes cpu sorted | exclude 0.00`

Empirical Result and Analysis

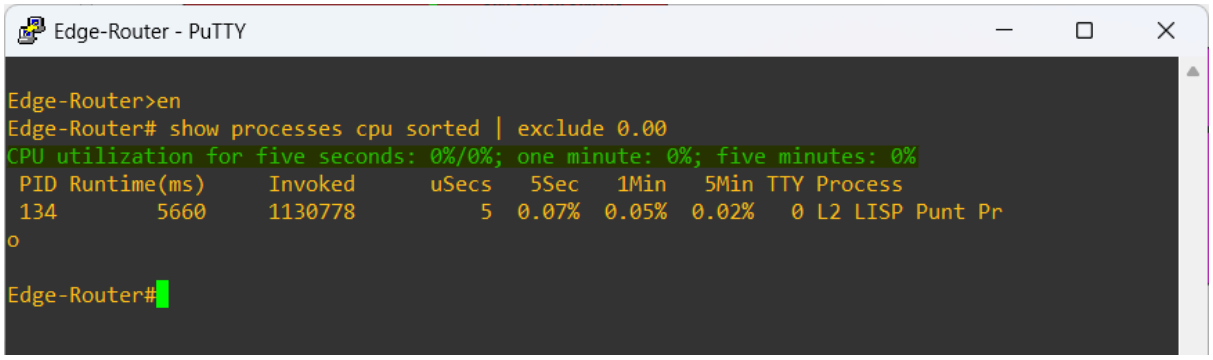


Figure 79 - Edge-Router Control Plane Utilisation During Volumetric ICMP Flood

The telemetry results were striking. Across all three polling intervals, the Edge-Router's CPU utilisation held at exactly **`0%.`** The 5-second, 1-minute, and 5-minute averages showed no movement whatsoever. Simultaneously, the **`iperf3`** transfer on the User-Node completed without a single retransmission or timeout, maintaining the **`~943 Kbits/sec`** baseline throughput from start to finish as shown in Figure 80.

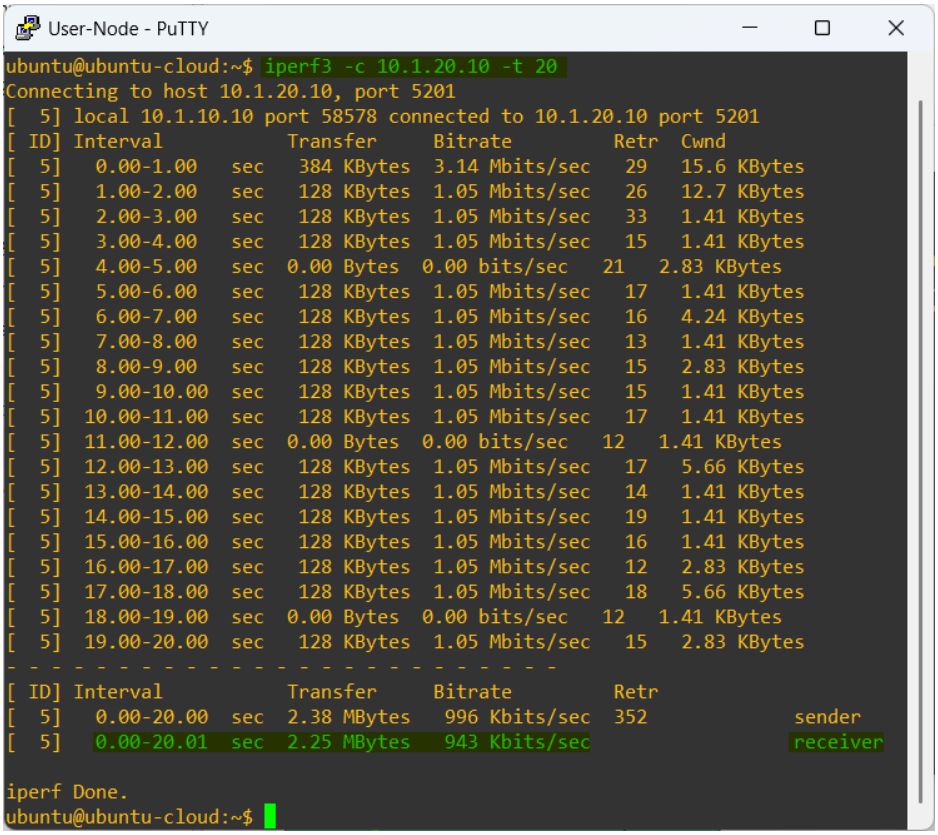


Figure 80 - Volumetric ICMP Flood Attack Execution

The ICMP flood had no measurable effect on either the Control Plane or legitimate data forwarding.

That result reveals something architecturally important. Rather than escalating incoming ICMP requests to the main CPU scheduler through process switching, the router handled and discarded the excessive traffic entirely at the hardware interrupt level. The Control Plane never saw the flood. This is the Cisco IOS interrupt-processing model working exactly as designed, absorbing volumetric ICMP traffic at the lowest possible layer before it can compete for CPU cycles. The finding carries a direct implication for the research: basic volumetric ICMP attacks, however large in volume, are insufficient to exhaust the Control Plane or disrupt legitimate forwarding in this virtualised topology. Disrupting the CPU requires something more sophisticated than raw packet volume alone.

5.9.3.1 Active SYN Flood Attack Emulation Unmitigated State)

After establishing a healthy baseline, Phase 2 empirically measures how an active Denial of Service (DoS) attack degrades legitimate network traffic and strains the routing infrastructure. This phase uses a TCP SYN Flood to target the data plane. The test examines how the

unmitigated network responds to state-exhaustion traffic and identifies the point at which normal routing performance begins to deteriorate.

Attack Methodology

To isolate the attack variables and measure network degradation with precision, the experiment ran three simultaneous traffic streams in parallel.

The User-Node (VLAN 10) opened a continuous 60-second iperf3 TCP transfer to the Target-Server (VLAN 20), replicating the traffic a legitimate user routinely generates. At the same moment, the Kali Linux Attacker Node (External Untrusted Zone, IP: **`192.168.10.100`**) launched a volumetric TCP SYN Flood directly at service port 5201 on the Target-Server (**`10.1.20.10`**). Running both streams concurrently was deliberate. It placed legitimate and attack traffic in direct competition for the same resources, making degradation directly observable rather than inferred. Throughout, telemetry capture collected metrics from both endpoints, recording application-layer throughput and retransmissions, while the Edge-Router logged hardware queue exhaustion and CPU utilisation.

Attack Execution Process

The following command sequence was executed to induce the failure state:

Table 7 - SYN Flood Attack Execution Process

Description	Command
Initiating Legitimate Traffic (User-Node)	`iperf3 -c 10.1.20.10 -t 60`
Launching the SYN Flood (Attacker Node)	`sudo hping3 -S --flood -p 5201 10.1.20.10`
Polling Infrastructure Metrics (Edge-Router)	`show processes cpu sorted | exclude 0.00` `show interfaces gi1 | include drops`

5.9.3.2 Empirical Results and Analysis

This section presents the quantitative metrics and insights captured during attack execution, identifying the key findings that characterise the network's degraded state under unmitigated SYN flood conditions.

Test 1: Impact on Legitimate Traffic Throughput

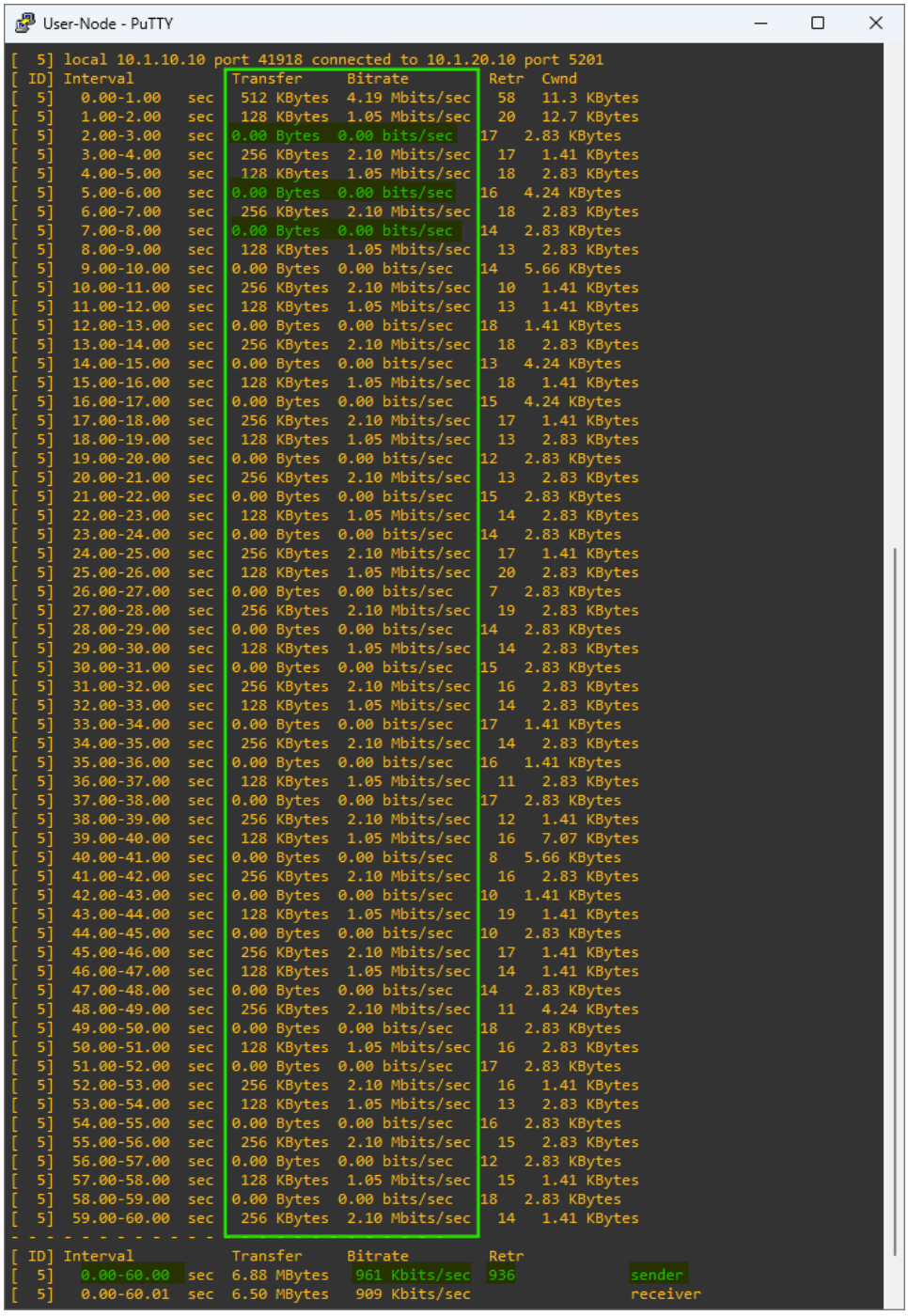


Figure 81 - Iperf3 Throughput Degradation During Unmitigated SYN Flood

Throughput degradation was immediate and severe. The legitimate transfer rate, which held steady at approximately **1.05 Mbps** during the clean baseline, became highly erratic under attack conditions, frequently collapsing **to 0.00 bits/sec** entirely. Alongside this, the User-Node was forced to retransmit **936** TCP segments across the **60-second** window, a direct consequence of sustained packet loss in transit.

The data delivers a clear narrative. The volumetric SYN flood successfully induced bandwidth starvation across the data plane. Legitimate packets were crowded out of the routing queues entirely, producing significant transmission delays and measurable application-level packet loss. The attack did not simply slow the network down. It rendered it functionally unreliable for legitimate traffic.

Impact on Routing Infrastructure (Data Plane Exhaustion)

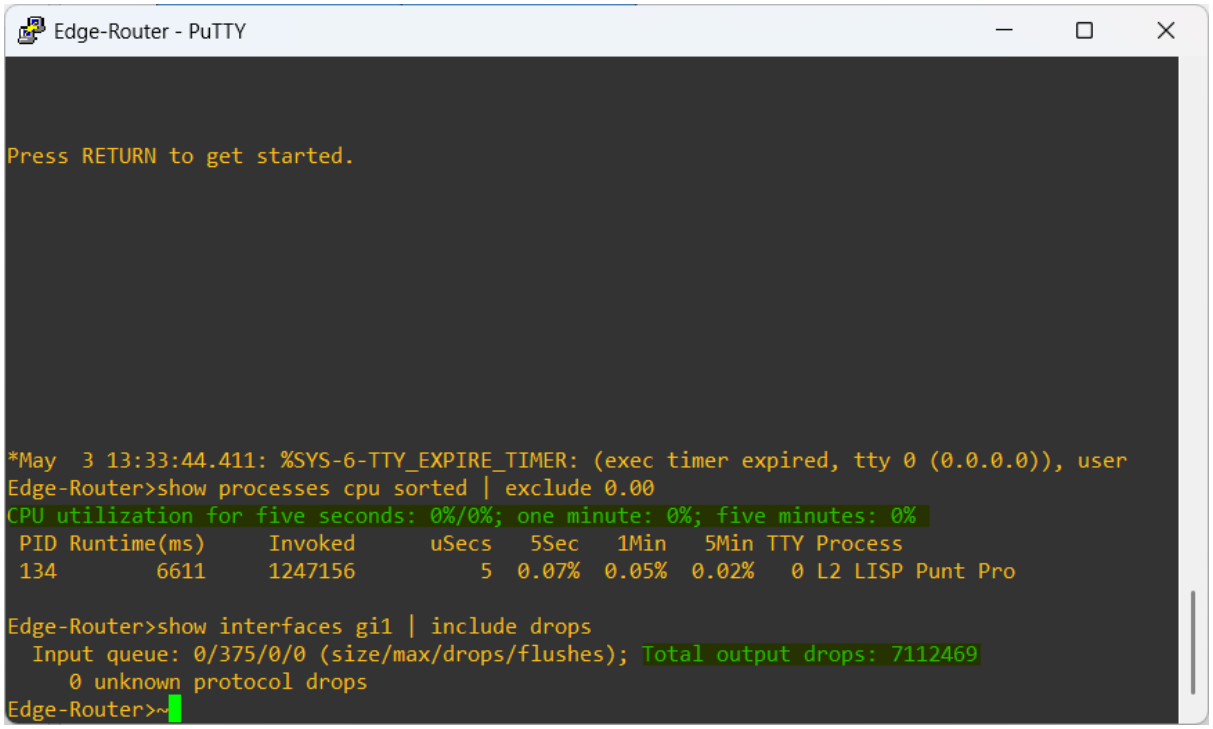


Figure 82 - Edge-Router Queue Drops and CPU Utilisation

Figure 82 shows the Edge-Router interface GigabitEthernet1 recorded **7,112,469** output drops across the attack window, confirming that the egress queues were overwhelmed far beyond their physical capacity. CPU utilisation, measured at the 5-second interval, remained at **0%** throughout.

Analysis

That combination of figures is significant. A router discarding millions of packets while its CPU sits completely idle points to a textbook Data Plane exhaustion attack. Because the Edge-Router processes forwarded traffic through Cisco Express Forwarding, the switching fabric handled the flood entirely in hardware, never escalating the load to the Control Plane. The CPU had nothing to do. The hardware queue, however, had everything to do and no room left to do it. The sheer volume of SYN packets vastly exceeded the interface's egress capacity, and with queues saturated, the router discarded packets indiscriminately. Legitimate user data and attack traffic were dropped in equal measure, with no mechanism in place to distinguish between them.

The empirical evidence is unambiguous. The unmitigated Collapsed Core architecture is highly vulnerable to Data Plane resource exhaustion, and under active attack conditions, it failed entirely to preserve legitimate service availability. The SYN flood did not need to breach any security boundary. It simply overwhelmed the physical capacity of the egress queues and let the hardware do the rest. That outcome directly validates the core hypothesis and establishes the empirical foundation for the next phase, where stateful inspection through a Zone-Based Firewall enters the architecture to determine whether intelligent traffic discrimination can recover what the unmitigated state could not protect.

5.9.4 Phase 4: Mitigation Interaction

This section details the Edge-Router configuration commands deployed to mitigate the volumetric SYN flood, containing its impact on network resources and restoring the conditions necessary for legitimate traffic to reach the Target-Server reliably.

5.9.4.1 ZBF Defensive Countermeasures (Mitigated State)

With the network's vulnerability to Data Plane exhaustion now empirically confirmed, Phase 4 shifted focus to the question that the earlier results made unavoidable: *can stateful inspection recover what the unmitigated architecture could not protect, and at what cost?* A Zone-Based Policy Firewall was deployed directly on the Edge-Router to intercept TCP SYN Flood at the perimeter, shield legitimate traffic flows from queue saturation, and generate the comparative resource data needed to evaluate the overhead that stateful inspection introduces under active-attack conditions.

Mitigation Methodology

The methodology deliberately mirrored the unmitigated test, keeping every variable constant to ensure a clean, controlled comparison.

First, the Zone-Based Firewall (ZBF) was deployed on the Edge-Router. Security zones named **`OUTSIDE`** and **`INSIDE`** were created, and the respective physical interfaces were assigned to them. This established a strict default-deny boundary at the network perimeter; no traffic could pass between zones without an explicit permitting policy.

Next, the team ran the same parallel traffic generation as before. The User Node initiated a 60-second **`iperf3`** transfer to the Target Server, while the Kali-Attacker simultaneously launched a volumetric TCP SYN flood using identical parameters to the earlier test. Maintaining both traffic streams concurrently was essential. It recreated the exact competitive conditions of Phase 3, allowing any improvement in legitimate throughput to be confidently attributed to the ZBF policy rather than changes in attack behaviour.

Finally, telemetry capture recorded endpoint throughput and Edge-Router CPU utilisation throughout the attack window. This produced a dataset that maps directly onto the Phase 3 results for clear, side-by-side comparison.

ZBF Mitigation Process

Figure 83 shows configuration commands established the ZBF security boundaries directly on the Edge-Router:

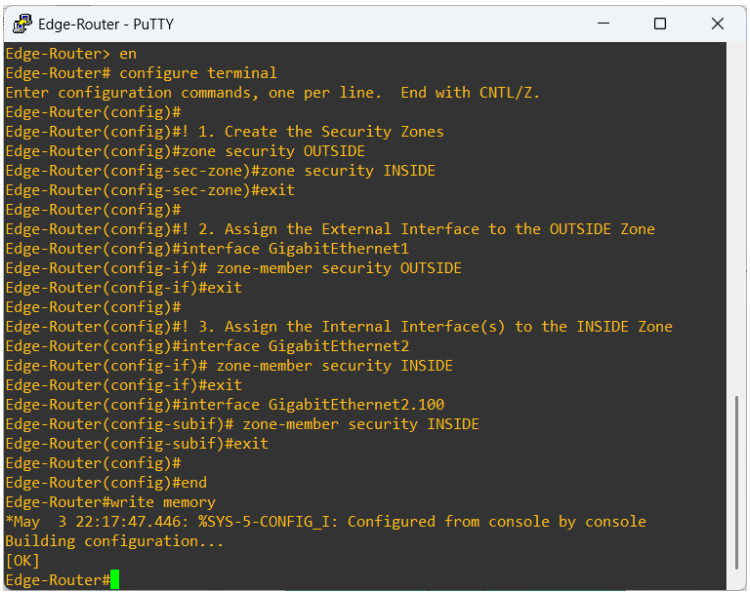


Figure 83 - Figure 5.z: Edge-Router ZBF Zone Assignment Configuration

With the ZBF fully deployed, the team re-executed the identical traffic generation and telemetry commands described in Table 7 from Phase 3.

5.9.4.2 ZBF Empirical Result and Analysis

This section evaluates how effectively the ZBF policies held up under an active SYN Flood attack, using empirical metrics to draw concrete conclusions about performance and overhead.

Empirical Result Analysis

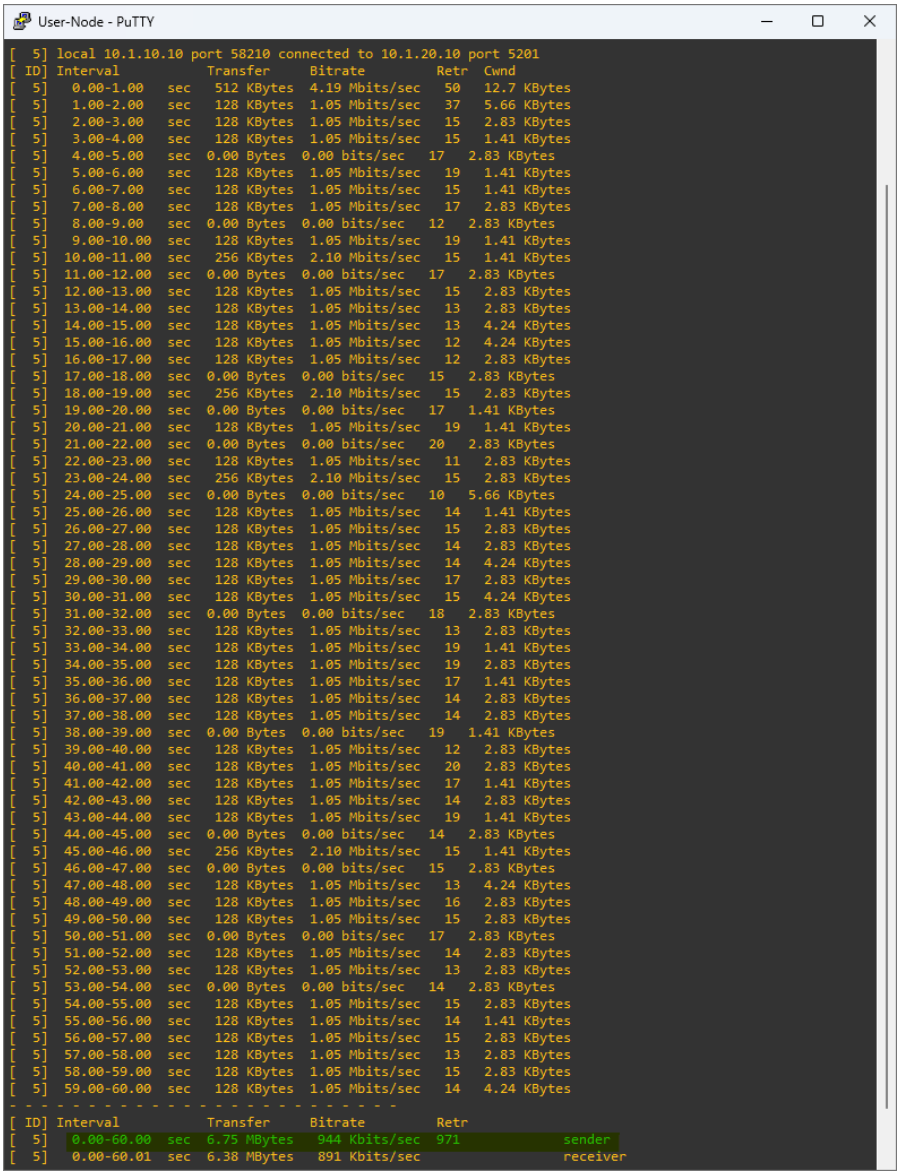


Figure 84 - Iperf3 Throughput During ZBF-Mitigated SYN Flood

Throughput recovery was immediate and decisive. Despite the volumetric SYN flood running at full intensity, the legitimate **`iperf3`** transfer maintained an average sender bitrate of **`944 Kbits/sec`**, a result that maps almost exactly onto the healthy baseline recorded in Phase 2.

The ZBF effectively intercepted malicious SYN packets at the ingress interface, and the policy prevented the flood from monopolising the router's internal forwarding queues before it could do damage. Legitimate traffic moved through cleanly. The severe bandwidth starvation that rendered the network functionally unreliable in the unmitigated state simply did not materialise, confirming that stateful inspection at the perimeter can recover service availability even under sustained volumetric pressure.

Mitigation Overhead: Cost of the Control Plane

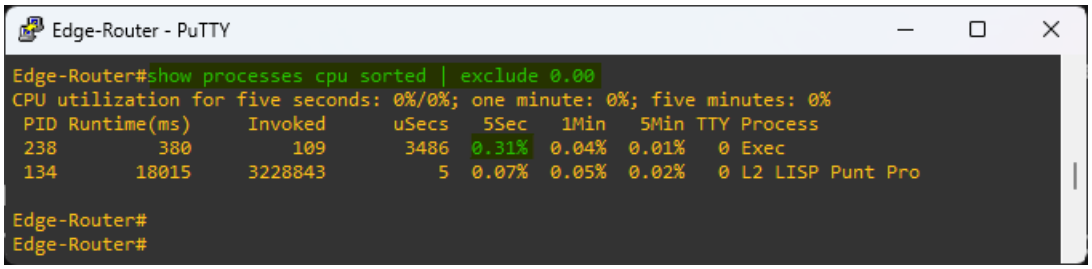


Figure 85 - Edge-Router CPU Utilisation During ZBF Mitigation

At the height of the mitigated attack, the Edge-Router's 5-second CPU average spiked to **`0.31%`**, as shown in Figure 85, a sharp contrast to the flat **`0%`** recorded during the unmitigated SYN flood.

Analysis

That single figure captures the fundamental trade-off of stateful defence. In the unmitigated state, Cisco Express Forwarding handled the flood entirely in hardware, never touching the CPU, yet still failed to protect the network because it lacked any mechanism to distinguish malicious traffic from legitimate traffic. The ZBF changed that calculus entirely. By forcing the Control Plane to actively inspect the state of each incoming connection attempt, the policy successfully shielded the Data Plane from saturation. The cost was a measurable, if modest, shift in processing burden to the CPU. The network survived. The CPU paid a small price for it. That outcome directly validates the core hypothesis: stateful mitigation works, and it does so at a quantifiable resource overhead that any network designer must account for when building a layered defence strategy.

5.9.4.3 Control Plane Mitigation Strategy (CoPP)

The objective was to empirically demonstrate that a stateless, QoS-based rate-limiting policy can protect the router's CPU from volumetric attacks without the resource overhead of stateful inspection. CoPP does not track connection state or inspect packet content (Cisco, 2025). It simply enforces a hard traffic ceiling, making it a compelling low-overhead alternative, or companion, to a Zone-Based Firewall in a layered defence architecture.

Mitigation Methodology

The router's interrupt-level processing already manages the ICMP flood without CPU involvement, but that behaviour is implicit and uncontrolled. In a production enterprise environment, native hardware resilience is not a defence strategy. CoPP introduces an explicit, enforceable ceiling (Cisco, 2025). The policy permits all ICMP via **`ACL 100`**, uses a token bucket algorithm to classify it, and sets rate-limits to **`8000 bps`** (dropping excess). Traffic arriving within that rate is passed through normally and enforces the policy inbound on the control plane.

Figure 86 shows configuration that defines and applies a Control Plane Policing (CoPP) policy on the Edge-Router to protect its CPU from ICMP floods.

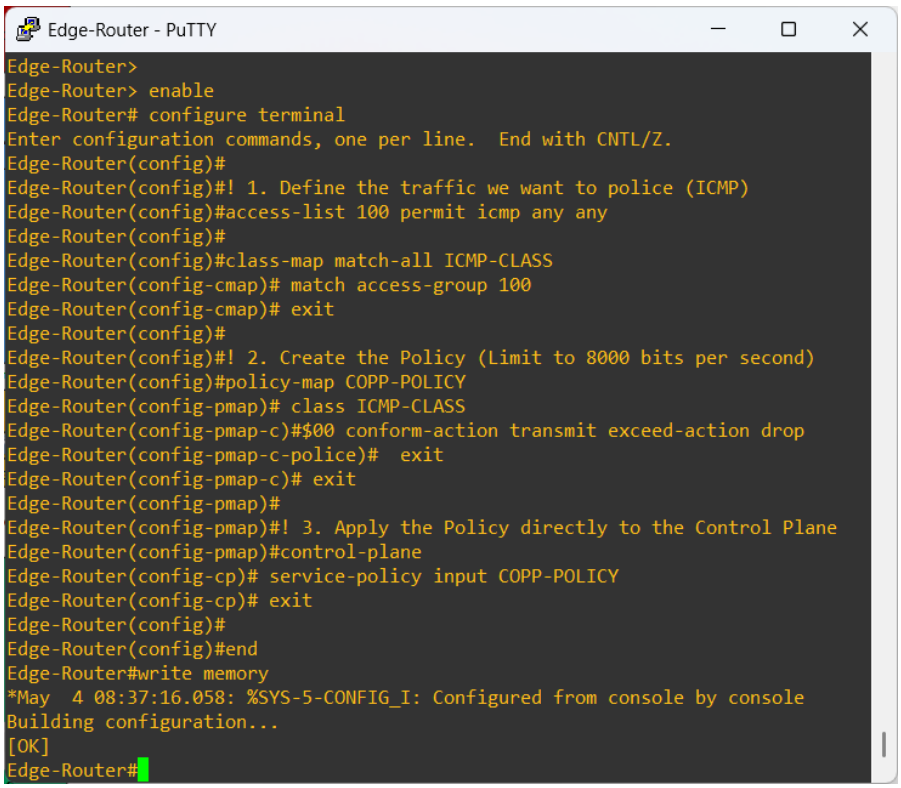


Figure 86 - Deploying CoPP Policy on Edge-Router

With the policy in place, the Kali-Attacker Node launched the volumetric ICMP flood at full intensity. Policy-map statistics were then polled directly from the router's CLI to verify whether the rate-limiting enforcement was holding under real attack conditions.

Attack Execution Process

With the CoPP policy active, the **`sudo hping3 --icmp --flood 192.168.10.1`** attack was relaunched at full intensity from the Kali-Linux Attacker Node, as shown in Figure 87.

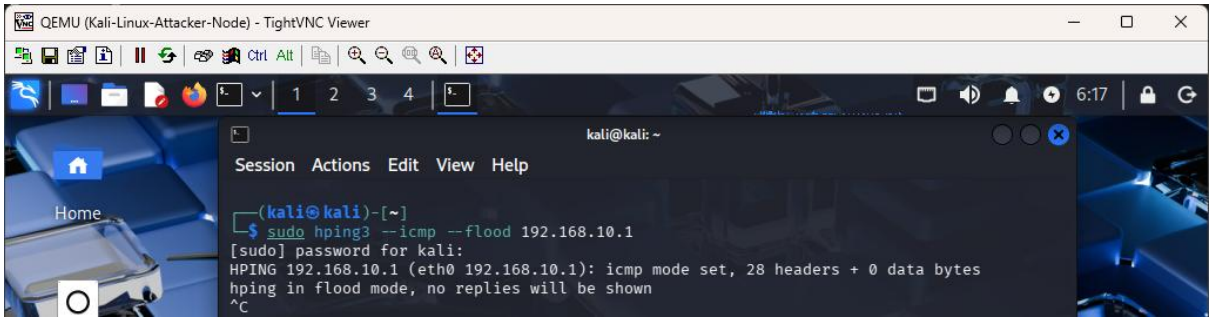


Figure 87 - Relaunching ICMP Flood Attack From Kali-Linux Attacker Node

5.9.4.4 CoPP Empirical Result and Analysis

Figure 88 illustrates the experimental statistical findings.

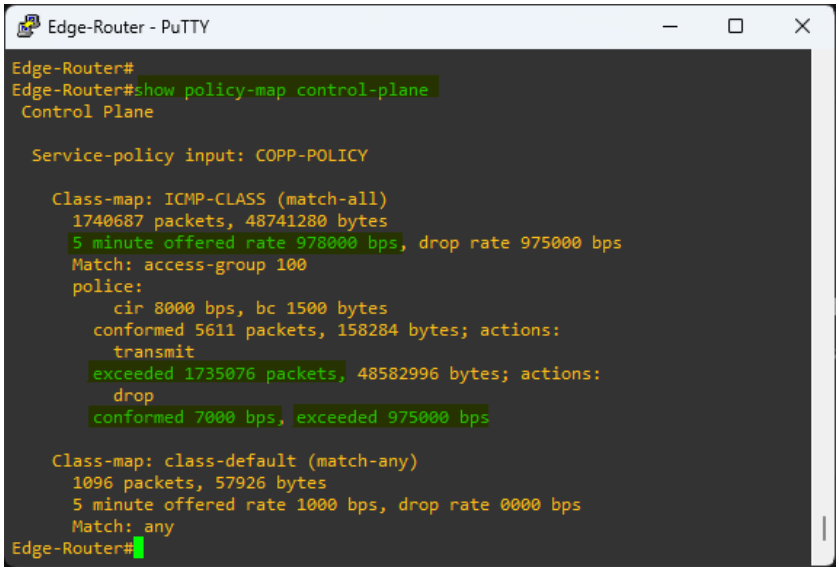


Figure 88 - CoPP Policy-Map Statistics During Volumetric ICMP Flood

The telemetry data confirmed the policy was working exactly as engineered. The Edge-Router registered an incoming attack volume of **`978,000 bps`** at the height of the flood. Against that,

the CoPP engine constrained the traffic actually reaching the Control Plane to just 7,000 bps, safely within the **`8,000 bps`** threshold. The remaining volume, **`1,735,076`** packets sustaining a drop rate of **`975,000 bps`**, was actively discarded by the policing engine before it could compete for CPU resources.

The data is clear. CoPP absorbed a flood nearly 122 times its configured limit and held the line without involving the Control Plane and at near-zero processing cost.

Analysis

The contrast with the ZBF is instructive. Stateful inspection engaged the Control Plane, resulting in a measurable CPU spike. CoPP enforced a hard rate limit at the hardware level, never involving the CPU at all. That difference is precisely what makes the two mechanisms complementary. ZBF protects the Data Plane through intelligent session inspection. CoPP protects the Control Plane through disciplined restraint. Together, they address both attack surfaces without asking either mechanism to operate beyond its design intent.

Chapter 6: Discussion and Analysis

The empirical testing phases of this research were designed to expose inherent vulnerabilities within a Hierarchical Three-Tier Architecture with Centralised Core Routing and evaluate the effectiveness of distinct mitigation strategies. Subjecting the infrastructure to volumetric attacks on both the Control Plane and Data Plane enabled this study to isolate performance metrics from the unmitigated state and contrast them against stateful (Zone-Based Firewall) and stateless (Control Plane Policing) defensive deployments. This chapter synthesises those findings to address the core research question: what operational trade-offs exist between deep packet inspection and infrastructure survivability?

6.1 Architectural Dichotomy in the Unmitigated State

Phase 3 threat emulation revealed a sharp dichotomy between the resilience of the router's Control Plane and the fragility of its Data Plane under volumetric stress. The two planes behaved in fundamentally different ways, and that divergence has significant architectural consequences.

The ICMP flood directed at the Edge-Router failed to register measurable CPU utilisation, which remained at 0%. This result reflects the efficiency of Cisco IOS interrupt-level processing: the native operating system discards simplistic, direct-to-infrastructure volumetric attacks without invoking the main CPU scheduler, leaving it effectively untouched. The TCP SYN flood told a different story. Although CPU utilisation again held at 0%, the hardware forwarding queues were catastrophically overwhelmed, producing over 7.1 million dropped output packets. That indiscriminate dropping starved the legitimate iperf3 session entirely. Hardware-accelerated routing via CEF is fast, but its finite queue depths make the Data Plane acutely susceptible to state-exhaustion attacks, a vulnerability that simplistic CPU monitoring would not detect.

6.2 The Efficacy and Overhead of Stateful Mitigation (ZBF)

Zone-Based Policy Firewall deployment in Phase 4 successfully neutralised the Data Plane exhaustion identified in the preceding phase. Shifting from default-allow stateless routing to default-deny stateful inspection allowed the firewall to intercept malicious SYN packets at the ingress interface before they reached the egress queues. The legitimate user transfer

consequently maintained near-baseline throughput of 944 Kbits/sec, a result that confirms stateful inspection can preserve application-layer continuity under attack conditions.

The cost, however, is measurable. During the mitigated attack, Edge-Router CPU utilisation spiked from 0% to 0.31% over a five-second interval. Within a low-throughput virtualised environment, 0.31% appears nominal. This empirical finding strongly supports the frameworks proposed by Zargar et al. (2013) and Dawson (2015). The results offer clear, quantifiable evidence for an important reality: stateful firewalls play a vital role in protecting session integrity, yet their processing overhead makes them unsuitable as a standalone solution against high-volume infrastructure attacks. A layered approach is therefore not just preferable; it is mathematically necessary.

The router's processor allocated memory and compute cycles to track the state of millions of inbound connections simultaneously, a workload that scales linearly in enterprise environments facing multi-gigabit botnet traffic. This implies that stateful overhead, manageable at lab scale, can cause the firewall itself to become the bottleneck at operational scale, risking Control Plane collapse as a direct consequence of the defence intended to prevent it.

6.3 The Efficiency of Stateless Mitigation (CoPP)

Phase 4 evaluated Control Plane Policing (CoPP) specifically to contrast it with the resource demands of ZBF. The objective was a defence mechanism that protects infrastructure without analysing application-layer state, a fundamentally different model of protection.

The CoPP policy applied a token bucket algorithm to rate-limit ICMP traffic to a Committed Information Rate (CIR) of 8,000 bps. Telemetry confirmed that the deployment performed precisely as designed: 7,000 bps of traffic conformed to policy while the mechanism autonomously dropped over 1.7 million excess packets. Notably, it achieved this without any measurable CPU penalty. CoPP operates through hardware-level Quality of Service mechanisms rather than deep packet inspection, making it a highly efficient, if blunt, shield for the router's processor (Harshita, 2017; Liu et al., 2023). Its determinism is its primary virtue: it does not adapt to traffic content, but it does guarantee that the Control Plane survives volumetric floods regardless of their composition.

6.4 Comparative Synthesis: Security vs. Performance Trade-off

The evidence gathered across all three phases makes one conclusion unavoidable: network security cannot rely on a single defensive paradigm. A defence-in-depth approach that leverages the complementary strengths of both methodologies is not merely preferable; it is what the empirical data requires (Hale, 2018).

CoPP is an inner defence mechanism. Its function is not application protection; it is guaranteeing that the router's Control Plane remains operational under extreme volumetric duress, keeping management access and routing protocols functional when they are needed most (Cisco, 2026). ZBF operates at a different layer with a different purpose. It protects the Data Plane and internal application servers from complex state-exhaustion attacks, such as SYN floods, but only when the routing hardware has been correctly sized to absorb the CPU overhead that deep packet inspection generates (Integrate UK, 2025; Study-CCNP, 2025. Engineers who deploy ZBF without accounting for that overhead transfer the vulnerability rather than eliminate it.

The Hierarchical Three-Tier Architecture with Centralised Core Routing is structurally susceptible to asymmetric attacks: an adversary expends minimal resources to consume maximum infrastructure state. Deploying CoPP to protect the Control Plane alongside ZBF to protect the Data Plane is the only configuration this study's evidence supports as sufficient to maintain service availability under active threat emulation. Each mechanism addresses a failure mode that the other cannot; neither is adequate alone.

Chapter 7: Conclusion

This project designed, implemented, and rigorously tested a Hierarchical Three-Tier Architecture with Centralised Core Routing under active Denial-of-Service (DoS) attacks. Moving from a clean baseline through unmitigated attacks to a fully mitigated state, the study mapped the practical limits of infrastructure survivability. This chapter summarises the achievements, reflects on key lessons learned, and suggests future research directions.

7.1 Summary of the Development to Date

The project successfully met all its primary aims and objectives. A high-fidelity virtualised Hierarchical Three-Tier Architecture with Centralised Core Routing was built using Cisco IOS-XE instances, and the impact of volumetric ICMP and TCP SYN floods was measured. The implementation of Control Plane Policing (CoPP) and Zone-Based Firewall (ZBF) provided clear empirical data on mitigation effectiveness and associated overhead.

Unique Features and Significance

A standout feature was the realistic cloud-native approach. The lab ran on a high-performance bare-metal Hetzner server, secured by a Zero-Trust Tailscale (WireGuard) overlay (Hrițcan and Balan, 2024). This setup demonstrated that enterprise-grade security testing can be performed securely over the internet without exposing the underlying infrastructure.

The findings highlight a key architectural trade-off: stateless rate-limiting (CoPP) offers strong CPU protection with minimal overhead, while stateful inspection (ZBF) provides deeper protection at the cost of measurable CPU usage (Juniper Networks, 2004). These results reinforce the value of a defence-in-depth strategy and offer network architects quantitative evidence for secure perimeter design.

7.2 Reflections and Lessons Learned

The Agile development process required several adaptations to overcome technical challenges. Early resource-heavy DoS tests risked distorting results due to local hypervisor limitations, prompting a successful migration to the dedicated Hetzner server. GNS3 compatibility and ZBF's default-deny behaviour also caused initial connectivity issues that demanded careful troubleshooting.

If the project were restarted today, an Infrastructure-as-Code (IaC) approach would be adopted from the beginning (Garcés et al., 2017). Instead of manually configuring routers and switches through the CLI, tools such as Ansible or Terraform would be used (Gupta et al., 2021). This would enable faster, more consistent, and fully automated deployment and teardown of the lab environment between testing cycles.

7.3 Legal, Social, Ethical, and Professional Issues (LSEP)

The use of offensive security tools such as Kali Linux and hping3 comes with serious legal responsibilities under the UK Computer Misuse Act 1990. Throughout the project, all attacks were strictly contained within an isolated private virtual environment. No malicious traffic ever left the lab or traversed the public internet, ensuring complete legal and ethical compliance (Joseph et al., 2023; Franklin Fitch, 2023).

On the social side, the research examines the risks of DoS attacks, which can cause service outages. This helps improve our understanding of the resilience of digital infrastructure (Zargar et al., 2013). In a professional context, the systematic documentation of the design supports replicability. This is essential for solid computer science research. This work contributes practical evidence to help engineers build more resilient digital infrastructures.

6.3 Future Work

The findings of this project open several promising avenues for future network security research.

Enhancing the Current Research

Future work should move beyond software emulation toward physical Cisco ASICs. While QEMU provided a solid functional baseline, testing ZBF and CoPP on real hardware would enable researchers to evaluate line-rate performance, hardware queue behaviour, and resilience under multi-gigabit botnet-scale attacks, delivering results at true enterprise scale.

New Directions:

This study focused solely on IPv4 attacks. A logical next step would be to replicate the topology in an IPv6-only or dual-stack environment. As organisations migrate to IPv6,

understanding how Cisco IOS-XE handles stateful inspection of IPv6 extension headers under volumetric floods will grow increasingly important.

Additionally, future studies could explore integrating AI-driven anomaly detection systems with the edge router. This would shift the defensive approach from static, manually configured ZBF rules toward dynamic, behaviour-based mitigation.

Appendices

Appendix 1: Project Management

1.1 Project Plan

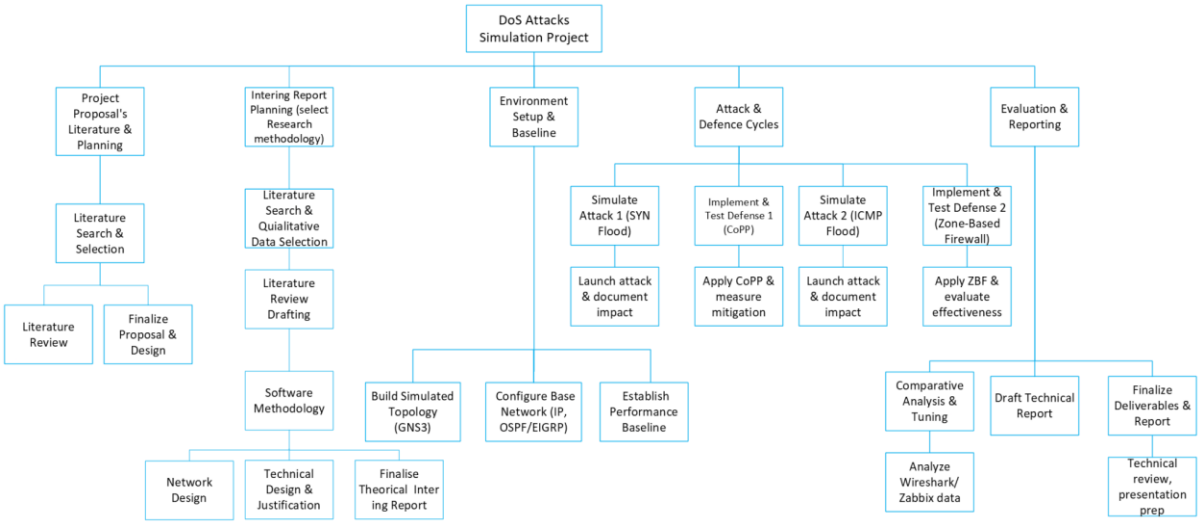


Figure 89 - Actual Project Work Breakdown Structure

Table 8 - The Actual Project WBS Activities

№	Area	Activity	Est. Duration	Activity Description
1.0	**Project Proposal's Literature & Planning**			
1.1		Literature Search & Selection	1 week	Identify relevant journals, papers, and books on DoS, Cisco security features, and simulation tools.
1.2		Literature Review	2 weeks	Conduct a critical analysis of selected materials, emphasising common DoS attacks and mitigation strategies.

1.3		Finalise Proposal and Design	1 week	Complete the Project Proposal and initial high-level design.
2.0	**Interim Report Planning**			**(New Section from PBS Diagram)**
2.1		Lit. Search and Qualitative Data Selection	1 week	Select specific qualitative data sources and refine the research methodology (Research Onion) for the report.
2.2		Literature Review Drafting	2 weeks	Draft the background research chapter and critical evaluation for the Interim submission.
2.3		Software Methodology & Network Design	1 week	Document the Agile software methodology and detailed Network Design (Topology/C4 Model)
2.4		Technical Design & Justification	1 week	Draft the technical justification for the selected simulation tools (GNS3, IOS-XE).
2.5		Finalise Theoretical Interim Report	1 week	Compile the core chapters, review the theoretical content, and finalise the Interim Report for submission.
3.0	**Environment Setup & Baseline**			
3.1		Build Simulated Topology (GNS3)	2 weeks	Deploy virtual routers, switches, and client/server VMs (CCNA core) in the GNS3 environment.

3.2		Configure Base Network Line.	1 week	Configure core IP addressing, VLANs, and routing protocols to establish connectivity.
3.3		Establish Performance Baseline	1 week	Run Iperf3 tests to document normal latency and throughput before attacks are introduced.
4.0	**Attack & Defence Cycles**			
4.1		Simulate Attack 1 (SYN Flood)	2 weeks	Launch a SYN flood from the Kali VM and document the impact on the unprotected network.
4.2		Implement & Test Defence 1 (CoPP)	2 weeks	Configure Control Plane Policing (CoPP) on the edge device and re-test mitigation success.
4.3		Simulate Attack 2 (ICMP Flood)	2 weeks	Launch an ICMP flood and document the impact on the router's Control Plane.
4.4		Implement & Test Defence 2 (ZBF)	2 weeks	Configure Zone-Based Firewall (ZBF) policies and assess the mitigation effectiveness.
5.0	**Evaluation & Reporting**			
5.1		Comparative Analysis & Tuning	1 week	Analyse data to compare resource overhead and efficacy of CoPP vs. ZB.
5.2		Draft Technical Report	2 weeks	Draft the final chapters, focusing on the Implementation and Evaluation findings.

5.3		Finalise Deliverables and Report	2 weeks	Complete the final technical review, proofreading, and presentation preparation.
		Total Estimated Duration	**27 weeks**	**(Strict Adherence)**

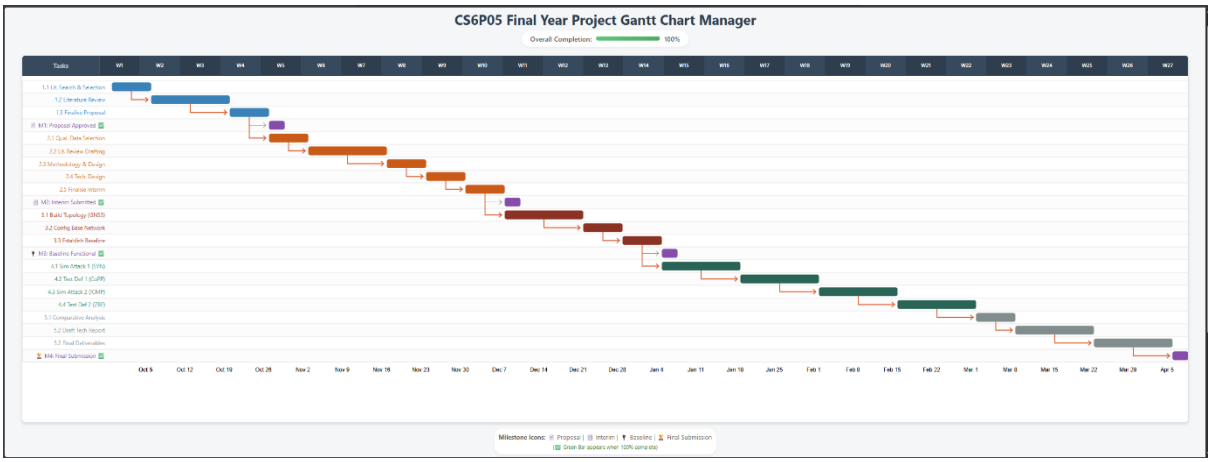


Figure 90 - Actual Project Gantt Chart Diagram

Appendix 2: Practical

This appendix provides the complete set of command-line interface (CLI) commands used throughout the project. It covers every major stage: provisioning the infrastructure, configuring the network fabric, executing the attacks, deploying the security mitigations and amendments to the project.

2.1 Amendments to the Original Plan

The project followed an Agile Scrum methodology, which enabled timely and effective adaptations to the original technical plan. These changes were essential to maintain the validity, performance, and security of the research environment. The main amendments included:

- **Migration to a Dedicated Server**: The simulation was moved from a local desktop hypervisor to a Hetzner bare-metal cloud server. This eliminated local hardware bottlenecks and hypervisor overload during intensive DoS attacks, ensuring that all measured performance degradation was purely due to network configuration.
- **Integration of Zero-Trust Security**: Hosting the lab in the cloud exposed the GNS3 management plane. To address this risk, Tailscale was deployed to create a secure, encrypted peer-to-peer tunnel. The server's public firewall was then configured to block all external connections.
- **Architectural Shift**: The original 802.1Q Router-on-a-Stick design was replaced with a Centralised Core Routing architecture. This change became necessary to overcome hypervisor compatibility issues that prevented reliable inter-VLAN encapsulation.
- **Simplification of Telemetry**: The out-of-band Monitor Station and Wireshark packet captures were removed. Instead, the project relied on more efficient native sources iperf3 endpoint metrics and direct Cisco IOS CLI logging of CPU usage and drop counters.
- **Iterative Configuration Tuning**: Agile sprints allowed continuous refinement of virtual machine RAM allocations and Layer 2 switching commands based on real-time diagnostic feedback from the simulation.

2.2 Practical Configuration Commands

This section details the exact commands used to access the Hetzner Rescue System and perform a clean installation of the Ubuntu host operating system on the dedicated bare-metal server.

```
# Connect to Hetzner Rescue System via SSH
ssh root@<hetzner_public_ip>
# Launch the OS installation utility
installimage
# Reboot the system to initialize the new OS
Reboot
```

This section lists the exact commands used on the Hetzner server to install and configure the Zero-Trust VPN (Tailscale) and set up secure remote SSH access.

```
# Download and execute the Tailscale installation script
curl -fsSL https://tailscale.com/install.sh | sh
# Authenticate and bind the server to the Tailscale network
tailscale up
# Enable SSH access exclusively over the Tailscale tunnel
tailscale up --ssh
# Connect to the server from the local machine using the private Tailscale
IP
ssh root@100.80.237.20
```

Commands executed on the Ubuntu server to install the GNS3 app and configure remote access.

```
# Navigate to the temporary directory
cd /tmp
# Download the GNS3 remote installation script
curl https://raw.githubusercontent.com/GNS3/gns3-
server/master/scripts/remote-install.sh > gns3-remote-install.sh
# Execute the script with Cisco IOU and 32-bit repository support
bash gns3-remote-install.sh --with-iou --with-i386-repository
# Open the GNS3 server configuration file to enable authentication and
Tailscale IP binding
```

```
nano /etc/gns3/gns3_server.conf
# Verify the installed GNS3 server version to match the local GUI client
apt policy gns3-server
```

Commands used to establish the Hierarchical Three-Tier architecture, VLANs, and Spanning Tree Protocol (Rapid PVST+).

Core-Switch (Primary L3 Nexus & STP Root)

```
enable
configure terminal
hostname Core-Switch
ip routing
vlan 10,20,100
exit
spanning-tree vlan 10,20,100 root primary
interface GigabitEthernet0/0
 switchport mode access
 switchport access vlan 100
 no shutdown
exit
interface Vlan100
 ip address 192.168.100.2 255.255.255.0
 no shutdown
exit
interface Vlan10
 ip address 10.1.10.1 255.255.255.0
 no shutdown
exit
interface Vlan20
 ip address 10.1.20.1 255.255.255.0
 no shutdown
exit
ip route 0.0.0.0 0.0.0.0 192.168.100.1
interface range GigabitEthernet0/1 - 2
 switchport trunk encapsulation dot1q
 switchport mode trunk
```

```
 no shutdown
exit
write memory
```

DS-1 (Distribution Switch 1 - STP Secondary Root)

```
enable
configure terminal
hostname DS-1
vlan 10,20,100
exit
spanning-tree vlan 10,20,100 root secondary
interface range GigabitEthernet0/0 - 1
 switchport trunk encapsulation dot1q
 switchport mode trunk
 no shutdown
exit
write memory
```

Access-Switch (Endpoint Segmentation)

```
enable
configure terminal
hostname Access-Switch
vlan 10,20
exit
interface range GigabitEthernet0/0 - 1
 switchport trunk encapsulation dot1q
 switchport mode trunk
 no shutdown
exit
interface GigabitEthernet0/2
 switchport mode access
 switchport access vlan 10
 spanning-tree portfast edge
 no shutdown
exit
interface GigabitEthernet0/3
```

```
 switchport mode access
 switchport access vlan 20
 spanning-tree portfast edge
 no shutdown
exit
write memory
```

Layer 3 Edge-Router Configuration

Commands executed to configure the WAN boundary and establish static routing to the Core-Switch.

```
enable
configure terminal
hostname Edge-Router
interface GigabitEthernet1
 description EXTERNAL_UNTRUSTED_ZONE
 ip address 192.168.10.1 255.255.255.0
 no shutdown
exit
interface GigabitEthernet2
 description TRANSIT_TO_CORE
 ip address 192.168.100.1 255.255.255.0
 no shutdown
exit
ip route 10.1.0.0 255.255.0.0 192.168.100.2
write memory
```

Linux Endpoints

Commands executed to assign static IP addresses and default gateways to the virtual appliances.

Kali Attacker Node (iproute2)

```
sudo ip addr add 192.168.10.100/24 dev eth0
sudo ip link set eth0 up
sudo ip route add default via 192.168.10.1
```

Ubuntu User-Node & Target-Server (Netplan via Here-Doc)

Note: IP addresses shown below have been adjusted for each node: 10.1.10.10 for the User Node and 10.1.20.10 for the Target Server.

```
sudo tee /etc/netplan/01-netcfg.yaml > /dev/null <<EOF
network:
  version: 2
  renderer: networkd
  ethernets:
    ens3:
      dhcp4: no
      addresses:
        - 10.1.10.10/24
      routes:
        - to: default
          via: 10.1.10.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
EOF

sudo chmod 600 /etc/netplan/01-netcfg.yaml
sudo netplan apply
```

Baseline Benchmarking and Diagnostics

Commands used to verify topology health, latency, and throughput capabilities.

```
# Network Diagnostics (Cisco IOS)
show spanning-tree vlan 10-20,100
show ip interface brief
show ip route

# ICMP Latency Test (Linux Endpoint)
ping -c 5 10.1.20.10

# TCP Throughput Baseline (Linux Endpoints)
iperf3 -s                    # Executed on Target-Server (Listener)
iperf3 -c 10.1.20.10 -t 10   # Executed on User-Node (Client)
```

DoS Threat Emulation (Attacker Node)

Commands executed via hping3 to generate volumetric floods.

```
# Volumetric ICMP Flood (Targeting the Control Plane)
sudo hping3 --icmp --flood 192.168.10.1

# Volumetric TCP SYN Flood (Targeting the Data Plane)
sudo hping3 -S --flood -p 5201 10.1.20.10
```

Infrastructure Telemetry Monitoring

Commands executed on the Edge-Router during active attacks to measure resource exhaustion.

```
# Measure real-time Control Plane utilisation

show processes cpu sorted | exclude 0.00

# Identify Data Plane egress queue exhaustion

show interfaces gi1 | include drops
```

Defensive Countermeasures (Mitigation Deployment)

Commands executed to implement stateful and stateless defensive policies on the Edge-Router.

Control Plane Policing (CoPP - Stateless Mitigation)

```
configure terminal
access-list 100 permit icmp any any
class-map match-all ICMP-CLASS
 match access-group 100
 exit
policy-map COPP-POLICY
 class ICMP-CLASS
  police 8000 conform-action transmit exceed-action drop
  exit
 exit
control-plane
 service-policy input COPP-POLICY
 exit
write memory
# Verification of CoPP policy drops
show policy-map control-plane
```

Zone-Based Policy Firewall (ZBF - Stateful Mitigation)

```
configure terminal
zone security OUTSIDE
zone security INSIDE
exit
interface GigabitEthernet1
 zone-member security OUTSIDE
exit
interface GigabitEthernet2
 zone-member security INSIDE
exit
write memory
```

References

AL-Musawi, B. Q. M. 2012. Mitigating DoS/DDoS Attacks Using Iptables. *International Journal of Engineering & Technology*, vol. 12, no. 3, pp. 101–106.

Alturki, R. (2021) 'Research Onion for Smart IoT-Enabled Mobile Applications', *Scientific Programming*, 2021(1), pp. 1–9. Available at: https://doi.org/10.1155/2021/4270998.

Andrews, L. (2023). *Enhancing Security Through Virtualization Technology*. [online] www.franklinfitch.com. Available at: https://www.franklinfitch.com/uk/resources/blog/enhancing-security-through-virtualization-technology/.

Anthony, P. and Wilkins, S. (2017) *Cisco CCNA in 60 Days: Exam 100-105, Exam 200-105, Exam 200-125*. Independently published.

Balyk, Anatoliy, et al. 'USING GRAPHIC NETWORK SIMULATOR 3 FOR DDOS ATTACKS SIMULATION'. *International Journal of Computing*, Dec. 2017, pp. 219–25. *DOI.org (Crossref)*, https://doi.org/10.47839/ijc.16.4.910.

Barrett, D.J. and Silverman, R.E. (2001) *SSH, the Secure Shell: The Definitive Guide*. First Edition. Edited by M. Loukides. O'Reilly & Associates, Inc.

Bekravi, M., Jamali, S. and Shaker, G. 2012. Defense against SYN-Flood Denial of Service Attacks Based on Learning Automata. *Computers and Mathematics with Applications*, vol. 63, no. 1, pp. 214–221.

Benjie (2022) *Zone-Based Firewall Cisco Configuration and Verification*, *Study CCNP*. Available at: https://study-ccnp.com/zone-based-firewall-cisco-configuration-verification/.

Bogdanoski, M., Shuminoski, T. and Risteski, A. 2013. Analysis of the SYN Flood DoS Attack. *I. J. Computer Network and Information Security*, vol. 8, pp. 1–11.

Bugnion, E. *et al.* (2012) 'Bringing Virtualization to the x86 Architecture with the Original VMware Workstation', *ACM Transactions on Computer Systems*, 30(4), pp. 1–51. Available at: https://doi.org/10.1145/2382553.2382554.

Canonical Ltd. (2015). *Certified Ubuntu Cloud Guest: The best of Ubuntu on the best clouds*. Retrieved May 1, 2026, from https://ubuntu.cloud/wp-content/uploads/2016/07/278f/eBook_Certified-Ubuntu-Cloud-Guest_2015.pdf

Canonical Ltd. (n.d.a). *Ubuntu Cloud Guest*. Retrieved May 1, 2026, from https://insights.ubuntu.com/wp-content/uploads/226b/Ubuntu-Cloud-Guest.pdf

Canonical Ltd. (n.d.b). *Ubuntu Public Cloud Documentation: Ubuntu base and minimal images*. Retrieved May 1, 2026, from https://documentation.ubuntu.com/public-cloud/all-clouds-explanation/ubuntu-base-and-minimal-images/

Canonical Ltd. (n.d.c). *Ubuntu Cloud Images Repository*. Retrieved May 1, 2026, from https://cloud-images.ubuntu.com/

Chowdappa, K.B., Huq, S.Z.U. and Begum, S.S. (2023) 'Unveiling Cyber Threats: A Comprehensive Analysis of Wireshark as a Vital Cybersecurity Tool', *Atlantis Press*, 10(4), pp. 358-365.

Cisco Community. 2025. CoPP (Control Plane Policing). Online Resource.

Cisco Developer. (2024) *Cisco Modeling Labs: IOSvL2*. Available at: https://developer.cisco.com/docs/modeling-labs/iosvl2/#features-tested-with-cml (Accessed: 11 March 2026).

Cisco Systems (2010) *SBA for Government: Large Borderless Networks - Collapsed Data Center and Campus Core Deployment Guide*. San Jose: Cisco Systems, Inc.

Cisco. (2020) *Cisco CSR 1000v Series Cloud Services Router Software Configuration Guide*. Available at: https://www.cisco.com/c/en/us/td/docs/routers/csr1000/software/configuration/b_CSR1000v_Configuration_Guide/b_CSR1000v_Configuration_Guide_chapter_00.html (Accessed: 11 March 2026).

Cisco. (2025). *Security - Configuring Control Plane Policing (CoPP) [Support]*. [online] Available at: https://www.cisco.com/en/US/docs/general/Test/dwerblo/broken_guide/copp.html.

Collapsed Core and Three-Tier Network Architectures (2021) *Study CCNA*. Available at: https://study-ccna.com/collapsed-core-and-three-tier-architectures/.

Dagenhardt, F., Moreno, J. and Dufresne, B., 2018. *Deploying ACI: The complete guide to planning, configuring, and managing Application Centric Infrastructure.* Cisco Press.

Dawson, M. (2015) 'A study of Distributed Denial of Service (DDoS) Attacks and Mitigation Strategies', *Journal of Information Security*, 6(3), pp. 230-245.

Diskinternals.com. (2024). *RAID 1 and Disk Mirroring - What is it?* [online] Available at: https://www.diskinternals.com/raid-recovery/what-is-disk-mirroring-raid-1/.

Eddy, W. M. 2006. Defenses Against TCP SYN Flooding Attacks. *The Internet Protocol Journal*, vol. 9, no. 4, pp. 2–16.

Forniet. (2025). *How CPU and Memory Utilization is Collected for Cisco IOS | External Systems Configuration Guide*. [online] Fortinet.com. Available at: https://docs.fortinet.com/document/fortisiem/6.7.0/external-systems-configuration-guide/852773/how-cpu-and-memory-utilization-is-collected-for-cisco-ios [Accessed 14 Dec. 2025].

Garcés, A., et al. (2017) 'A systematic mapping study of infrastructure as code research', *Information and Software Technology*, 91, pp. 1–17.

GeeksforGeeks (2019). *ip Command in Linux with Examples*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/linux-unix/ip-command-in-linux-with-examples/.

Global Journal of Computer Science and Technology (GJCST) (2020) 'Managing Requirements Volatility in Agile Software Development', *Global Journal of Computer Science and Technology*, 20(3), pp. 15-24.

GNS3 (2026). *GNS3 server configuration file | GNS3 Documentation*. [online] Gns3.com. Available at: https://docs.gns3.com/docs/using-gns3/administration/gns3-server-configuration-file/ [Accessed 6 Apr. 2026].

Goldschmidt, P. and Kučera, J. 2021. Defense Against SYN Flood DoS Attacks Using Network-based Mitigation Techniques. In *2021 IFIP/IEEE Int. Symp. Integr. Netw. Manag. (IM)*, pp. 115–123.

Gupta, M., Chowdary, M.N., Bussa, S. and Kumar, C. (2021) 'Deploying H[adoop cluster with automation?]', in *2021 5th International Conference on Information Systems and Computer Networks (ISCON)*. Mathura, India: GLA University, pp. 1-6.

Hammersley, M. (2013) *What is Qualitative Research?* London: Bloomsbury Academic.

Harshita. 2017. Detection and Prevention of ICMP Flood DDOS Attack. *International Journal of New Technology and Research (IJNTR)*, vol. 3, no. 3, pp. 63–69.

Hartpence, B. (2015) *CCNA Security 210-260 Official Cert Guide*. Indianapolis: Cisco Press.

Hetzner.com. (2025). *Server Auction FAQs - Hetzner Docs*. [online] Available at: https://docs.hetzner.com/robot/general/server-auction-faqs/ [Accessed 25 Mar. 2026].

Hrițcan, D.-F. and Balan, D. (2024) 'Using Tailscale and PfSense for Security and Anonymity of IoT Environments', *2024 IEEE International Conference on Advanced Trends in Information Theory (ATIT)*. IEEE, pp. 1-6.

Hrițcan, D.F. and Balan, D., 2024, May. Using tailscale and PfSense for security and anonymity of iot environments. In *2024 International Conference on Development and Application Systems (DAS)* (pp. 91-94). IEEE.

Hrițcan, D.-F., Graur, A., Bălan, D.-G. and Timofte, E.-M. (2025) 'A Performance Analysis of VPN Technologies Used in an IoT Environment', *Advances in Electrical and Computer Engineering*, 25(2), pp. 81-88.

IBM (2025) *Understanding Control Plane Policing (CoPP) and Network Security*. Available at: https://www.ibm.com/security (Accessed: 4 May 2026).

Install GNS3 on a remote server | GNS3 Documentation (2017) *Gns3.com*. Available at: https://docs.gns3.com/docs/getting-started/installation/remote-server/ (Accessed: 1 April 2026).

Integrate UK (2025) *Implementing Zone-Based Firewalls in Cisco Architectures*. Available at: https://www.integrate.co.uk/resources (Accessed: 4 May 2026).

Integrate UK. 2025. IOS-XE Zone-Based Firewall (ZBFW). Online Resource.

Jalloul, S. (2012) *Denial of Service Attacks: Threats and Countermeasures*. New York: Tech Press.

Jamali, S. and Shaker, G. 2012. PSO-SFDD: Defense against SYN flooding DoS attacks by employing PSO algorithm. *Computers and Mathematics with Applications*, vol. 63, no. 1, pp. 214–221.

Johnston, M.P., (2014) Secondary data analysis: 'A method of which the time has come'. *Qualitative and quantitative methods in libraries*, *3*(3), pp.619-626.

Jokste, L. and Kampars, J., 2024, October. Comprehensible software architecture visualization with C4 model for WRM ecosystem members. In *2024 IEEE 65th International Scientific Conference on Information Technology and Management Science of Riga Technical University (ITMS)* (pp. 1-5). IEEE.

Joseph, R., Smith, T. and Doe, J. (2023) *Penetration Testing with Kali Linux: Advanced Threat Emulation*. London: CyberSec Publishing.

Juniper Networks (2004) *Stateful Inspection Firewalls: An overview of firewall technology and how Juniper Networks implements it*. Sunnyvale, CA: Juniper Networks, Inc.

Juniper Networks. 2025. Understanding Session Table Flood Attacks. Technical Documentation.

Keysight. 2025. What metrics can synthetic traffic generators measure? Technical Documentation.

Khalaf, B. A. *et al*. 2025. A Comparative Simulation Study of SYN Flood and HTTP Flood Distributed Denial-of-Service (DDoS) Attacks in Cloud Environments. *International Journal of Research and Analytical Reviews*, vol. 9, no. 10, pp. 8328–8346.

Khalaf, B. A., Mostafa, S. A., Mustapha, A., Ismail, A., Mahmoud, M. A., Jubair, M. A., and Hassan, M. H. 2019. A Simulation Study of Syn Flood Attack In Cloud Computing Environment. *Revista AUS*, vol. 26, no. 1, pp. 187–198.

Liu, C., Du, D., Zhang, C., Peng, C. and Fei, M. 2023. Observability Analysis of Networked Control Systems Under DoS Attacks. In *IECON 2023—49th Annual Conference of the IEEE Industrial Electronics Society*, pp. 1–6.

Markets&Markets. 2022. *DDoS Protection and Mitigation Market Size, Share & Trends Analysis Report*, 2022-2027.

Moldstud (2024) *Essential Tools for Measuring Network Throughput - Boost Your Performance Analysis*. Available at: https://moldstud.com/articles/p-essential-tools-for-measuring-network-throughput-boost-your-performance-analysis (Accessed: 4 May 2026).

Netplan. (2026). *NETPLAN-IP*. [online] Available at: https://netplan.readthedocs.io/en/stable/netplan-ip/ [Accessed 1 May 2026].

Network Academy (2025) *Hierarchical Network Design and Configuration*. Available at: https://www.netacad.com (Accessed: 4 May 2026).

Nikitin, A. (2025) *Agile Methodology: Iterative Development for Modern Software*. Available at: https://www.agilealliance.org (Accessed: 4 May 2026).

Odom, W. (2020a) *CCNA 200-301 Official Cert Guide, Volume 1*. Indianapolis: Cisco Press.

Paguigan, J.S. and Albino, M.G. (2022) 'An Assessment and Design of Campus Network Using Collapsed-Core Architecture', *2022 IEEE 10th International Conference on Information, Communication and Networks (ICICN)*. IEEE, pp. 526-533.

Palo Alto Networks (2015) *Understanding Protocol Attacks and Denial of Service*. Available at: https://www.paloaltonetworks.com/cyberpedia (Accessed: 4 May 2026).

Palo Alto Networks (2019) *What is a denial of service attack (DoS) ? - Palo Alto Networks*, *Paloaltonetworks.com*. Available at: https://www.paloaltonetworks.com/cyberpedia/what-is-a-denial-of-service-attack-dos.

Paquet, C. 2013. *Implementing Cisco IOS Network Security: (IINS 640-554) Foundation Learning Guide*. Indianapolis, Indiana, USA: Cisco Press.

Peng, T., Leckie, C., and Ramamohanarao, K. 2007. Survey of Network-Based Defense Mechanisms Countering the DoS and DDoS Problems. *ACM Computing Surveys*, vol. 39, no. 1, pp. 1–42.

QEMU. (2024) *About QEMU*. Available at: https://www.qemu.org/docs/master/about/index.html (Accessed: 11 March 2026).

Rodgi, M. and Bhirud, S. (2024) 'The Agile Manifesto and its Implementation in Network Systems', *International Journal of Computer Applications*, 24(2), pp. 45-52.

Safa, H., Chouman, M., Artail, H., and Karam, M. 2008. A collaborative defense mechanism against SYN flooding attacks in IP networks. *Journal of Network and Computer Applications*, vol. 31, no. 4, pp. 509–534.

Sahay, A. (2016) 'Peeling Saunders's Research Onion', *Research Methods in Management*, 14(1), pp. 34-45.

Santos, O. 2020. *Cisco CyberOps Associate CBROPS 200-201 Official Cert Guide*. Cisco Press.

Shaikh, S. A., Kadam, V. and Jha, B. 2019. Comparing Context-Based Access Control to Zone-based Policy Firewalls. *International Journal of Research in Applied Science and Engineering Technology (IJRASET)*, vol. 7, no. 6, pp. 1475–1479.

Shea, R. and Liu, J., 2012, June. *Understanding the impact of denial of service attacks on virtual machines*. In 2012 IEEE 20th International Workshop on Quality of Service (pp. 1-9). IEEE.

Shroff, A., Ma, R. and Chang, L. 2022. Investigating DDoS Attack Mitigation Strategies and Simulation Tools Using GNS3. *International Journal of Research in Applied Science and Engineering Technology (IJRASET)*.

Stigasoft (2025) *Agile Methodology*. Available at: https://www.stigasoft.com/agile-scrum-methodology.html (Accessed: 5 November 2025).

Stübler, T., Amodei, A., Capriglione, D., Tomasso, G., Bonnotte, N. and Mohammed, S. 2024. An Investigation of Denial of Service Attacks on Autonomous Driving Software and Hardware in Operation. *arXiv:2409.01324v1*.

Study-CCNP (2025) *Cisco IOS-XE Zone-Based Policy Firewall Guide*. Available at: https://www.study-ccnp.com/zbf (Accessed: 4 May 2026).

Tailscale. (2024) *What is Tailscale?*. Available at: https://tailscale.com/docs/concepts/what-is-tailscale (Accessed: 11 March 2026).

Tailscale. (2026). *What firewall ports should I open to use Tailscale? · Tailscale Docs*. [online] Available at: https://tailscale.com/docs/reference/faq/firewall-ports [Accessed 3 Apr. 2026].

Tiso, J. (2012) *Designing Cisco Network Service Architectures (ARCH) Foundation Learning Guide*. 3rd edn. Indianapolis: Cisco Press.

Ubuntu (2024). *Introducing Netplan v1.0 – stable, declarative network management*. Retrieved May 1, 2026, from https://ubuntu.com/blog/introducing-netplan-v1

Zargar, S. T., Joshi, J. and Tipper, D. 2013. A Survey of Defense Mechanisms Against Distributed Denial of Service (DDoS) Flooding Attacks. *IEEE Communications Surveys & Tutorials*, vol. 15, no. 4, pp. 2046–2069.

Zhijun, W., Wenjing, L., Liang, L. and Meng, Y. 2020. Low-Rate DoS Attacks, Detection, Defense, and Challenges: A Survey. *IEEE Access*, vol. 8, pp. 43920–43943.